

ZPL II

Programming Guide

Volume One

© 2005 ZIH Corp.

The copyrights in this manual and the label print engine described therein are owned by Zebra Technologies Corporation. Unauthorized reproduction of this manual or the software in the label print engine may result in imprisonment of up to one year and fines of up to $10,000 (17 U.S.C.506). Copyright violators may be subject to civil liability.

This product may contain ZPL®, ZPL II®, and ZebraLink™ programs; Element Energy Equalizer® Circuit; E3®; and AGFA fonts. Software © ZIH Corp. All rights reserved worldwide.

ZebraLink and all product names and numbers are trademarks, and Zebra, the Zebra logo, ZPL, ZPL II, Element Energy Equalizer Circuit, and E3 Circuit are registered trademarks of ZIH Corp. All rights reserved worldwide.

CG Triumvirate is a trademark of AGFA Monotype Corporation. All rights reserved worldwide. CG Triumvirate™ font © AGFA Monotype Corporation. Intellifont® portion © AGFA Monotype Corporation. All rights reserved worldwide. UFST is a registered trademark of AGFA Monotype Corporation. All rights reserved worldwide.

All other brand names, product names, or trademarks belong to their respective holders.

Customer Order # 45541L

Manufacturer Part # 45541L-001 Rev. A

Contents

Contents i

Proprietary Statement xvii

Preface xix

Who Should Use This Document xx
How This Document Is Organized xx
Contacts xxi
Support xxi
Document Conventions. xxii
Related Documents xxiii

1 • Introduction 1

2 • ZPL Commands 3

Basic ZPL Exercises 5
Before you begin. 6
^A Scalable/Bitmapped Font 12
^A@ Use Font Name to Call Font 15
^B1 Code 11 Bar Code 17
^B2 Interleaved 2 of 5 Bar Code 19
^B3 Code 39 Bar Code 21
^B4 Code 49 Bar Code 25
^B5 Planet Code bar code 29
^B7 PDF417 Bar Code 30
^B8 EAN-8 Bar Code 35
^B9 UPC-E Bar Code 37
^BA Code 93 Bar Code 39

^BB CODABLOCK Bar Code 43
^BC Code 128 Bar Code (Subsets A, B, and C) 47
^BD UPS MaxiCode Bar Code 60
^BE EAN-13 Bar Code 63
^BF Micro-PDF417 Bar Code 65
^BI Industrial 2 of 5 Bar Codes 67
^BJ Standard 2 of 5 Bar Code 69
^BK ANSI Codabar Bar Code 71
^BL LOGMARS Bar Code 73
^BM MSI Bar Code 75
^B0 Aztec Bar Code Parameters 77
^BP Plessey Bar Code 79
^BQ QR Code Bar Code 81
^BR RSS Bar Code 88
^BS UPC/EAN Extensions 90
^BT TLC39 bar code 93
^BU UPC-A Bar Code 95
^BX Data Matrix Bar Code 97
^BY Bar Code Field Default 101
^BZ POSTNET Bar Code 103
^CC Change Carets 105
~CC Change Carets 106
^CD ~CD Change Delimiter 107
^CF Change Alphanumeric Default Font 108
^CI Change International Font 110
^CM Change Memory Letter Designation 113
^CO Cache On 114
^CT ~CT Change Tilde 116
^CV Code Validation 117
^CW Font Identifier 119
~DB Download Bitmap Font 121
~DE Download Encoding 123
^DF Download Format 124
~DG Download Graphics 125
~DN Abort Download Graphic 127
~DS Download Scalable Font 128
~DT Download TrueType Font 129
~DU Download Unbounded TrueType Font 130
~DY Download Graphics 131

~EF Erase Stored Formats 132
~EG Erase Download Graphics 133
^FB Field Block 134
^FC Field Clock (for Real-Time Clock) 136
^FD Field Data 137
^FH Field Hexadecimal Indicator 138
^FM Multiple Field Origin Locations 139
^FN Field Number 141
^FO Field Origin 142
^FP Field Parameter 143
^FR Field Reverse Print 144
^FS Field Separator 145
^FT Field Typeset 146
^FV Field Variable 148
^FW Field Orientation 149
^FX Comment 150
^GB Graphic Box 151
^GC Graphic Circle 153
^GD Graphic Diagonal Line 154
^GE Graphic Ellipse 155
^GF Graphic Field 156
^GS Graphic Symbol 158
~HB Battery Status 160
~HD Head Temperature Information 161
^HF Graphic Symbol 162
^HG Host Graphic 163
^HH Configuration Label Return 164
~HI Host Identification 165
~HM Host RAM Status 166
~HS Host Status Return 167
~HU Return ZebraNet Alert Configuration 170
^HV Host Verification 171
^HW Host Directory List 172
^HY Upload Graphics 174
^HZ Display Description Information 175
^ID Object Delete 177
^IL Image Load 179
^IM Image Move 181
^IS Image Save 182

~JA Cancel All 184
^JB Initialize Flash Memory 185
~JB Reset Optional Memory 186
~JC Set Media Sensor Calibration 187
~JD Enable Communications Diagnostics 188
~JE Disable Diagnostics 189
~JF Set Battery Condition 190
~JG Graphing Sensor Calibration 191
^JI Start ZBI (Zebra BASIC Interpreter) 192
~JI Start ZBI (Zebra BASIC Interpreter) 193
^JJ Set Auxiliary Port 194
~JL Set Label Length 196
^JM Set Dots per Millimeter 197
~JN Head Test Fatal 198
~JO Head Test Non fatal 199
~JP Pause and Cancel Format 200
~JQ Terminate Zebra BASIC Interpreter 201
~JR Power On Reset 202
^JS Sensor Select 203
~JS Change Backfeed Sequence 204
^JT Head Test Interval 206
^JU Configuration Update 207
^JW Set Ribbon Tension 208
~JX Cancel Current Partially Input Format 209
^JZ Reprint After Error 210
~KB Kill Battery (Battery Discharge Mode) 211
^KD Select Date and Time Format (for Real Time Clock) 212
^KL Define Language 213
^KN Define Printer Name 214
^KP Define Password 216
^LH Label Home 217
^LL Label Length 218
^LR Label Reverse Print 219
^LS Label Shift 220
^LT Label Top 221
^MC Map Clear 222
^MD Media Darkness 223
^MF Media Feed 224
^ML Maximum Label Length 225

^MM Print Mode . . . 226
^MN Media Tracking . . . 227
^MP Mode Protection . . . 228
^MT Media Type . . . 229
^MU Set Units of Measurement . . . 230
^MW Modify Head Cold Warning . . . 232
~NC Network Connect . . . 233
^NI Network ID Number . . . 234
~NR Set All Network Printers Transparent . . . 235
^NS Change Networking Settings . . . 236
~NT Set Currently Connected Printer Transparent . . . 237
^PF Slew Given Number of Dot Rows . . . 238
^PH ~PH Slew to Home Position . . . 239
^PM Printing Mirror Image of Label . . . 240
^PO Print Orientation . . . 241
^PP ~PP Programmable Pause . . . 242
^PQ Print Quantity . . . 243
^PR Print Rate . . . 244
~PR Applicator Reprint . . . 247
~PS Print Start . . . 248
^PW Print Width . . . 249
~RO Reset Advanced Counter . . . 250
^SC Set Serial Communications . . . 251
~SD Set Darkness . . . 252
^SE Select Encoding . . . 253
^SF Serialization Field (with a Standard ^FD String) . . . 254
^SL Set Mode and Language (for Real-Time Clock) . . . 256
^SN Serialization Data . . . 257
^SO Set Offset (for Real-Time Clock) . . . 259
^SP Start Print . . . 260
^SQ Halt ZebraNet Alert . . . 262
^SR Set Printhead Resistance . . . 263
^SS Set Media Sensors . . . 264
^ST Set Date and Time (for Real-Time Clock) . . . 266
^SX Set ZebraNet Alert . . . 267
^SZ Set ZPL . . . 270
~TA Tear-off Adjust Position . . . 271
^TO Transfer Object . . . 272
~WC Print Configuration Label . . . 274

^WD Print Directory Label 275
^XA Start Format 277
^XB Suppress Backfeed 278
^XF Recall Format 279
^XG Recall Graphic 280
^XZ End Format 281
^ZZ Printer Sleep 282

3 • ZBI Commands 283

AND 285
Arrays 285
AUTONUM 285
Boolean Expression 286
BREAK 286
Channels. 286
CLOSE 287
CLRERR 287
CTRL-C. 287
DEBUG 287
Declaration 288
DECLARE. 288
DELETE 288
DIR 289
DO-LOOP 289
ECHO 291
END 291
! (EXCLAMATION MARK) 291
EXIT 291
FOR-LOOP 292
Functions 292
Integer Functions 293
String Functions 297
GOTO 301
GOSUB-RETURN. 301
IF Statements 302
INBYTE. 303
INPUT 303
LET 304
LIST 305
LOAD 305
NEW 306
NOT 306
Numeric Expressions 306
ON ERROR 308
Open 308
OR 309
OUTBYTE. 309
Ports <CHANNEL EXPRESSION>. 310
PRINT 310

REM 311
RENUM. 311
RESTART 312
RUN 312
SEARCHTO$ (A,B$). 313
SEARCHTO$ (A,B$,C) 313
SETERR 314
SLEEP 314
STEP 315
STORE 315
STRING CONCATENATION (&). 316
STRING VARIABLE 316
UB-STRINGS 317
TRACE 318

4 • ZBI Tutorial 319

Starting ZBI 320
ZBI Interpreter Modes. 320
Basic ZBI Examples 323

A • RFID Commands 333

RFID Command Overview 334
^HR Calibrate RFID Transponder Position 335
^RB Define EPC Data Structure 338
^RF Read or Write RFID Format 341
^RM Enable RFID Motion 343
^RN Detect Multiple RFID Tags in Encoding Field 344
~RO Reset Advanced Counters 345
^RR Specify RFID Retries for a Block 347
^RS RFID Setup 348
^RT Read RFID Tag 352
^RW Set RFID Read and Write Power Levels 354
^RZ Set RFID Tag Password 355
^WT Write Tag 356
^WV Verify RFID Write Operation 358

B • Wireless Print Server Commands 359

^KP Define Printer Password 360
^NB Search for Wired Print Server during Network Boot 361
^NN Set SNMP 362
^NP Set Primary/Secondary Device 363
^NS Change Wired Networking Settings 364

^NT Set SMTP 366
^NW Set Web Authentication Timeout Value 367
^WA Set Antenna Parameters 368
^WE Set Wireless Encryption Values 369
^WI Change Wireless Network Settings 371
^WL Set LEAP Parameters 372
~WL Print Network Configuration Label 373
^WP Set Wireless Password 374
^WR Set Transmit Rate 375
~WR Reset Wireless Card 376
^WS Set Wireless Card Values 377

Index 379

Contents

Functional Table of Contents

Abort Download Graphic 127
ANSI Codabar Bar Code 71
Applicator Reprint 247
Aztec Bar Code Parameters 77
Bar Code Field Default 101
Battery Status 160
Cache On 114
Calibrate RFID Transponder Position 335
Cancel All 184
Cancel Current Partially Input Format 209
Change Alphanumeric Default Font 108
Change Backfeed Sequence 204
Change Carets 105
Change Carets 106
Change Delimiter 107
Change International Font 110
Change Memory Letter Designation 113
Change Networking Settings 236
Change Tilde 116
Change Wired Networking Settings 364
Change Wireless Network Settings 371
CODABLOCK Bar Code 43
Code 11 Bar Code 17
Code 128 Bar Code (Subsets A, B, and C) 47
Code 39 Bar Code 21
Code 49 Bar Code 25
Code 93 Bar Code 39
Code Validation 117

Comment 150
Configuration Label Return 164
Configuration Update 207
Data Matrix Bar Code 97
Define EPC Data Structure 338
Define Language 213
Define Password 216
Define Printer Name 214
Define Printer Password 360
Detect Multiple RFID Tags in Encoding Field 344
Disable Diagnostics 189
Display Description Information 175
Download Bitmap Font 121
Download Encoding 123
Download Format 124
Download Graphics 125
Download Graphics 131
Download Scalable Font 128
Download TrueType Font 129
Download Unbounded TrueType Font 130
EAN-13 Bar Code 63
EAN-8 Bar Code 35
Enable Communications Diagnostics 188
Enable RFID Motion 343
End Format 281
Erase Download Graphics 133
Erase Stored Formats 132
Field Block 134
Field Clock (for Real-Time Clock) 136
Field Data 137
Field Hexadecimal Indicator 138
Field Number 141
Field Orientation 149
Field Origin 142
Field Parameter 143
Field Reverse Print 144
Field Separator 145
Field Typeset 146
Field Variable 148
Font Identifier 119
Graphic Box 151
Graphic Circle 153
Graphic Diagonal Line 154

Graphic Ellipse 155
Graphic Field 156
Graphic Symbol 158
Graphic Symbol 162
Graphing Sensor Calibration 191
Halt ZebraNet Alert 262
Head Temperature Information 161
Head Test Fatal 198
Head Test Interval 206
Head Test Non fatal 199
Host Directory List 172
Host Graphic 163
Host Identification 165
Host RAM Status 166
Host Status Return 167
Host Verification 171
Image Load 179
Image Move 181
Image Save 182
Industrial 2 of 5 Bar Codes 67
Initialize Flash Memory 185
Interleaved 2 of 5 Bar Code 19
Kill Battery (Battery Discharge Mode) 211
Label Home 217
Label Length 218
Label Reverse Print 219
Label Shift 220
Label Top 221
LOGMARS Bar Code 73
Map Clear 222
Maximum Label Length 225
Media Darkness 223
Media Feed 224
Media Tracking 227
Media Type 229
Micro-PDF417 Bar Code 65
Mode Protection 228
Modify Head Cold Warning 232
MSI Bar Code 75
Multiple Field Origin Locations 139
Network Connect 233
Network ID Number 234
Object Delete 177

Pause and Cancel Format 200
PDF417 Bar Code 30
Planet Code bar code 29
Plessey Bar Code 79
POSTNET Bar Code 103
Power On Reset 202
Print Configuration Label 274
Print Directory Label 275
Print Mode 226
Print Network Configuration Label 373
Print Orientation 241
Print Quantity 243
Print Rate 244
Print Start 248
Print Width 249
Printer Sleep 282
Printing Mirror Image of Label 240
Programmable Pause 242
QR Code Bar Code 81
Read or Write RFID Format 341
Read RFID Tag 352
Recall Format 279
Recall Graphic 280
Reprint After Error 210
Reset Advanced Counter 250
Reset Advanced Counters 345
Reset Optional Memory 186
Reset Wireless Card 376
Return ZebraNet Alert Configuration 170
RFID Setup 348
RSS Bar Code 88
Scalable/Bitmapped Font 12
Search for Wired Print Server during Network Boot 361
Select Date and Time Format (for Real Time Clock) 212
Select Encoding 253
Sensor Select 203
Serialization Data 257
Serialization Field (with a Standard ^FD String) 254
Set All Network Printers Transparent 235
Set Antenna Parameters 368
Set Auxiliary Port 194
Set Battery Condition 190
Set Currently Connected Printer Transparent 237

Set Darkness 252
Set Date and Time (for Real-Time Clock) 266
Set Dots per Millimeter 197
Set Label Length 196
Set LEAP Parameters 372
Set Media Sensor Calibration 187
Set Media Sensors 264
Set Mode and Language (for Real-Time Clock) 256
Set Offset (for Real-Time Clock) 259
Set Primary/Secondary Device 363
Set Printhead Resistance 263
Set RFID Read and Write Power Levels 354
Set RFID Tag Password 355
Set Ribbon Tension 208
Set Serial Communications 251
Set SMTP 366
Set SNMP 362
Set Transmit Rate 375
Set Units of Measurement 230
Set Web Authentication Timeout Value 367
Set Wireless Card Values 377
Set Wireless Encryption Values 369
Set Wireless Password 374
Set ZebraNet Alert 267
Set ZPL 270
Slew Given Number of Dot Rows 238
Slew to Home Position 239
Specify RFID Retries for a Block 347
Standard 2 of 5 Bar Code 69
Start Format 277
Start Print 260
Start ZBI (Zebra BASIC Interpreter) 192
Start ZBI (Zebra BASIC Interpreter) 193
Suppress Backfeed 278
Tear-off Adjust Position 271
Terminate Zebra BASIC Interpreter 201
TLC39 bar code 93
Transfer Object 272
UPC/EAN Extensions 90
UPC-A Bar Code 95
UPC-E Bar Code 37
Upload Graphics 174
UPS MaxiCode Bar Code 60

Use Font Name to Call Font 15
Verify RFID Write Operation 358
Write Tag 356

Proprietary Statement

This manual contains proprietary information of Zebra Technologies Corporation and its subsidiaries (“Zebra Technologies”). It is intended solely for the information and use of parties operating and maintaining the equipment described herein. Such proprietary information may not be used, reproduced, or disclosed to any other parties for any other purpose without the expressed written permission of Zebra Technologies.

Product Improvements

Continuous improvement of products is a policy of Zebra Technologies. All specifications and designs are subject to change without notice.

Liability Disclaimer

Zebra Technologies takes steps to ensure that its published Engineering specifications and manuals are correct; however, errors do occur. Zebra Technologies reserves the right to correct any such errors and disclaims liability resulting therefrom.

Limitation of Liability

In no event shall Zebra Technologies or anyone else involved in the creation, production, or delivery of the accompanying product (including hardware and software) be liable for any damages whatsoever (including, without limitation, consequential damages including loss of business profits, business interruption, or loss of business information) arising out of the use of, the results of use of, or inability to use such product, even if Zebra Technologies has been advised of the possibility of such damages. Some jurisdictions do not allow the exclusion or limitation of incidental or consequential damages, so the above limitation or exclusion may not apply to you.



Notes • _____

_____

_____

_____

_____

_____

_____

_____

_____

_____

Preface

This section provides you with contact information, document structure and organization, and additional reference documents.

Contents

Who Should Use This Document xx
How This Document Is Organized xx
Contacts xxi
Document Conventions xxii
Related Documents xxiii

Who Should Use This Document

This document is intended for use by any person who needs to operate or troubleshoot problems with the printer.

How This Document Is Organized

This document is set up as follows:

Section	Description
Introduction	Provides a high-level overview about this guide and Zebra Programming Language (ZPL).
ZPL Commands	Provides an alphabetical, detailed description of each ZPL command.
ZBI Commands	Provides an alphabetical, detailed description of Zebra Basic Interpreter (ZBI).
ZBI Tutorial	Provides hands-on experience using ZBI.
RFID Commands	Provides an alphabetical, detailed description of each ZPL RFID command, and some examples of how to use them.
Wireless Print Server Commands	Provides new and modified ZPL commands for the Wireless Print Server.

Contacts

You can contact Zebra Technologies at any of the following:

Visit us at: http://www.zebra.com

Our Mailing Addresses:
Zebra Technologies Corporation
333 Corporate Woods Parkway
Vernon Hills, Illinois 60061.3109 U.S.A
Telephone: +1 847.634.6700
Fax: +1 847.913.8766

Zebra Technologies Europe Limited
Zebra House
The Valley Centre, Gordon Road
High Wycombe
Buckinghamshire HP13 6EQ, UK
Telephone: +44 (0)1494 472872
Fax: +44 (0)1494 450103

Support

You can contact Zebra support at:

Web Address: www.zebra.com/SS/service_support.htm

US Phone Number +1 847.913.2259

UK/International Phone Number +44 (0) 1494 768289

Document Conventions

The following conventions are used throughout this document to convey certain information:

Alternate Color (online only) Cross-references contain links to other sections in this guide. If you are viewing this guide online, click the blue text to jump to its location.

Command Line Examples All command line examples appear in `Courier New` font. For example, type the following to get to the Post-Install scripts in the `bin` directory:

```
Ztools
```

Files and Directories All file names and directories appear in `Courier New` font. For example, the `Zebra<version number>.tar` file and the `/root` directory.

Cautions, Important, Note, and Example

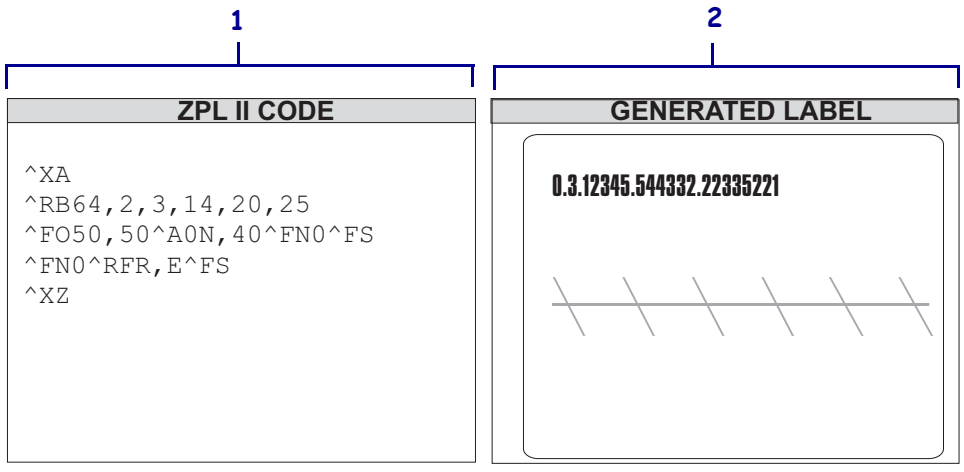
Important • Advises you of information that is essential to complete a task.

Note • Indicates neutral or positive information that emphasizes or supplements important points of the main text.

Example • Provides an example, often a scenario, to better clarify a section of text.

Illustration Callouts Callouts are used when an illustration contains information that needs to be labeled and described. A table that contains the labels and descriptions follows the graphic. Figure 1 provides an example.

Figure 1 • Sample Figure with Callouts



1	ZPL II Code
2	Generated label

Related Documents

The following documents might be helpful references:

- *ZPL II® Programming Guide Volume I* (part number 45541L) and *Volume II* (part number 45542L).
- *ZBI Guide* (part number 45543LB)
- The ZebraLink Solution Application White-Paper



Notes • _____

_____

_____

_____

_____

_____

_____

_____

_____

_____

1 Introduction

This guide is the unabridged, alphabetical reference of programming commands supported in the firmware.

Firmware You can get the printer's firmware version by printing out a configuration label.
Note • Firmware upgrades are available at: `www.zebra.com`.

If you are using a previous version of Zebra printer firmware, some of the commands are the same and function as they did before— but equally as many are new and are not recognized by firmware that is earlier than X.10. Other commands have been redesigned and significantly enhanced to support innovations like:

- ZebraNet® ALERT
- Real-Time Clock
- Zebra BASIC Interpreter (ZBI)

Any word processor or text editor capable of creating ASCII-only files can be used to recreate the examples in this guide. Most of the examples are made up of a series of instruction lines. When you finish typing a line, press Enter. Continue this process for all of the lines in the example you are experimenting with.

To provide more information and convenient cross-referencing, commands that are directly related to features discussed in *Volume Two* have been noted under their Comments heading, pointing to the appendix or section that applies.

ZPL and ZPL II To see the difference between ZPL and ZPL II, see the *ZPL II Programming Guide, Volume Two*.



Notes • _____

_____

_____

_____

_____

_____

_____

_____

_____

_____

2

ZPL Commands

This section contains the complete alphabetical listing of ZPL II commands.

Description This heading provides an explanation of how the command is used, what it is capable of, and any defining characteristics it has.

Format Format explains how the command is syntactically arranged and what parameters it contains.

For Example The ^B8 command prints a EAN-8 bar code. The format of the ^B8 command is: ^B8o,h,f,g. It is arranged with the caret symbol (^), the command code (B8), and the parameters and are replaced with supported values.

Parameters If a command has values that can be defined to make its function more specific, these are outlined as parameters. Parameters typically have *Accepted Values* and *Default Values*.

Still using the ^B8 example, the h parameter is defined as:

h = bar code height (in dots)

Accepted Values: 1 to 32000

Default Value: value set by ^BY

If the command has no parameters – for example ~JA (Cancel All) – the parameter heading is removed, indicating that the format of the command (~JA) is acceptable ZPL II code.

Example • When the command is best clarified in context, an example of the ZPL II code is provided. Text indicating exact code entered is printed in an easily recognizable Courier font. An example of code using the ^B8 command looks like this:

```
^XA
^FO50,50
^B8N,100,Y,N
^FD1234567^FS
^XZ
```

Notice that the ^B8 parameter letters have been replaced with real values that apply to the command. In this example N,100,Y,N have been entered.

Comment This section is reserved for notes that are of value to a programmer, warnings of potential command interactions, or command-specific information that should be taken into consideration.

Example • An example comment is: **This command works only when the printer is idle,** or **This command is ignored if a value exceeds the parameter limits**.

Comments are also included next to parameters if they apply directly to a particular

Basic ZPL Exercises

The purpose of these exercises is to introduce basic ZPL commands to novice ZPL users.

Make sure this checklist is complete:

- ❑ Load the printer with labels that are big enough to give you ample space to work with.
- ❑ Print a configuration label (CANCEL test).
- ❑ Look at the configuration label and make sure that the `LEFT POSITION` is set to `000` and `LABEL TOP` is set to `000`.
- ❑ Determine the printer's resolution. It is listed on the configuration label. 8/MM = 200 dpi, 12/MM = 300 dpi and 24/MM = 600 dpi.

Tips

These are some tips when using ZPL:

- Use the DOS text editor to write ZPL files.
- Save the file as a `.txt` file and copy it to the printer from DOS command line.

Before you begin

Some things that are important to understand before you begin are:

- 200 dpi means the resolution of the printhead is 200 dots per inch. If you program the printer to draw a line 100 dots long that equals a half inch. 100 dots on a 300 dpi printer prints a line 1/3 inch long.
- The home position that all your coordinates are referencing is at the left-hand trailing edge of the label as the label comes out of the printer. (There are some exceptions to this.)

Exercises

The exercises start simple and gradually progress to give you an opportunity to try a variety of commonly used ZPL commands. Not all commands are covered, but this should be a good core of commands to learn. Some commands may not be supported due to the firmware version in your printer.

Exercise 1 • This exercise shows you how to specify a location for an entered name.

1. Print your name on the label.
2. Start by printing just your name on the label using the following format as a model.



Important • Your name goes where you see **xxxxxxxxxxx** in the second line of code.

```
^XA
^FO50,50^ADN,36,20^FDxxxxxxxxxxx^FS
^XZ
Send the above format to the printer.
^XA every format must start with this command
^XZ every format must end with this command
^FD field data
^FS field separator
^FO field origin
```

3. When the label prints correctly, alter the first number after the ^FOx and see how that effects the print position. Alter the second number after the ^FO50,x and see how that effects the print position.

Font instruction

^ADN

1. Alter the numbers after the ^ADN,x,x command.
   - 18,10 is the smallest size you can make the **D** font.
   - The first number is the height of the font in dots, and the second is the width in dots.
   - You can use direct multiples up to ten times that size as a maximum.

Example • 180,100 is the largest you can make the **D** font.

- 25,18 would not be a valid size. The printer rounds to the next recognizable size.

2. In the *ZPL II Programming Guide Volume Two*, see Appendix E to check the font matrices tables for other fonts to try.
3. Try the zero scalable font ^A0N,x,x.

   This font is scalable and you can choose any height and width.

Rotation commands

1. Change ^ADN to ^ADR, then ^ADI, then ^ADB.

   See how the print position changes.
2. Add more fields.
3. Add two more fields to print directly under your name using the ^ADN,36,20 font and size:

   Your street address

   Your city, state, zip
4. You must add two more lines of code that start off with:

   ```
   ^XA
   ^FO50,50^ADN,36,20^FDxxxxxxxxxxx^FS
   ^FO    (fill in the rest)
   ^FO    (fill in the rest)
   ^XZ
   ```

Make sure all these fields print in the same font and size and left side of fields has same vertical alignment.

Your name

1200 W Main Street

Anytown, Il 60061

Exercise 2 • Boxes and lines

1. Use the address format from **Exercise 1**.
2. Add this new line to your existing format:

   ```
   ^FO50,200^GB200,200,2^FS
   ```

   This prints a box one wide by one inch long and the thickness of the line is 2 dots.
3. Reposition and resize the square so that it goes around the name and address uniformly.
4. Print a line by adding:

   ```
   ^FO50,300^GB400,0,4,^FS
   ```

   This prints a horizontal line two inches wide by 4 dots thick.

5. Print a vertical line using this code:

   ^F0100,50^GBO,400,4^FS

Exercise 3 • Bar codes — ^B3 code 39 bar code

1. Write the following format and send to the printer:

   ^XA

   ^FO50,50^B3N,N,100,Y,N^FD123456^FS

   ^XZ

2. Try changing each of the parameters in the ^B3 string so you can see the effects.



Important • For valid parameter choices, see *^B3 on page 21*.

   ^B3o,e,h,f,g

   ^BY

3. Insert the ^BY command just before the ^B3 to see how the narrow bar width can be altered.

   ^FO50,50^BY2^B3..etc ^BYx, acceptable values for x are 1 through 10

4. Alter the ratio of the narrow to wide bar.

   ^FO50,50^BY2,3^B3..etc ^BY2,x acceptable values for x are 2.1 through 3 in .1 increments

5. Print out a ^B3 bar code with the interpretation line on top of the bar code and the bar code rotated 90 degrees.

6. Add a ^PQ just before the ^XZ to print several labels.

   ^PQ4

   ^XZ

   ^PR Print rate (in inches per second)

7. Add a ^PR command after the ^XA at the beginning of the format to change the print rate (print speed).

   ^XA

   ^PR4 then try ^PR6 ^PRx acceptable values for x are 2 through 12 (check printer specs)

   See how the print speed affects the print quality of the bar code. You may need to increase the printer darkness setting at higher print speeds.

Exercise 4 • ^SN — Serial Number command

1. Send this format to the printer:

   ```
   ^XA
   ^FO100,100^ADN,36,20^SN001,1,Y^FS
   ^PQ3
   ^XZ
   ```

   To vary the ^SNv,n,z to exercise the increment/decrement and leading zeros functions, consult this guide.

   If your serial number contains alpha and numeric characters, you can increment or decrement a specific segment of the data even if it is in the middle, as this sample sequence shows:

   ```
   ABCD1000EFGH, ABCD1001EFGH, ABCD1002EFGH
   ```

2. Send this file to the printer and to see how it increments the serial number. The ^SF command can also work with alpha characters.

   ```
   ^XA
   ^FO100,100^ADN,36,20^FDABCD1000EFGH^SF%%%%dddd%%%%,10000^FS
   ^PQ15
   ^XZ
   ```

   Notice how the field data character position aligns with ^SF data string:

^	F	D	A	B	C	D	1	0	0	0	E	F	G	H
^	S	F	%	%	%	%	d	d	d	d	%	%	%	%
										1	0	0	0	0
										2	0	0	0	0
										3	0	0	0	0

   And on through....

							1	0	1	4	0	0	0	0

The last label prints **ABCD1014EFGH**.

The % is placed in positions that you do not want to increment/decrement,
d = decimal, 10000=increment value.

For more details on ^SF, see *^SF* on page 254.

Exercise 5 • Saving a template to memory. ^IS and image save and image load.



Note • This exercise requires you to type a lot of data, and a single typing error will cause problems. It also serves as an exercise to troubleshoot your code against the errors you see on your labels.

1. Send this format to the printer:

```
^XA
^FO20,30^GB750,1100,4^FS
^FO20,30^GB750,200,4^FS
^FO20,30^GB750,400,4^FS
^FO20,30^GB750,700,4^FS
^FO20,226^GB325,204,4^FS
^FO30,40^ADN,36,20^FDShip to:^FS
^FO30,260^ADN,18,10^FDPart number #^FS
^FO360,260^ADN,18,10^FDDescription:^FS
^FO30,750^ADN,36,20^FDFrom:^FS
^ISR:SAMPLE.ZPL^FS
^XZ
```

2. Send this format:

```
^XA
^ILR:SAMPLE.ZPL^FS
^FO150,125^ADN,36,20^FDAcme Printing^FS
^FO60,330^ADN,36,20^FD14042^FS
^FO400,330^ADN,36,20^FDScrew^FS
^FO70,480^BY4^B3N,,200^FD12345678^FS
^FO150,800^ADN,36,20^FDMacks Fabricating^FS
^XZ
```

In this way the template only needs to be sent one time to the printer's memory. Subsequent formats can be sent recalling the template and merging variable data into the template. In this exercise, the file was saved in the printers `R:` memory, which is volatile.

DF and ^XF — Download format and recall format

Similar concept to ^IS and ^IL command. ^IS and ^IL in general processes faster in the printer then ^DF and ^XF.

This is the way the ^DF and ^XF format structure produces a label similar to the ^IS/^IL sample you just tried.

```
^XA
^DFR:SAMPLE.ZPL^FS
^FO20,30^GB750,1100,4^FS
^FO20,30^GB750,200,4^FS
^FO20,30^GB750,400,4^FS
^FO20,30^GB750,700,4^FS
^FO20,226^GB325,204,4^FS
^FO30,40^ADN,36,20^FDShip to:^FS
^FO30,260^ADN,18,10^FDPart number #^FS
^FO360,260^ADN,18,10^FDDescription:^FS
^FO30,750^ADN,36,20^FDFrom:^FS
^FO150,125^ADN,36,20^FN1^FS (ship to)
^FO60,330^ADN,36,20^FN2^FS(part num)
^FO400,330^ADN,36,20^FN3^FS(description)
^FO70,480^BY4^B3N,,200^FN4^FS(barcode)
^FO150,800^ADN,36,20^FN5^FS (from)
^XZ
^XA
^XFR:SAMPLE.ZPL
^FN1^FDAcme Printing^FS
^FN2^FD14042^FS
^FN3^FDScrew^FS
^FN4^FD12345678^FS
^FN5^FDMacks Fabricating^FS
^XZ
```

^A

Scalable/Bitmapped Font

Description The ^A command is a scalable/bitmapped font that uses built-in or TrueType® fonts. ^A designates the font for the current ^FD statement or field. The font specified by ^A is used only once for that ^FD entry. If a value for ^A is not specified again, the default ^CF font is used for the next ^FD entry.

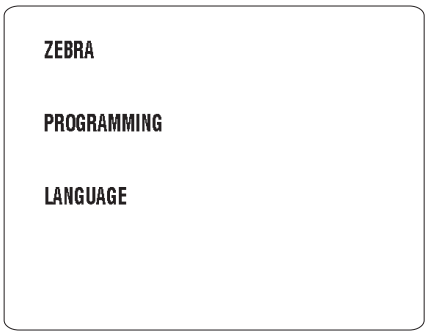
Format ^Afo,h,w

This table identifies the parameters for this format:

Command	Details
f = font name	*Accepted Values:* A through Z, and 1 to 9 *Default Value:* A Any font in the printer (downloaded, EPROM, stored fonts, fonts A through Z and 1 to 9).
o = font orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* the last accepted ^FW value or the ^FW default
h = Character Height (in dots)	**Scalable** *Accepted Values:* 10 to 32000 *Default Value:* 15 or the last accepted ^CF value **Bitmapped** *Accepted Values:* multiples of height from 2 to 10 times the standard height, in increments of 1 *Default Value:* the standard matrix height for a specified font
w = width (in dots)	**Scalable** *Accepted Values:* 10 to 32000 *Default Value:* 12 or last accepted ^CF value **Bitmapped** *Accepted Values:* multiples of width from 2 to 10 times the standard width, in increments of 1 *Default Value:* the standard matrix width for a specified font.

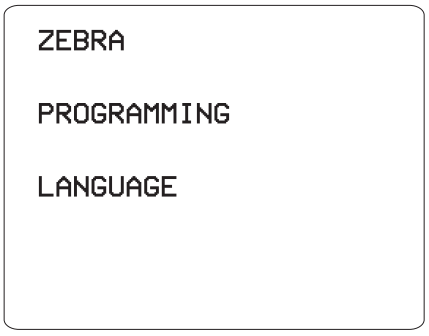
Scalable Font Command

Example • This is an example of a scalable font command:

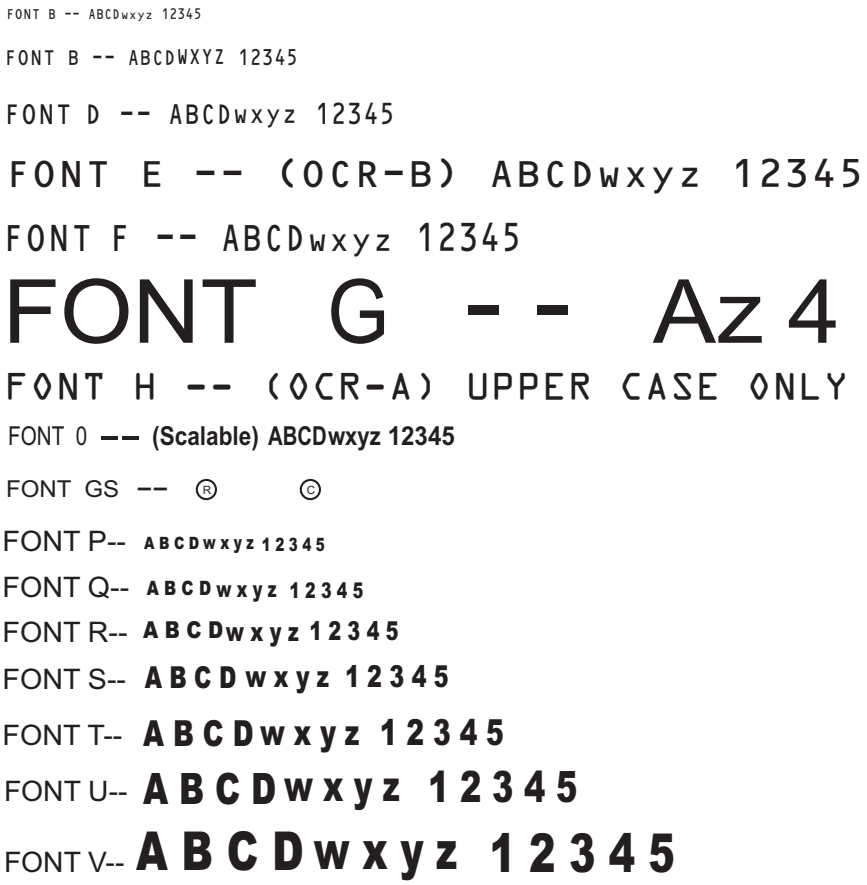
ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^A0,32,25 ^FDZEBRA^FS ^FO50,150 ^A0,32,25 ^FDPROGRAMMING^FS ^FO50,250 ^A0,32,25^FDLANGUAGE^FS ^XZ	

Bitmap Font Command

Example • This is an example of a bitmap font command:

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^ADN,36,20 ^FDZEBRA^FS ^FO50,150 ^ADN,36,20 ^FDPROGRAMMING^FS ^FO50,250 ^ADN,36,20^FDLANGUAGE^FS ^XZ	

Example • This is an example of the P - V fonts:



Comments Fonts are built using a matrix that defines standard height-to-width ratios. If you specify only the height or width value, the standard matrix for that font automatically determines the other value. If the value is not given or a 0 (zero) is entered, the height or width is determined by the standard font matrix.

^A@

Use Font Name to Call Font

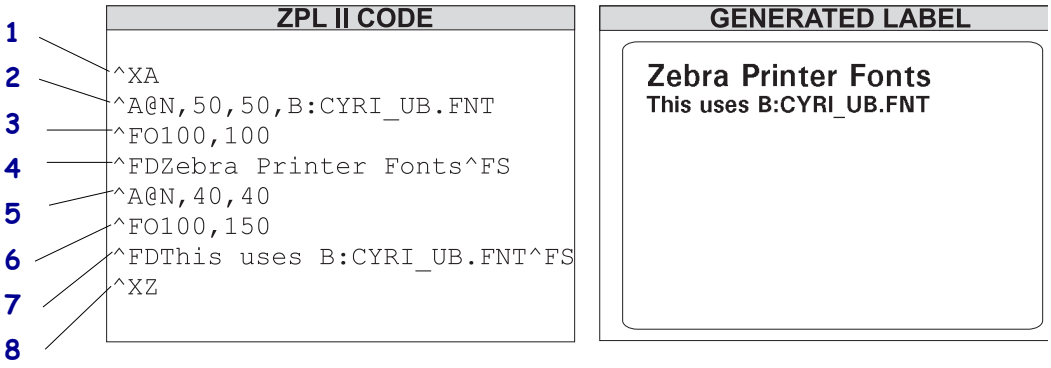
Description The ^A@ command uses the complete name of a font, rather than the character designation used in ^A. Once a value for ^A@ is defined, it represents that font until a new font name is specified by ^A@.

Format ^A@o,h,w,d:o.x

This table identifies the parameters for this format:

Parameters	Details
o = font orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* N or the last ^FW value
h = character height (in dots)	*Default Value:* magnification specified by **w** (character width) or the last accepted ^CF value. The base height is used if none is specified. **Scalable** the value is the height in dots of the entire character block. Magnification factors are unnecessary, because characters are scaled. **Bitmapped** the value is rounded to the nearest integer multiple of the font's base height, then divided by the font's base height to give a magnification nearest limit.
w = width (in dots)	*Default Value:* magnification specified by **h** (height) or the last accepted ^CF value. The base width is used if none is specified. **Scalable** the value is the width in dots of the entire character block. Magnification factors are unnecessary, because characters are scaled. **Bitmapped** the value is rounded to the nearest integer multiple of the font's base width, then divided by the font's base width to give a magnification nearest limit.
d = drive location of font	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = font name	*Accepted Values:* any valid font *Default Value:* if an invalid or no name is entered, the default set by ^CF is used. If no font has been specified in ^CF, font A is used. The font named carries over on all subsequent ^A@ commands without a font name.
x = extension	*Fixed Value:* .FTN

Example • This example is followed by a table that identifies the callouts:



1	Starts the label format.
2	Searches non-volatile printer memory (`B:`) for CYRI_UB.FNT. When the font is found, the `^A@` command sets the print orientation to normal and the character size to 50 dots by 50 dots.
3	Sets the field origin at 100,100.
4	Prints the field data, *Zebra Printer Fonts* on the label.
5	Calls the font again and character size is decreased to 40 dots by 40 dots.
6	Sets the new field origin at 100,150.
7	Prints the field data, *This uses the B:CYRI_UB.FNT* on the label.
8	Ends the label format.

Comments For more information on scalable and bitmap fonts, see *ZPL II Programming Guide: Volume Two.*

^B1

Code 11 Bar Code

Description The ^B1 command produces the Code 11 bar code, also known as USD-8 code. In a Code 11 bar code, each character is composed of three bars and two spaces, and the character set includes 10 digits and the hyphen (-).

- ^B1 supports print ratios of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^B1o,e,h,f,g

Important • If additional information about the Code 11 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value: current* ^FW *value*
e = check digit	*Accepted Values:* Y (yes) = 1 digit N (no) = 2 digits *Default Value:* N
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of the Code 11 bar code:

CODE 11 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^B1N,N,150,Y,N ^FD123456^FS ^XZ

CODE 11 BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9 -

Internal Start/Stop Character:△

When used as a stop character:

△ is used with 1 check digit
△ is used with 2 check digits

^B2

Interleaved 2 of 5 Bar Code

Description The ^B2 command produces the Interleaved 2 of 5 bar code, a high-density, self-checking, continuous, numeric symbology.

Each data character for the Interleaved 2 of 5 bar code is composed of five elements: five bars or five spaces. Of the five elements, two are wide and three are narrow. The bar code is formed by interleaving characters formed with all spaces into characters formed with all bars.

- ^B2 supports print ratios of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^B2o,h,f,g,e



Important • If additional information about the Code 11 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
e = calculate and print Mod 10 check digit	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of an Interleaved 2 of 5 bar code:

INTERLEAVED 2 OF 5 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^B2N,150,Y,N,N ^FD123456^FS ^XZ

INTERLEAVED 2 OF 5 BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9

Start/Stop (internal)

Comments The total number of digits in an Interleaved 2 of 5 bar code must be even. The printer automatically adds a leading 0 (zero) if an odd number of digits is received.

The Interleaved 2 of 5 bar code uses the Mod 10 check-digit scheme for error checking. For more information on Mod 10 check digits, see Appendix C in *ZPL II Programming Guide Volume Two*.

^B3

Code 39 Bar Code

Description The Code 39 bar code is the standard for many industries, including the U.S. Department of Defense. It is one of three symbologies identified in the American National Standards Institute (ANSI) standard MH10.8M-1983. Code 39 is also known as USD-3 Code and 3 of 9 Code.

Each character in a Code 39 bar code is composed of nine elements: five bars, four spaces, and an inter-character gap. Three of the nine elements are wide; the six remaining elements are narrow.

- ^B3 supports print ratios of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.
- Code 39 automatically generates the start and stop character (*).
- Asterisk (*) for start and stop character prints in the interpretation line, if the interpretation line is turned on.
- Code 39 is capable of encoding the full 128-character ASCII set.

Format ^B3o,e,h,f,g

Important • If additional information about the Code 39 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
e = Mod-43 check digit	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a Code 39 bar code:

CODE 39 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^B3N,N,100,Y,N ^FD123ABC^FS ^XZ

CODE 39 BAR CODE CHARACTERS

			0		1		2		3		4		5		6		7		8		9			
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
								-	.	$	/	+	%	Space										

Comments Extended ASCII is a function of the scanner, not of the bar code. Your scanner must have extended ASCII enabled for this feature to work. To enable extended ASCII in the Code 39, you must first encode *+$* in your ^FD statement. To disable extended ASCII, you must encode *-$* in your ^FD statement.

Example • This example encodes a carriage return with line feed into a Code 39 bar code:

ZPL II CODE	GENERATED LABELS
^XA ^FO20,20 ^B3N,N,100,Y ^FDTEST+$$M$J-$^FS ^XZ	

Full ASCII Mode for Code 39

Code 39 can generate the full 128-character ASCII set using paired characters as shown in these tables:

Table 1 • Code 39 Full ASCII Mode

ASCII	Code 39
SOH	$A
STX	$B
ETX	$C
EOT	$D
ENQ	$E
ACK	$F
BEL	$G
BS	$H
HT	$I
LF	$J
VT	$K
FF	$L
CR	$M
SO	$N
SI	$O
DLE	$P
DC1	$Q
DC2	$R
DC3	$S
DC4	$T
NAK	$U
SYN	$V
ETB	$W
CAN	$X
EM	$Y
SUB	$Z
ESC	%A
FS	%B
FS	%C
RS	%D
US	%E

ASCII	Code 39
SP	Space
!	/A
“	/B
#	/C
$	/D
%	/E
&	/F
‘	/G
(	/H
)	/I
*	/J
++	/K
‘	/L
-	-
.	.
/	/O
0	O
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
:	/Z
;	%F
<	%G
=	%H
>	%I
?	%J

Table 2 • Code 39 Full ASCII Mode

ASCII	Code 39
@	%V
A	A
B	B
C	C
D	D
E	E
F	F
G	G
H	H
I	I
J	J
K	K
L	L
M	M
N	N
O	O
P	P
Q	Q
R	R
S	S
T	T
U	U
V	V
W	W
X	X
Y	Y
Z	Z
[	%K
\	%L
]	%M
^	%N
_	%O

ASCII	Code 39
‘	%W
a	+A
b	+B
c	+C
d	+D
e	+E
f	+F
g	+G
h	+H
I	+I
j	+J
k	+K
l	+L
m	+M
n	+N
o	+O
p	+P
q	+Q
r	+R
s	+S
t	+T
u	+U
v	+V
w	+W
x	+X
y	+Y
z	+Z
{	%P
|	%Q
}	%R
~	%S
DEL	%T, %X

^B4

Code 49 Bar Code

Description The ^B4 command creates a multi-row, continuous, variable-length symbology capable of encoding the full 128-character ASCII set. It is ideally suited for applications requiring large amounts of data in a small space.

The code consists of two to eight rows. A row consists of a leading quiet zone, four symbol characters encoding eight code characters, a stop pattern, and a trailing quiet zone. A separator bar with a height of one module separates each row. Each symbol character encodes two characters from a set of Code 49 characters.

- ^B4 has a fixed print ratio.
- Rows can be scanned in any order.

Format ^B4o,h,f,m

Important • For additional information about the Code 11 bar code is required, see go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = height multiplier of individual rows	*Accepted Values:* 1 to height of label *Default Value:* value set by ^BY This number multiplied by the module equals the height of the individual rows in dots. 1 is not a recommended value.

Parameters	Details
f = print interpretation line	*Accepted Values:* N = no line printed A = print interpretation line above code B = print interpretation line below code *Default Value:* N When the field data exceeds two rows, expect the interpretation line to extend beyond the right edge of the bar code symbol.
m = starting mode	*Accepted Values:* 0 = Regular Alphanumeric Mode 1 = Multiple Read Alphanumeric 2 = Regular Numeric Mode 3 = Group Alphanumeric Mode 4 = Regular Alphanumeric Shift 1 5 = Regular Alphanumeric Shift 2 A = Automatic Mode. The printer determines the starting mode by analyzing the field data. *Default Value:* A

Example • This is an example of a Code 49 bar code:

CODE 49 BAR CODE



ZPL II CODE

```
^XA
^FO150,100^BY3
^B4N,20,A,A
^FD12345ABCDE^FS
^XZ
```

Table 3 • Code 49

Field Data Set	Unshifted Character Set	Shift 1 Character Set	Shift 2 Character Set
0	0	’	
1	1	ESC	;
2	2	FS	<
3	3	GS	=
4	4	RS	>
5	5	US	?
6	6	!	@
7	7	“	[
8	8	#	\
9	9	&	]
A	A	SOH	a
B	B	STX	b
C	C	ETX	c
D	D	EOT	d
E	E	ENQ	e
F	F	ACK	f
G	G	BEL	g
H	H	BS	h
I	I	HT	I
J	J	LF	j
K	K	VT	k
L	L	FF	l
M	M	CR	m
N	N	SO	n
O	O	SI	o
P	P	DLE	p
Q	Q	DC1	q
R	R	DC2	r
S	S	DC3	s
T	T	DC4	t
U	U	NAK	u
V	V	SYN	v
W	W	ETB	w
X	X	CAN	x
Y	Y	EM	y
Z	Z	SUB	z
-	-	(	_
.	.	)	‘
SPACE	SPACE	Null	DEL
$	$	*	{
/	/	,	|
++	++	:	}
%	%	reserved	~
< (Shift 1)			
> (Shift 2)			
: (N.A.)			
; (N.A.)			
? (N.A.)			
= (Numeric Shift)			

Code 49 Shift 1 and 2 Character Substitutions

Code 49 Field Data Character Set

The ^FD data sent to the printer when using starting modes 0 to 5 is based on the Code 49 Internal Character Set. This is shown in the first column of the Code 49 table on the previous page. These characters are Code 49 control characters:

: ; < = > ?

Valid field data must be supplied when using modes 0 to 5. Shifted characters are sent as a two-character sequence of a shift character followed by a character in the unshifted character set.

Example • To encode a lowercase **a**, send **a >** (Shift 2) followed by an uppercase **A**. If interpretation line printing is selected, a lowercase *a* prints in the interpretation line. This reflects what the output from the scanner reads. Code 49 uses uppercase alphanumeric characters only.

If an invalid sequence is detected, the Code 49 formatter stops interpreting field data and prints a symbol with the data up to the invalid sequence. These are examples of invalid sequences:

- Terminating numeric mode with any characters other than 0 to 9 or a Numeric Space.
- Starting in Mode 4 (Regular Alphanumeric Shift 1) and the first field data character is not in the Shift 1 set.
- Starting in Mode 5 (Regular Alphanumeric Shift 2) and the first field data character is not in the Shift 2 set.
- Sending Shift 1 followed by a character not in the Shift 1 set.
- Sending Shift 2 followed by a character not in the Shift 2 set.
- Sending two Shift 1 or Shift 2 control characters.

Advantages of Using the Code 49 Automatic Mode

Using the default (Automatic Mode) completely eliminates the need for selecting the starting mode or manually performing character shifts. The Automatic Mode analyzes the incoming ASCII string, determines the proper mode, performs all character shifts, and compacts the data for maximum efficiency.

Numeric Mode is selected or shifted only when five or more continuous digits are found. Numeric packaging provides no space advantage for numeric strings consisting of fewer than eight characters.

^B5

Planet Code bar code

Description The ^B5 command is supported in all printers as a resident bar code.

Format ^B5o,h,f,g

This table identifies the parameters for this format:

Parameters	Details
o = orientation code	*Accepted Values:* N = normal R = rotated I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 9999 *Default Value:* value set by ^BY
f = interpretation line	*Accepted Values:* N = no default Y = yes
g = determines if the interpretation line is printed above the bar code	*Accepted Values:* N = no default Y = yes

Example • This is an example of a Planet Code bar code:

ZPL II CODE

```
^XA
^FO150,100^BY3
^B5N,100,Y,0
^FD12345678901$FS
^XZ
```

GENERATED LABEL



^B7

PDF417 Bar Code

Description The ^B7 command produces the PDF417 bar code, a two-dimensional, multirow, continuous, stacked symbology. PDF417 is capable of encoding over 1,000 characters per bar code. It is ideally suited for applications requiring large amounts of information at the time the bar code is read.

The bar code consists of three to 90 stacked rows. Each row consists of start and stop patterns and symbol characters called *code-words*. A code-word consists of four bars and four spaces. A three code-word minimum is required per row.

The PDF417 bar code is also capable of using the structured append option (^FM), which allows you to extend the field data limitations by printing multiple bar codes. For more information on using structured append, see *^FM* on page 139.

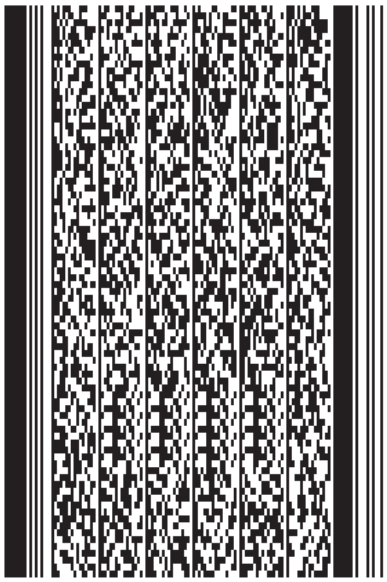
- PDF417 has a fixed print ratio.
- Field data (^FD) is limited to 3K of character data.

Format ^B7o,h,s,c,r,t

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height for individual rows (in dots)	*Accepted Values:* 1 to height of label *Default Value:* value set by ^BY This number multiplied by the module equals the height of the individual rows in dots. 1 is not a recommended value.
s = security level	*Accepted Values:* 1 to 8 (error detection and correction) *Default Value:* 0 (error detection only) This determines the number of error detection and correction code-words to be generated for the symbol. The default level provides only error detection without correction. Increasing the security level adds increasing levels of error correction and increases the symbol size.
c = number of data columns to encode	*Accepted Values:* 1 to 30 *Default Value:* 1:2 (row-to-column aspect ratio) You can specify the number of code-word columns giving control over the width of the symbol.
r = number of rows to encode	*Accepted Values:* 3 to 90 *Default Value:* 1:2 (row-to-column aspect ratio) You can specify the number of symbol rows giving control over the height of the symbol. For example, with no row or column values entered, 72 code-words would be encoded into a symbol of six columns and 12 rows. Depending on code-words, the aspect ratio is not always exact.
t = truncate right row indicators and stop pattern	*Accepted Values:* Y (perform truncation) and N (no truncation) *Default Value:* N

Example • This is an example of a PDF417 bar code:

PDF417 BAR CODE	ZPL II CODE
	^XA ^BY2,3 ^FO10,10^B7N,5,5,,83,N ^FDZebra Technologies Corporation strives to be the expert supplier of innovative solutions to speciality demand labeling and ticketing problems of business and government. We will attract and retain the best people who will understand our customer's needs and provide them with systems, hardware, software, consumables and service offering the best value, high quality, and reliable performance, all delivered in a timely manner. ^FS^XZ

Example • This is an example of a PDF417 without and with truncation selected:

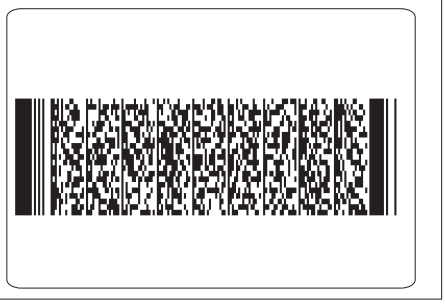


PDF417 without Truncation being selected



PDF417 with Truncation being selected

Example • This example shows the ^B7 command used with field hex (^FH) characters:

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50^BY3,3.0^B7N,8,5,7,21,N ^FH_^FD[)>_1E06_1DP12345678_1DQ160 _1D1JUN123456789A2B4C6D8E_1D20LA6-987 _1D21L54321 ZES_1D15KG1155 _1DBSC151208_1D7Q10GT_1E_04^FS ^XZ	

Comments Noted in this bulleted list:

- If both columns and rows are specified, their product must be less than 928.
- No symbol is printed if the product of columns and rows is greater than 928.
- No symbol is printed if total code-words are greater than the product of columns and rows.
- Serialization is not allowed with this bar code.
- The truncation feature can be used in situations where label damage is not likely. The right row indicators and stop pattern is reduced to a single module bar width. The difference between a nontruncated and a truncated bar code is shown in *This is an example of a PDF417 without and with truncation selected:* on page 32.

Special Considerations for ^BY When Using PDF417

When used with ^B7, the parameters for the ^BY command are:

w = module width (in dots)

Accepted Values: 2 to 10

Default Value: 2

r = ratio

Fixed Value: 3 (ratio has no effect on PDF417)

h = height of bars (in dots)

Accepted Values: 1 to 32000

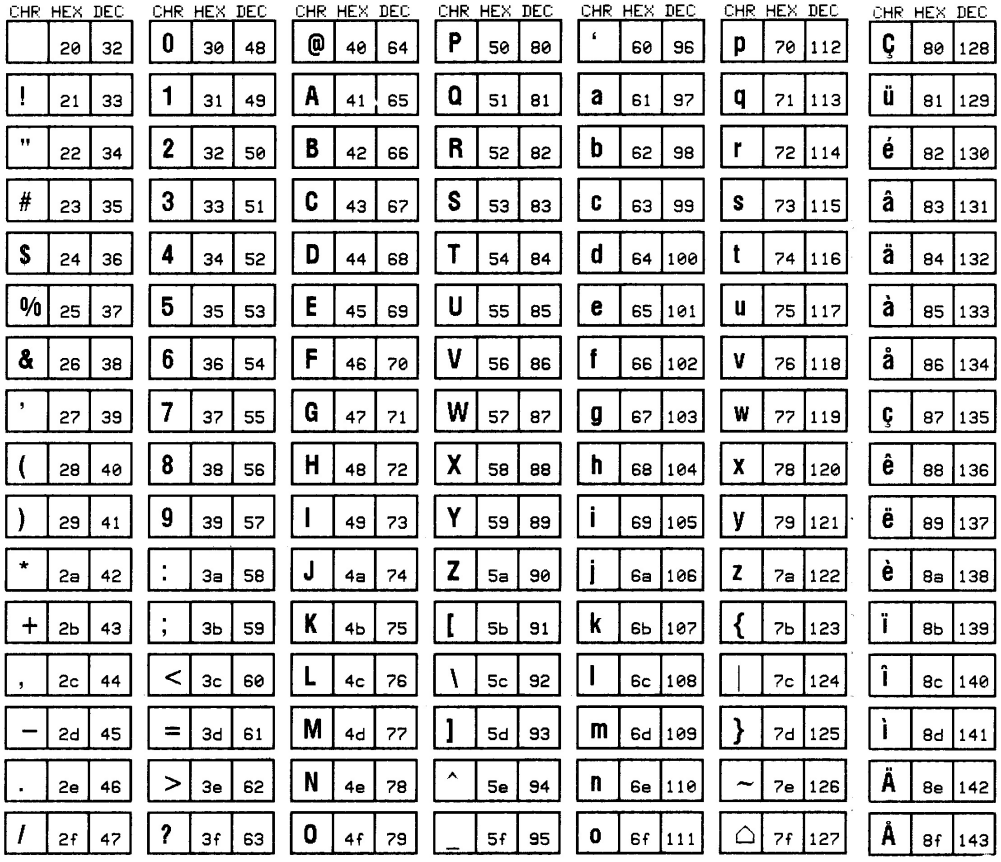
Default Value: 10

PDF417 uses this only when row height is not specified in the ^B7 h parameter.

Special Considerations for ^FD When Using PDF417

The character set sent to the printer with the ^FD command includes the full ASCII set, except for those characters with special meaning to the printer.

Page Number 850 table



CHR	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC	CHR	HEX	DEC
	20	32	0	30	48	@	40	64	P	50	80	‘	60	96	p	70	112	Ç	80	128
!	21	33	1	31	49	A	41	65	Q	51	81	a	61	97	q	71	113	ü	81	129
"	22	34	2	32	50	B	42	66	R	52	82	b	62	98	r	72	114	é	82	130
#	23	35	3	33	51	C	43	67	S	53	83	c	63	99	s	73	115	â	83	131
$	24	36	4	34	52	D	44	68	T	54	84	d	64	100	t	74	116	ä	84	132
%	25	37	5	35	53	E	45	69	U	55	85	e	65	101	u	75	117	à	85	133
&	26	38	6	36	54	F	46	70	V	56	86	f	66	102	v	76	118	å	86	134
’	27	39	7	37	55	G	47	71	W	57	87	g	67	103	w	77	119	ç	87	135
(	28	40	8	38	56	H	48	72	X	58	88	h	68	104	x	78	120	ê	88	136
)	29	41	9	39	57	I	49	73	Y	59	89	i	69	105	y	79	121	ë	89	137
*	2a	42	:	3a	58	J	4a	74	Z	5a	90	j	6a	106	z	7a	122	è	8a	138
+	2b	43	;	3b	59	K	4b	75	[	5b	91	k	6b	107	{	7b	123	ï	8b	139
,	2c	44	<	3c	60	L	4c	76	\	5c	92	l	6c	108	|	7c	124	î	8c	140
-	2d	45	=	3d	61	M	4d	77	]	5d	93	m	6d	109	}	7d	125	ì	8d	141
.	2e	46	>	3e	62	N	4e	78	^	5e	94	n	6e	110	~	7e	126	Ä	8e	142
/	2f	47	?	3f	63	O	4f	79	_	5f	95	o	6f	111	⌂	7f	127	Å	8f	143

CR and LF are also valid characters for all ^FD statements. This scheme is used:

\& = carriage return/line feed

\\ = backslash (\)

- ^CI13 must be selected to print a backslash (\).

^B8

EAN-8 Bar Code

Description The ^B8 command is the shortened version of the EAN-13 bar code. EAN is an acronym for European Article Numbering. Each character in the EAN-8 bar code is composed of four elements: two bars and two spaces.

- ^B8 supports a fixed ratio.
- Field data (^FD) is limited to exactly seven characters. ZPL II automatically pads or truncates on the left with zeros to achieve the required number of characters.
- When using JAN-8 (Japanese Article Numbering), a specialized application of EAN-8, the first two non-zero digits sent to the printer are always 49.

Format ^B8o,h,f,g



Important • If additional information about the EAN-8 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of an EAN-8 bar code:

EAN-8 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^B8N,100,Y,N ^FD1234567^FS ^XZ

EAN-8 BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

^B9

UPC-E Bar Code

Description The ^B9 command produces a variation of the UPC symbology used for number system 0. It is a shortened version of the UPC-A bar code, where zeros are suppressed, resulting in codes that require less printing space. The 6 dot/mm, 12 dot/mm, and 24 dot/mm printheads produce the UPC and EAN symbologies at 100 percent of their size. However, an 8 dot/mm printhead produces the UPC and EAN symbologies at a magnification factor of 77 percent.

Each character in a UPC-E bar code is composed of four elements: two bars and two spaces. The ^BY command must be used to specify the width of the narrow bar.

- ^B9 supports a fixed ratio.
- Field data (^FD) is limited to exactly 10 characters, requiring a five-digit manufacturer's code and five-digit product code.
- When using the zero-suppressed versions of UPC, you must enter the full 10-character sequence. ZPL II calculates and prints the shortened version.

Format ,h,f,g,e

Important • If additional information about the UPC-E bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
e = print check digit	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y

Example • This is an example of a UPC-E bar code:

UPC-E BAR CODE	ZPL II CODE
	^XA ^FO150,100^BY3 ^B9N,100,Y,N,Y ^FD1230000045^FS ^XZ

UPC-E BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

Rules for Proper Product Code Numbers

- If the last three digits in the manufacturer's number are 000, 100, or 200, valid product code numbers are 00000 to 00999.
- If the last three digits in the manufacturer's number are 300, 400, 500, 600, 700, 800, or 900, valid product code numbers are 00000 to 00099.
- If the last two digits in the manufacturer's number are 10, 20, 30, 40, 50, 60, 70, 80, or 90, valid product code numbers are 00000 to 00009.
- If the manufacturer's number does not end in zero (0), valid product code numbers are 00005 to 00009.

^BA

Code 93 Bar Code

Description The ^BA command creates a variable length, continuous symbology. The Code 93 bar code is used in many of the same applications as Code 39. It uses the full 128-character ASCII set. ZPL II, however, does not support ASCII control codes or escape sequences. It uses the substitute characters shown below.

Control Code	ZPL II Substitute
Ctrl $	&
Ctrl %	‘
Ctrl /	(
Ctrl +	)

Each character in the Code 93 bar code is composed of six elements: three bars and three spaces. Although invoked differently, the human-readable interpretation line prints as though the control code has been used.

- ^BA supports a fixed print ratio.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BAo,h,f,g,e

Important • If additional information about the Code 93 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y

Parameters	Details
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
e = print check digit	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a Code 93 bar code:

CODE 93 BAR CODE



ZPL II CODE

```
^XA
^FO100,75^BY3
^BAN,100,Y,N,N
^FD12345ABCDE^FS
^XZ
```

CODE 93 BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9
A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
- . $ / + % & ’ ( )
SPACE

□ *Denotes an internal start/stop character that must precede and follow every bar code message.*

Comments All control codes are used in pairs.

Code 93 is also capable of encoding the full 128-character ASCII set. For more details, see *Table 4 on page 41*.

Full ASCII Mode for Code 93

Code 93 can generate the full 128-character ASCII set using paired characters as shown in *Table 4 on page 41* and .

Table 4 • Code 93 Full ASCII Mode

ASCII	Code 93
NUL	‘U
SOH	&A
STX	&B
ETX	&C
EOT	&D
ENQ	&E
ACK	&F
BEL	&G
BS	&H
HT	&I
LF	&J
VT	&K
FF	&L
CR	&M
SO	&N
SI	&O
DLE	&P
DC1	&Q
DC2	&R
DC3	&S
DC4	&T
NAK	&U
SYN	&V
ETB	&W
CAN	&X
EM	&Y
SUB	&Z
ESC	‘A
FS	‘B
FS	‘C
RS	‘D
US	‘E

ASCII	Code 93
SP	Space
!	(A
“	(B
#	(C
$	(D
%	(E
&	(F
‘	(G
(	(H
)	(I
*	(J
++	++
‘	(L
-	-
.	.
/	/
0	O
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
:	(Z
;	‘F
<	‘G
=	‘H
>	‘I
?	‘J

Table 5 • Code 93 Full ASCII Mode

ASCII	Code 93
@	‘V
A	A
B	B
C	C
D	D
E	E
F	F
G	G
H	H
I	I
J	J
K	K
L	L
M	M
N	N
O	O
P	P
Q	Q
R	R
S	S
T	T
U	U
V	V
W	W
X	X
Y	Y
Z	Z
[	‘K
\	‘L
]	‘M
^	‘N
_	‘O

ASCII	Code 93
‘	‘W
a	)A
b	)B
c	)C
d	)D
e	)E
f	)F
g	)G
h	)H
I	)I
j	)J
k	)K
l	)L
m	)M
n	)N
o	)O
p	)P
q	)Q
r	)R
s	)S
t	)T
u	)U
v	)V
w	)W
x	)X
y	)Y
z	)Z
{	‘P
|	‘Q
}	‘R
~	‘S
DEL	‘T

^BB

CODABLOCK Bar Code

Description The ^BB command produces a two-dimensional, multirow, stacked symbology. It is ideally suited for applications that require large amounts of information.

Depending on the mode selected, the code consists of one to 44 stacked rows. Each row begins and ends with a start and stop pattern.

- CODABLOCK A supports variable print ratios.
- CODABLOCK E and F support only fixed print ratios.

Format ^BBo,h,s,c,r,m

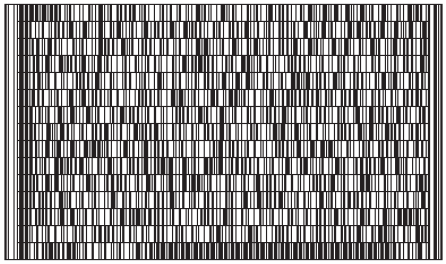
Important • If additional information about the CODABLOCK bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* N
h = bar code height for individual rows (in dots)	*Accepted Values:* 2 to 32000 *Default Value:* 8 This number, multiplied by the module, equals the height of the individual row in dots.
s = security level	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y Security level determines whether symbol check-sums are generated and added to the symbol. Check sums are never generated for single-row symbols. This can be turned off only if parameter m is set to A.
c = number of characters per row (data columns)	*Accepted Values:* 2 to 62 characters This is used to encode a CODABLOCK symbol. It gives the you control over the width of the symbol.

Parameters	Details
r = number of rows to encode	*Accepted Values:* for CODABLOCK A: 1 to 22 for CODABLOCK E and F: 2 to 4 • If values for c and r are not specified, a single row is produced. • If a value for r is not specified, and c exceeds the maximum range, a single row equal to the field data length is produced. • If a value for c is not specified, the number of characters per row is derived by dividing the field data by the value of r. • If both parameters are specified, the amount of field data must be less than the product of the specified parameters. If the field data exceeds the value of the product, either no symbol or an error code is printed (if ^CV is active). • If the data field contains primarily numeric data, fewer than the specified rows might be printed. If the field data contains several shift and code-switch characters, more than the specified number of rows might be printed.
m = mode	*Accepted Values:* A, E, F CODABLOCK A uses the Code 39 character set. CODABLOCK F uses the Code 128 character set. CODABLOCK E uses the Code 128 character set and automatically adds FNC1. *Default Value:* F

Example • This is an example of a CODABLOCK bar code:

CODABLOCK BAR CODE	ZPL II CODE
	^XA ^BY2,3 ^FO10,10^BBN,30,,30,44,E ^FDZebra Technologies Corporation strives to be the expert supplier of innovative solutions to speciality demand labeling and ticketing problems of business and government. We will attract and retain the best people who will understand our customer's needs and provide them with systems, hardware, software, consumables and service offering the best value, high quality, and reliable performance, all delivered in a timely manner.^FS ^XZ

Special Considerations for the ^BY Command When Using ^BB

The parameters for the ^BYw,r,h command, when used with a ^BB code, are as follows:

w = module width (in dots)

Accepted Values: 2 to 10 (CODABLOCK A only)

Default Value: 2

r = ratio

Fixed Value: 3 (ratio has no effect on CODABLOCK E or F)

h = height of bars (in dots)

Accepted Values: 1 to 32,32000

Default Value: 10

CODABLOCK uses this as the overall symbol height only when the row height is not specified in the ^BB h parameter.

Special Considerations for ^FD Character Set When Using ^BB

The character set sent to the printer depends on the mode selected in parameter m.

CODABLOCK A: CODABLOCK A uses the same character set as Code 39. If any other character is used in the ^FD statement, either no bar code is printed or an error message is printed (if ^CV is active).

CODABLOCK E: The Automatic Mode includes the full ASCII set except for those characters with special meaning to the printer. Function codes or the Code 128 Subset A <nul> character can be inserted using of the ^FH command.

<fnc1> = 80 hex	**<fnc3> = 82 hex**
<fnc2> = 81 hex	**<fnc4> = 83 hex**
<nul> = 84 hex	

For any other character above 84 hex, either no bar code is printed or an error message is printed (if ^CV is active).

CODABLOCK F: CODABLOCK F uses the full ASCII set, except for those characters with special meaning to the printer. Function codes or the Code 128 Subset A <nul> character can be inserted using of the ^FH command.

<fnc1> = 80 hex	**<fnc3> = 82 hex**
<fnc2> = 81 hex	**<fnc4> = 83 hex**
<nul> = 84 hex	

^BC

Code 128 Bar Code (Subsets A, B, and C)

Description The ^BC command creates the Code 128 bar code, a high-density, variable length, continuous, alphanumeric symbology. It was designed for complexly encoded product identification.

Code 128 has three subsets of characters. There are 106 encoded printing characters in each set, and each character can have up to three different meanings, depending on the character subset being used. Each Code 128 character consists of six elements: three bars and three spaces.

- ^BC supports a fixed print ratio.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BCo,h,f,g,e,m



Important • If additional information about the Code 128 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y The interpretation line can be printed in any font by placing the font command before the bar code command.
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Parameters	Details
e = UCC check digit	*Accepted Values:* Y (turns on) or N (turns off) **Note** • Mod 103 check digit is always there. It cannot be turned on or off. Mod 10 and 103 appear together with e turned on. *Default Value:* N
m = mode	*Accepted Values:* N = no selected mode U = UCC Case Mode • More than 19 digits in ^FD or ^SN are eliminated. • Fewer than 19 digits in ^FD or ^SN add zeros to the right to bring the count to 19. This produces an invalid interpretation line. A = Automatic Mode. This analyzes the data sent and automatically determines the best packing method. The full ASCII character set can be used in the ^FD statement — the printer determines when to shift subsets. A string of four or more numeric digits causes an automatic shift to Subset C. D = New Mode (x.11.x and newer firmware) This allows dealing with UCC/EAN with and without chained application identifiers. The code starts in the appropriate subset followed by FNC1 to indicate a UCC/EAN 128 bar code. The printer automatically strips out parentheses and spaces for encoding, but prints them in the human-readable section. The printer automatically determines if a check digit is required, calculate it, and print it. Automatically sizes the human readable. *Default Value:* N

Example • This is an example of a Code 128 bar code:

CODE 128 BAR CODE



ZPL II CODE

```
^XA
^FO100,100^BY3
^BCN,100,Y,N,N
^FD123456^FS
^XZ
```

Code 128 Subsets

The Code 128 character subsets are referred to as Subset A, Subset B, and Subset C. A subset can be selected in these ways:

- A special Invocation Code can be included in the field data (^FD) string associated with that bar code.
- The desired Start Code can be placed at the beginning of the field data. If no Start Code is entered, Subset B are used.

To change subsets within a bar code, place the Invocation Code at the appropriate points within the field data (^FD) string. The new subset stays in effect until changed with the Invocation Code. For example, in Subset C, >7 in the field data changes the Subset to A.

Table 6 shows the Code 128 Invocation Codes and Start Characters for the three subsets.

Table 6 • Code 128 Invocation Characters

Invocation Code	Decimal Value	Subset A Character	Subset B Character	Subset C Character
><	62			
>0	30	>	>	
>=	94		~	
>1	95	USQ	DEL	
>2	96	FNC 3	FNC 3	
>3	97	FNC 2	FNC 2	
>4	98	SHIFT	SHIFT	
>5	99	CODE C	CODE C	
>6	100	CODE B	FNC 4	CODE B
>7	101	FNC 4	CODE A	CODE A
>8	102	FNC 1	FNC 1	FNC 1
Start Characters				
>9	103	Start Code A	(Numeric Pairs give Alpha/Numerics)	
>:	104	Start Code B	(Normal Alpha/Numeric)	
>;	105	Start Code C	(All numeric (00 - 99)	

Table 7 shows the character sets for Code 128:

Table 7 • Code 128 character sets

Value	Code A	Code B	Code C
0	SP	SP	00
1	!	!	01
2	"	"	02
3	#	#	03
4	$	$	04
5	%	%	05
6	&	&	06
7	'	'	07
8	(	(	08
9	)	)	09
10	*	*	10
11	++	++	11
12	,	,	12
13	-	-	13
14	.	.	14
15	/	/	15
16	0	0	16
17	1	1	17
18	2	2	18
19	3	3	19
20	4	4	20
21	5	5	21
22	6	6	22
23	7	7	23
24	8	8	24
25	9	9	25
26	:	:	26
27	;	;	27
28	<	<	28
29	=	=	29
30	>	>	30
31	?	?	31
32	@	@	32
33	A	A	33
34	B	B	34
35	C	C	35
36	D	D	36
37	E	E	37
38	F	F	38
39	G	G	39
40	H	H	40
41	I	I	41
42	J	J	42
43	K	K	43
44	L	L	44
45	M	M	45
46	N	N	46
47	O	O	47
48	P	P	48
49	Q	Q	49
50	R	R	50
51	S	S	51
52	T	T	52

Value	Code A	Code B	Code C
53	U	U	53
54	V	V	54
55	W	W	55
56	X	X	56
57	Y	Y	57
58	Z	Z	58
59	[	[	59
60	\	\	60
61	]	]	61
62	^	^	62
63	_	_	63
64	NUL	.	64
65	SOH	a	65
66	STX	b	66
67	ETX	c	67
68	EOT	d	68
69	ENQ	e	69
70	ACK	f	70
71	BEL	g	71
72	BS	h	72
73	HT	i	73
74	LF	j	74
75	VT	k	75
76	FF	l	76
77	CR	m	77
78	SO	n	78
79	SI	o	79
80	DLE	p	80
81	DC1	q	81
82	DC2	r	82
83	DC3	s	83
84	DC4	t	84
85	NAK	u	85
86	SYN	v	86
87	ETB	w	87
88	CAN	x	88
89	EM	y	89
90	SUB	z	90
91	ESC	{	91
92	FS	|	92
93	GS	}	93
94	RS	~	94
95	US	DEL	95
96	FNC3	FNC3	96
97	FNC2	FNC2	97
98	SHIFT	SHIFT	98
99	Code C	Code C	99
100	Code B	FNC4	Code B
101	FNC4	Code A	Code A
102	FNC1	FNC1	FNC1
103		*START (Code A)*	
104		*START (Code B)*	
105		*START (Code C)*	

Example • Figures A and B are examples of identical bar codes, and Figure C is an example of switching from Subset C to B to A, as follows:



Figure A: Subset B with no start character



Figure B: Subset B with start character

Because Code 128 Subset B is the most commonly used subset, ZPL II defaults to Subset B if no start character is specified in the data string.

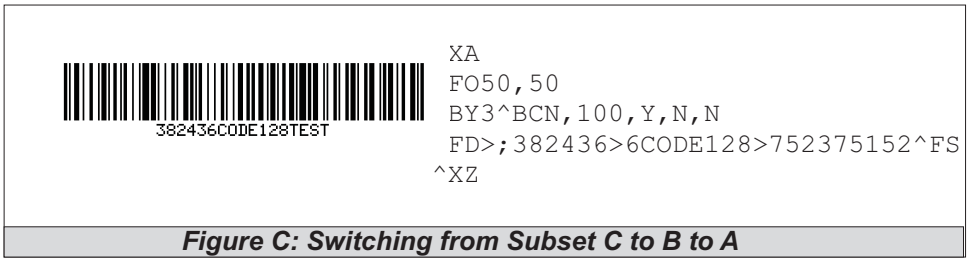


Figure C: Switching from Subset C to B to A

How ^BC Works Within a ZPL II Script

^XA – the first command starts the label format.

^FO100,75 – the second command sets the field origin at 100 dots across the x-axis and 75 dots down the y-axis from the upper-left corner.

^BCN,100,Y,N,N – the third command calls for a Code 128 bar code to be printed with no rotation (N) and a height of 100 dots. An interpretation line is printed (Y) below the bar code (N). No UCC check digit is used (N).

^FDCODE128^FS (**Figure A**) ^FD>:CODE128^FS (**Figure B**) – the field data command specifies the content of the bar code.

^XZ – the last command ends the field data and indicates the end of the label.

The interpretation line prints below the code with the UCC check digit turned off.

The ^FD command for Figure A does not specify any subset, so Subset B is used. In Figure B, the ^FD command specifically calls Subset B with the >: Start Code. Although ZPL II defaults to Code B, it is good practice to include the Invocation Codes in the command.

Code 128 – Subset B is programmed directly as ASCII text, except for values greater than 94 decimal and a few special characters that must be programmed using the invocation codes. Those characters are:

^ > ~

Example • Code 128 – Subsets A and C

Code 128, Subsets A and C are programmed in pairs of digits, 00 to 99, in the field data string. For details, see *Table 6 on page 49*.

In Subset A, each pair of digits results in a single character being encoded in the bar code; in Subset C, characters are printed as entered. Figure E below is an example of Subset A (>9 is the Start Code for Subset A).

Nonintegers programmed as the first character of a digit pair (D2) are ignored. However, nonintegers programmed as the second character of a digit pair (2D) invalidate the entire digit pair, and the pair is ignored. An extra unpaired digit in the field data string just before a code shift is also ignored.

Figure C and Figure D below are examples of Subset C. Notice that the bar codes are identical. In the program code for Figure D, the *D* is ignored and the 2 is paired with the 4.




```
^XA
^FO100,75^BY3
^BCN,100,Y,N,N
^FD>;382436^FS
^XZ
```

Figure C: Subset C with normal data




```
^XA
^FO100,75^BY3
^BCN,100,Y,N,N
^FD>;38D2436^FS
^XZ
```

Figure D: Subset C with ignored alpha character

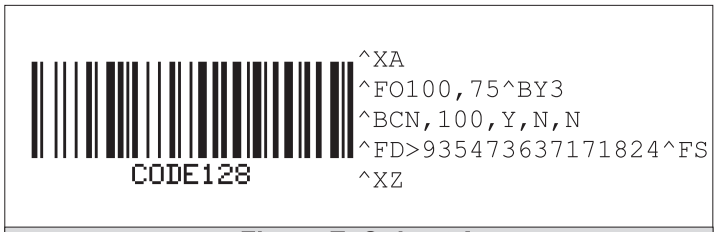


Figure E: Subset A

The UCC/EAN-128 Symbology

The symbology specified for the representation of Application Identifier data is UCC/EAN-128, a variant of Code 128, exclusively reserved to EAN International and the Uniform Code Council (UCC).

Note • t is not intended to be used for data to be scanned at the point of sales in retail outlets.

UCC/EAN-128 offers several advantages. It is one of the most complete, alphanumeric, one-dimensional symbologies available today. The use of three different character sets (A, B and C), facilitates the encoding of the full 128 ASCII character set. Code 128 is one of the most compact linear bar code symbologies. Character set C enables numeric data to be represented in a double density mode. In this mode, two digits are represented by only one symbol character saving valuable space. The code is concatenated. That means that multiple AIs and their fields may be combined into a single bar code. The code is also very reliable. Code 128 symbols use two independent self-checking features which improves printing and scanning reliability.

UCC/EAN-128 bar codes always contain a special non-data character known as function 1 (FNC 1), which follows the start character of the bar code. It enables scanners and processing software to auto-discriminate between UCC/EAN-128 and other bar code symbologies, and subsequently only process relevant data.

The UCC/EAN-128 bar code is made up of a leading quiet zone, a Code 128 start character A, B, or C, a FNC 1 character, Data (Application Identifier plus data field), a symbol check character, a stop character, and a trailing quiet zone.

UCC/EAN, UCC/128 are a couple of ways you'll hear someone refer to the code. This just indicates that the code is structured as dictated by the application identifiers that are used.

SSCC (Serial Shipping Container Code) formatted following the data structure layout for Application Identifier 00. See Table 8, *UCC Application Identifier Table on page 57*. It could be 00 which is the SSCC code. The customer needs to let us know what application identifiers are used for their bar code so we can help them.

There are several ways of writing the code to print the code to Application Identifier '00' structure.

Using N for the mode (m) parameter

Application Identifier 00 structure

Example • This example shows with application identifier 00 structure

N FOR THE M PARAMETER	ZPL II CODE
	^XA ^FO90,200^BY4 ^BCN,256,Y,N,Y,N ^FD>;>8001234512345123451234512^FS ^XZ

- >;>8' sets it to subset C, function 1
- '00' is the application identifier followed by '17 characters', the check digit is selected using the 'Y' for the (e) parameter to automatically print the 20th character.
- you are not limited to 19 characters with mode set to N

Using U for the mode (m) parameter

Example • The example shows the application identifier 00 format:

U FOR THE M PARAMETER	ZPL II CODE
	^XA ^FO90,200 ^BY4^BC,256,Y,N,,U ^FD0012345123451234512^FS ^XZ

UCC Case Mode

- Choosing U selects UCC Case mode. You will have exactly 19 characters available in ^FD.
- Subset C using FNC1 values are automatically selected.
- Check digit is automatically inserted.

Using D for the mode (m) parameter

Example • This example shows application identifier 00 format ((x.11.x and above):

D FOR THE M PARAMETER	ZPL II CODE
	^XA ^FO50,200^BCN,150,Y,N,,D ^FD(00)10084423 7449200940^FS ^XZ

(0 at end of field data is a bogus character that is inserted as a place holder for the check digit the printer will automatically insert.

- Subset C using FNC1 values are automatically selected.
- Parentheses and spaces can be in the field data. '00' application identifier, followed by 17 characters, followed by bogus check digit place holder.
- Check digit is automatically inserted. The printer will automatically calculate the check digit and put it into the bar code and interpretation line.
- The interpretation line will also show the parentheses and spaces but will strip them out from the actual bar code.

Printing the Interpretation Line

Example • This example shows printing the interpretation in a different font with firmware x.11.x and above:

INTERPRETATION LINE	ZPL II CODE
	^XA ^FO50,200 ^A0N,40,30^BCN,150,Y,N,Y ^FD>;>8001234512345123451234512^FS

The font command (^A0N,40,30) can be added and changed to alter the font and size of the interpretation line.

With firmware version older than x.10.x

- A separate text field needs to be written.
- The interpretation line needs to be turned off.
- ^A0N,50,40 is the font and size selection for the separate text field.

- You have to make sure you enter the correct check digit in the text field.
- Creating a separate text field allows you to format the interpretation line with parentheses and spaces.

FIRMWARE OLDER THAN X.10.X	ZPL II CODE
	^XA ^FO25,25 ^BCN,150,N,N,Y ^FD>;>8001234512345123451234512^FS ^FO100,190 ^A0N,50,40 ^FD(00) 1 2345123 451234512 0^FS ^XZ

Application Identifiers — UCC/EAN APPLICATION IDENTIFIER

An Application Identifier is a prefix code used to identify the meaning and the format of the data that follows it (data field).

There are AIs for identification, traceability, dates, quantity, measurements, locations, and many other types of information.

For example, the AI for batch number is 10, and the batch number AI is always followed by an alphanumeric batch code not to exceed 20-characters.

The UCC/EAN Application Identifiers provide an open standard that can be used and understood by all companies in the trading chain, regardless of the company that originally issued the codes.

Table 8 • UCC Application Identifier Table

Data Content	AI	Plus The Following Data Structure
Serial Shipping Container Code (SSCC)	00	exactly 18 digits
Shipping Container Code	01	exactly 14 digits
Batch Numbers	10	up to 20 alpha numerics
Production Date (YYMMDD)	11	exactly 6 digits
Packaging Date (YYMMDD)	13	exactly 6 digits
Sell By Date (YYMMDD)	15	exactly 6 digits
Expiration Date (YYMMDD)	17	exactly 6 digits
Product Variant	20	exactly 2 digits
Serial Number	21	up to 20 alpha numerics
HIBCC Quantity, Date, Batch and Link	22	up to 29 alpha numerics
Lot Number	23*	up to 19 alpha numerics
Quantity Each	30	
Net Weight (Kilograms)	310**	exactly 6 digits
Length, Meters	311**	exactly 6 digits
Width or Diameter (Meters)	312**	exactly 6 digits
Depths (Meters)	313**	exactly 6 digits
Area (Sq. Meters)	314**	exactly 6 digits
Volume (Liters)	315**	exactly 6 digits
Volume (Cubic Meters)	316**	exactly 6 digits
Net Weight (Pounds)	320**	exactly 6 digits
Customer PO Number	400	up to 29 alpha numerics
Ship To (Deliver To) Location Code using EAN 13 or DUNS Number with leading zeros	410	exactly 13 digits
Bill To (Invoice To) Location Code using EAN 13 or DUNS Number with leading zeros	411	exactly 13 digits
Purchase from	412	exactly 13 digits
Ship To (Deliver To) Postal Code within single postal authority	420	up to 9 alpha numerics
Ship To (Deliver To) Postal Code with 3-digit ISO Country Code Prefix	421	3 digits plus up to 9 alpha numerics

Table 8 • UCC Application Identifier Table

Data Content	AI	Plus The Following Data Structure
Roll Products - width, length, core diameter, direction and splices	8001	exactly 14 digits
Electronic Serial number for cellular mobile phone	8002	up to 20 alpha numerics

Note • Table 8 on page 57 is a partial table showing the application identifiers. For more current and complete information, search the Internet for **UCC Application Identifier**.

For date fields that only need to indicate a year and month, the day field is set to "00".

* Plus one digit for length indication.

** Plus one digit for decimal point indication.

Chaining several application identifiers (firmware x.11.x and later)

The FNC1, which is invoked by >8, is inserted just before the AI's so that the scanners reading the code sees the FNC1 and knows that an AI follows.

Example • This is an example with the mode parameter set to A (automatic):

```
^XA
^BY2,2.5,193
^FO33,400
^BCN,,N,N,N,A
^FD>;>8020401777300348610000 8535>8910001>837252^FS
^FT33,625^AEN,0,0^FD(02)04017773003486(10)0008535(91)
0001(37)252^FS
^XZ
```

Example • This is an example with the mode parameter set to U:

```
^XA
^BY3,2.5,193
^FO33,200
^BCN,,N,N,N,U
^FD>;>8020401777300348 6>8100008535>8910001>837252^FS
^FT33,455^A0N,30,30^FD(02)04017773003486(10)0008535(9
1)0001(37)252^FS
^XZ
```

Example • This is an example with the mode parameter set to D*:

```
^XA
^PON
^LH0,0
^BY2,2.5,145
^FO218,343
^BCB,,Y,N,N,D
^FD(91)0005886>8(10)0000410549>8(99)05^FS
^XZ
```

D* — When trying to print the last Application Identifier with an odd number of characters, a problem existed when printing EAN128 bar codes using Mode D. The problem was fixed in firmware 60.13.0.6.

^BD

UPS MaxiCode Bar Code

Description The ^BD command creates a two-dimensional, optically read (not scanned) code. This symbology was developed by UPS (United Parcel Service).

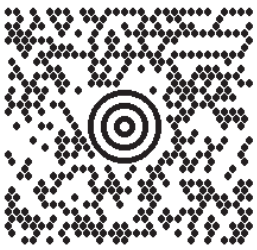
Notice that there are no additional parameters for this code and it does not generate an interpretation line. The ^BY command has no effect on the UPS MaxiCode bar code. However, the ^CV command can be activated.

Format ^BDm,n,t

This table identifies the parameters for this format:

Parameters	Details
m = mode	*Accepted Values:* 2 = structured carrier message: numeric postal code (U.S.) 3 = structured carrier message: alphanumeric postal code (non-U.S.) 4 = standard symbol, secretary 5 = full EEC 6 = reader program, secretary *Default Value:* 2
n = symbol number	*Accepted Values:* 1 to 8 can be added in a structured document *Default Value:* 1
t = total number of symbols	*Accepted Values:* 1 to 8, representing the total number of symbols in this sequence *Default Value:* 1

Example • This is an example of UPS MAXICODE - MODE 2 bar code:

UPS MAXICODE - MODE 2	ZPL II CODE
	^XA ^FO50,50 ^CVY ^BD^FH^FD001840152382802 [)>_1E01_1D961Z00004951_1DUPSN_ 1D_06X610_1D159_1D1234567_1D1/1_ 1D_1DY_1D634 ALPHA DR_ 1DPITTSBURGH_1DPA_1E_04^FS ^FO30,300^A0,30,30^FDMode2^FS ^XZ

Special Considerations for ^FD when Using ^BD

The ^FD statement is divided into two parts: a high priority message (hpm) and a low priority message (lpm). There are two types of high priority messages. One is for a U.S. Style Postal Code; the other is for a non-U.S. Style Postal Code. The syntax for either of these high priority messages must be exactly as shown or an error message is generated.

Format ^FD <hpm><lpm>

This table identifies the parameters for this format:

Parameters	Details
<hpm>= high priority message (applicable only in Modes 2 and 3)	*Accepted Values:* 0 to 9, except where noted U.S. Style Postal Code (Mode 2) **<hpm>** = aaabbbccccddddd aaa = three-digit class of service bbb = three-digit country zip code ccccc = five-digit zip code dddd = four-digit zip code extension (if none exists, four zeros (0000) must be entered) **non-U.S. Style Postal Code (Mode 3)** **<hpm>** = aaabbbcccccc aaa = three-digit class of service bbb = three-digit country zip code ccccc = six-digit zip code (A through Z or 0 to 9)
<lpm> = low priority message (only applicable in Modes 2 and 3)	GS is used to separate fields in a message (0x1D). RS is used to separate format types (0x1E). EOT is the end of transmission characters. **Message Header** []>RS **Transportation Data** **Format Header** 01GS96 **Tracking Number*** <tracking number> **SCAC*** GS<SCAC> **UPS Shipper Number** GS<shipper number> **Julian Day of Pickup** GS<day of pickup> **Shipment ID Number** GS<shipment ID number> **Package n/x** GS<n/x> **Package Weight** GS<weight> **Address Validation** GS<validation> **Ship to Street Address** GS<street address> **Ship to City** GS<city> **Ship to State** GS<state> **RS** RS **End of Message** EOT (***** Mandatory Data for UPS)

Comments

- The formatting of **<hpm>** and **<lpm>** apply only when using Modes 2 and 3. Mode 4, for example, takes whatever data is defined in the ^FD command and places it in the symbol.
- UPS requires that certain data be present in a defined manner. When formatting MaxiCode data for UPS, always use uppercase characters. When filling in the *fields* in the **<lpm>** for UPS, follow the data size and types specified in *Guide to Bar Coding with UPS*.
- If you do not choose a mode, the default is Mode 2. If you use non-U.S. Postal Codes, you probably get an error message (invalid character or message too short). When using non-U.S. codes, use Mode 3.
- ZPL II doesn't automatically change your mode based on the zip code format.
- When using special characters, such as GS, RS, or EOT, use the ^FH command to tell ZPL II to use the hexadecimal value following the underscore character (_).

^BE

EAN-13 Bar Code

Description The ^BE command is similar to the UPC-A bar code. It is widely used throughout Europe and Japan in the retail marketplace.

The EAN-13 bar code has 12 data characters, one more data character than the UPC-A code. An EAN-13 symbol contains the same number of bars as the UPC-A, but encodes a 13th digit into a parity pattern of the left-hand six digits. This 13th digit, in combination with the 12th digit, represents a country code.

- ^BE supports fixed print ratios.
- Field data (^FD) is limited to exactly 12 characters. ZPL II automatically truncates or pads on the left with zeros to achieve the required number of characters.
- When using JAN-13 (Japanese Article Numbering), a specialized application of EAN-13, the first two non-zero digits sent to the printer must be 49.

Format ^BEo,h,f,g



Important • If additional information about the EAN-13 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of an EAN-13 bar code:

EAN-13 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BEN,100,Y,N ^FD12345678^FS ^XZ

EAN-13 BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

Comments The EAN-13 bar code uses the Mod 10 check-digit scheme for error checking. For more information on Mod 10, see Appendix C in *ZPL II Programming Guide Volume Two*.

^BF

Micro-PDF417 Bar Code

Description The ^BF command creates a two-dimensional, multi-row, continuous, stacked symbology identical to PDF417, except it replaces the 17-module-wide start and stop patterns and left/right row indicators with a unique set of 10-module-wide row address patterns. These reduce overall symbol width and allow linear scanning at row heights as low as 2X.

Micro-PDF417 is designed for applications with a need for improved area efficiency but without the requirement for PDF417's maximum data capacity. It can be printed only in specific combinations of rows and columns up to a maximum of four data columns by 44 rows.

Field data (^FD) and field hexadecimal (^FH) are limited to:

- 250 7-bit characters
- 150 8-bit characters
- 366 4-bit numeric characters

Format ^BFo,h,m

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 9999 *Default Value:* value set by ^BY or 10 (if no ^BY value exists).
m = mode	*Accepted Values:* 0 to 33 (see *Micro-PDF417 Mode* on page 66) *Default Value:* 0 (see *Micro-PDF417 Mode* on page 66)

Example • This is an example of a MICRO-PDF417 bar code:

MICRO-PDF417 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY6 ^BFN,8,3 ^FDABCDEFGHIJKLMNOPQRSTUV^FS ^XZ

To encode data into a Micro-PDF417 Bar Code, complete these steps:

1. Determine the type of data to be encoded (for example, ASCII characters, numbers, 8-bit data, or a combination).
2. Determine the maximum amount of data to be encoded within the bar code (for example, number of ASCII characters, quantity of numbers, or quantity of 8-bit data characters).
3. Determine the percentage of check digits that are used within the bar code. The higher the percentage of check digits that are used, the more resistant the bar code is to damage — however, the size of the bar code increases.
4. Use the chart *Micro-PDF417 Mode* on page 66 with the information gathered from the questions above to select the mode of the bar code.

Table 9 • Micro-PDF417 Mode

Mode (M)	Number of Data Columns	Number of Data Rows	% of Cws for EC	Max Alpha Characters	Max Digits
0	1	11	64	6	8
1	1	14	50	12	17
2	1	17	41	18	26
3	1	20	40	22	32
4	1	24	33	30	44
5	1	28	29	38	55
6	2	8	50	14	20
7	2	11	41	24	35
8	2	14	32	36	52
9	2	17	29	46	67
10	2	20	28	56	82
11	2	23	28	64	93
12	2	26	29	72	105
13	3	6	67	10	14
14	3	8	58	18	26
15	3	10	53	26	38
16	3	12	50	34	49
17	3	15	47	46	67
18	3	20	43	66	96
19	3	26	41	90	132
20	3	32	40	114	167
21	3	38	39	138	202
22	3	44	38	162	237
23	4	6	50	22	32
24	4	8	44	34	49
25	4	10	40	46	67
26	4	12	38	58	85
27	4	15	35	76	111
28	4	20	33	106	155
29	4	26	31	142	208
30	4	32	30	178	261
31	4	38	29	214	313
32	4	44	28	250	366
33	4	4	50	14	20

^BI

Industrial 2 of 5 Bar Codes

Description The ^BI command is a discrete, self-checking, continuous numeric symbology. The Industrial 2 of 5 bar code has been in use the longest of the 2 of 5 family of bar codes. Of that family, the Standard 2 of 5 (^BJ) and Interleaved 2 of 5 (^B2) bar codes are also available in ZPL II.

With Industrial 2 of 5, all of the information is contained in the bars. Two bar widths are employed in this code, the wide bar measuring three times the width of the narrow bar.

- ^BI supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BIo,h,f,g

Important • If additional information about the Industrial 2 of 5 bar code, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of an Industrial 2 of 5 bar code:

INDUSTRIAL 2 OF 5 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BIN,150,Y,N ^FD123456^FS ^XZ

INDUSTRIAL 2 OF 5 BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9

Start/Stop (internal)

^BJ

Standard 2 of 5 Bar Code

Description The ^BJ command is a discrete, self-checking, continuous numeric symbology.

With Standard 2 of 5, all of the information is contained in the bars. Two bar widths are employed in this code, the wide bar measuring three times the width of the narrow bar.

- ^BJ supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BJo,h,f,g

Important • If additional information about the Standard 2 of 5 bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a Standard 2 of 5 bar code:

STANDARD 2 OF 5 BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BJN,150,Y,N ^FD123456^FS ^XZ

STANDARD 2 OF 5 BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9

Start/Stop (automatic)

^BK

ANSI Codabar Bar Code

Description The ANSI Codabar bar code is used in a variety of information processing applications such as libraries, the medical industry, and overnight package delivery companies. This bar code is also known as USD-4 code, NW-7, and 2 of 7 code. It was originally developed for retail price labeling.

Each character in this code is composed of seven elements: four bars and three spaces. Codabar bar codes use two character sets, numeric and control (start and stop) characters.

- ^BK supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BKo,e,h,f,g,k,l



Important • If additional information about the ANSI Codabar bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
e = check digit	*Fixed Value:* N
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
k = designates a start character	*Accepted Values:* A, B, C, D *Default Value:* A
l = designates stop character	*Accepted Values:* A, B, C, D *Default Value:* A

Example • This is an example of an ANSI Codabar bar code:

ANSI CODABAR BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BKN,N,150,Y,N,A,A ^FD123456^FS ^XZ

ANSI CODABAR BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9

Control Characters

\- : . $ / +

Start/Stop Characters

A B C D

^BL

LOGMARS Bar Code

Description The ^BL command is a special application of Code 39 used by the Department of Defense. LOGMARS is an acronym for Logistics Applications of Automated Marking and Reading Symbols.

- ^BL supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label. Lowercase letters in the ^FD string are converted to the supported uppercase LOGMARS characters.

Format ^BLo,h,g



Important • If additional information about the LOGMARS bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a LOGMARS bar code:

LOGMARS BAR CODE	ZPL II CODE
 12ABO	^XA ^FO100,75^BY3 ^BLN,100,N ^FD12AB^FS ^XZ

LOGMARS BAR CODE CHARACTERS

0 1 2 3 4 5 6 7 8 9

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z

\- . $ / + %

SPACE

Comments The LOGMARS bar code produces a *mandatory* check digit using Mod 43 calculations. For further information on the Mod 43 check digit, see Appendix D in *ZPL II Programming Guide Volume Two*.

^BM

MSI Bar Code

Description The ^BM command is a pulse-width modulated, continuous, non-self- checking symbology. It is a variant of the Plessey bar code (^BP).

Each character in the MSI bar code is composed of eight elements: four bars and four adjacent spaces.

- ^BM supports a print ratio of 2.0:1 to 3.0:1.
- For the bar code to be valid, field data (^FD) is limited to 1 to 14 digits when parameter e is B, C, or D. ^FD is limited to 1 to 13 digits when parameter e is A, plus a quiet zone.

Format ^BMo,e,h,f,g,e2

Important • If additional information about the MSI bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
e = check digit selection	*Accepted Values:* A = no check digits B = 1 Mod 10 C = 2 Mod 10 D = 1 Mod 11 and 1 Mod 10 *Default Value:* B
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
e2 = inserts check digit into the interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a MSI bar code:

MSI BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BMN,B,100,Y,N,N ^FD123456^FS ^XZ

MSI BAR CODE CHARACTERS
1 2 3 4 5 6 7 8 9

^B0

Aztec Bar Code Parameters

Description The ^BO command creates a two-dimensional matrtix symbology made up of square mdoules arranged around a bulls-eye pattern at the center.

Note • The Aztec bar code works with firmware v60.13.0.11A and higher.

Format ^B0a,b,c,d,e,f,g

This table identifies the parameters for this format:

Parameters	Details
a = orientation	*Accepted Values:* N = normal R = rotated I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
b = magnification factor	*Accepted Values:* 1 to 10 *Default Value:* 1 on 150 dpi printers 2 on 200 dpi printers 3 on 300 dpi printers 6 on 600 dpi printers
c = extended channel interpretation code indicator	*Accepted Values:* Y = if data contains ECICs N = if data does not contain ECICs. *Default Value:* N
d = error control and symbol size/type indicator	*Accepted Values:* 0 = default error correction level 01 to 99 = error correction percentage (minimum) 101 to 104 = 1 to 4-layer compact symbol 201 to 232 = 1 to 32-layer full-range symbol 300 = a simple Aztec “Rune” *Default Value:* 0
e = menu symbol indicator	*Accepted Values:* Y = if this symbol is to be a menu (bar code reader initialization) symbol N = if it is not a menu symbol *Default Value:* N

Parameters	Details
f = number of symbols for structured append	*Accepted Values:* 1 through 26 *Default Value:* 1
g = *optional* ID field for structured append	The ID field is a text string with 24-character maximum *Default Value:* no ID

Example • This is an example of the ^B0 command:

ZPL II CODE

```
^XA
^B0R,7,N,0,N,1,0
^FD 7. This is testing label 7^FS
^XZ
```

GENERATED LABEL



^BP

Plessey Bar Code

Description The ^BP command is a pulse-width modulated, continuous, non-self- checking symbology.

Each character in the Plessey bar code is composed of eight elements: four bars and four adjacent spaces.

- ^BP supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BPo,e,h,f,g



Important • If additional information about the Plessey bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
e = print check digit	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
h = bar code height (in dots)	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of a Plessey bar code:

PLESSEY BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BPN,N,100,Y,N ^FD12345^FS ^XZ

PLESSEY BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9 A B C D E F

^BQ

QR Code Bar Code

Description The ^BQ command produces a matrix symbology consisting of an array of nominally square modules arranged in an overall square pattern. A unique pattern at three of the symbol's four corners assists in determining bar code size, position, and inclination.

A wide range of symbol sizes is possible, along with four levels of error correction. User-specified module dimensions provide a wide variety of symbol production techniques.

QR Code Model 1 is the original specification, while QR Code Model 2 is an enhanced form of the symbology. Model 2 provides additional features and can be automatically differentiated from Model 1.

Model 2 is the recommended model and should normally be used.

This bar code is printed using field data specified in a subsequent ^FD string.

Encodable character sets include numeric data, alphanumeric data, 8-bit byte data, and Kanji characters.

Format ^BQa,b,c,d,e

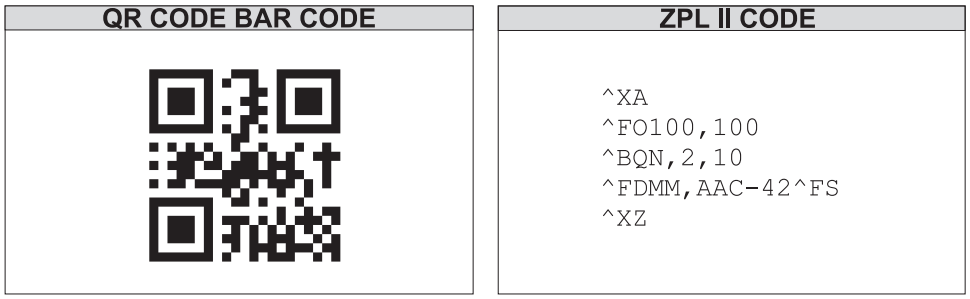
Important • If additional information about the QR Code bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
a = field position	*Fixed Value:* normal (^FW has no effect on rotation)
b = model	*Accepted Values:* 1 (original) and 2 (enhanced – recommended) *Default Value:* 2
c = magnification factor	*Accepted Values:* 1 to 10 *Default Value:* 1 on 150 dpi printers 2 on 200 dpi printers 3 on 300 dpi printers 6 on 600 dpi printers

Parameters	Details
d = H,Q,M,L	*Accepted Values:* H = ultra-high reliability level Q = high reliability level M = standard level L = high density level *Default Value:* Q = if empty M = invalid values
e = N,A,B.K	*Accepted Values:* 1 - 7 *Default Value:* 7

Example • This is an example of a QR Code bar code:



On the pages that follow are specific commands for formatting the ^BQ command with the ^FD statements that contain the information to be coded.

Considerations for ^FD When Using the QR Code:

QR Switches (formatted into the ^FD field data)

mixed mode <D>

D = allows mixing of different types of character modes in one code.

code No. <01 16>

Value = subtracted from the Nth number of the divided code (must be two digits).

No. of divisions <02 16>

Number of divisions (must be two digits).

parity data <1 byte>

Parity data value is obtained by calculating at the input data (the original input data before divided byte-by-byte through the EX-OR operation).

error correction level <H, Q, M, L>

H = ultra-high reliability level

Q = high reliability level

M = standard level (default)

L = high density level

character Mode <N, A, B, K>

N = numeric

A = alphanumeric

Bxxxx = 8-bit byte mode. This handles the 8-bit Latin/Kana character set in accordance with JIS X 0201 (character values 0x00 to 0xFF).

xxxx = number of data characters is represented by two bytes of BCD code.

K = Kanji — handles only Kanji characters in accordance with the Shift JIS system based on JIS X 0208. This means that all parameters after the character mode *K* should be 16-bit characters. If there are any 8-bit characters (such as ASCII code), an error occurs.

data character string <Data>

Follows character mode or it is the last switch in the ^FD statement.

data input <A, M>

A = Automatic Input (default). Data character string JIS8 unit, Shift JIS. When the input mode is Automatic Input, the binary codes of 0x80 to 0x9F and 0xE0 to 0xFF cannot be set.

M = Manual Input

Two types of data input mode exist: Automatic (A) and Manual (M). If A is specified, the character mode does not need to be specified. If M is specified, the character mode must be specified.

^FD Field Data (Normal Mode)

Automatic Data Input (A) with Switches

```
^FD
<error correction level>A,
<data character string>
^FS
```

Example • QR Code, normal mode with automatic data input.

GENERATED LABEL	ZPL II CODE
	^XA ^FO20,20^BQ,2,10^FDQA,0123456789ABCD 2D code^FS ^XZ 1 2 3

1	Q = error correction level
2	A, = automatic setting
3	data string character

Manual Data Input (M) with Switches

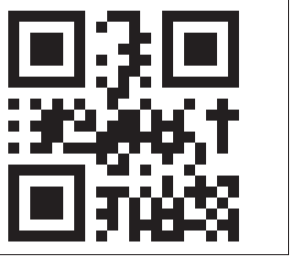
```
^FD
<error correction level>M,
<character mode><data character string>
^FS
```

Example • QR Code, normal mode with manual data input:

GENERATED LABEL	ZPL II CODE
	^XA ^FO20,20^BQ,2,10 ^FDHM,N123456789012345^FS ^XZ 1 2 3 4

1	H = error correction level (ultra-high reliability level
2	M, = input mode (manual input)
3	N = character mode (numeric data)
4	data character string

Example • QR Code, normal mode with standard reliability and manual data input:

GENERATED LABEL	ZPL II CODE
	^XA ^FO20,20^BQ,2,10^FDMM,AAC-42^FS ^XZ

1
2
3
4

1	M = error correction level (standard-high reliability level
2	M, = manual input
3	A = alphanumeric data
4	AC-42 = data character string

^FD Field Data (Mixed Mode – requires more switches)

Automatic Data Input (A) with Switches

```
^FD
<D><code No.> <No. of divisions> <parity data>,
<error correction level> A,
<data character string>,
<data character string>,
<  :  >,
<data character string n**>
^FS
```

Manual Data Input (M) with Switches

```
^FD
<code No.> <No. of divisions> <parity data>,
<error correction level> M,
<character mode 1> <data character string 1>,
<character mode 2> <data character string 2>,
< :  > <  :  >,
<character mode n> <data character string n**>
^FS
```

n** up to 200 in mixed mode

Example • QR Code, mixed mode with manual data input:

GENERATED LABEL	ZPL II CODE
	^XA ^FO,20,20^BQ,2,10 ^FDD03048F,LM,N0123456789,A12AABB,B0006qrcode^FS ^XZ

<mixed mode identifier>	D	(mixed)
<code No.>	M	(code number)
<No. of divisions>	D	(divisions)
<parity data>	M	(0x8F)
	‘	
<error correction level>	L	(high-density level)
<input mode>	M	(manual input)
	‘	
<character mode>	N	(numeric data)
<data character string>		0123456789
	‘	
<character mode>	A	(alphanumeric data)
<data character string>		12AABB
	‘	
<character mode>	B	(8-bit byte data)
	0006	(number of bytes)
<data character string>		qrcode

Example • This is an example of QR Code, mixed mode with automatic data input:

```
^XA
^FO20,20^BQ,2,10
^FDD03048F,LA,012345678912AABBqrcode^FS
^XZ
```

<mixed mode identifier>	D	(mixed)
<code No.>	M	(code number)
<No. of divisions>	D	(divisions)
<parity data>	M	(0x8F)
<error correction level>	L	(high-density level)
<input mode>	A	(automatic input)
<data character string>		012345678912AABBqrcode

^BR

RSS Bar Code

Description This is specific to Compact FLASH for the *XiIII*Plus printers.

Format ^BRa,b,c,d,e,f

This table identifies the parameters for this format:

Parameters	Details
a = orientation	*Accepted Values:* N = Normal R = Rotated I = Inverted B = Bottom-up *Default Value:* R
b = symbology type in the RSS-14 family	*Accepted Values:* 1 = RSS14 2 = RSS14 Truncated 3 = RSS14 Stacked 4 = RSS14 Stacked Omnidirectional 5 = RSS Limited 6 = RSS Expanded 7 = UPC-A 8 = UPC-E 9 = EAN-13 10 = EAN-8 11 = UCC/EAN-128 & CC-A/B 12 = UCC/EAN-128 & CC-C *Default Value:* 1
c = magnification factor	*Accepted Values:* 1 to 10 *Default Values:* 24 dot = 6, 12 dot is 3, 8 dot and lower is 2 12 dot = 6, > 8 dot is 3, 8 dot and less is 2)
d = separator height	*Accepted Values:* 1 or 2 *Default Value:* 1

Parameters	Details
e = bar code height	The bar code height only affects the linear portion of the bar code. Only UCC/EAN & CC-A/B/C. *Accepted Values:* 1 to 32000 dots *Default Value:* 25
f = the segment width (RSS expanded only)	*Accepted Values:* 2 to 22, even numbers only, in segments per line *Default Value:* 22

Example • This is an example of Symbology Type 7 - UPC-A:

```
^XA
^FO10,10^BRN,7,5,2,100^FD12345678901|this is composite
info^FS
^XZ
```

Example • This is an example of Symbology Type 1 - RSS14:

```
^XA
^FO10,10^BRN,1,5,2,100^FD12345678901|this is composite
info^FS
^XZ
```

^BS

UPC/EAN Extensions

Description The ^BS command is the two-digit and five-digit add-on used primarily by publishers to create bar codes for ISBNs (International Standard Book Numbers). These extensions are handled as separate bar codes.

The ^BS command is designed to be used with the UPC-A bar code (^BU) and the UPC-E bar code (^B9).

- ^BS supports a fixed print ratio.
- Field data (^FD) is limited to exactly two or five characters. ZPL II automatically truncates or pads on the left with zeros to achieve the required number of characters.

Format ^BSo,h,f,g

Important • If additional information about the UPC/EAN bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y

Example • This is an example of a UPC/EAN Two-digit bar code:

UPC/EAN 2-DIGIT BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BSN,100,Y,N ^FD12^FS ^XZ

UPC/EAN 2-DIGIT BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

Example • This is an example of a UPC/EAN Five-digit bar code:

UPC/EAN 5-DIGIT BAR CODE	ZPL II CODE
	^XA ^FO100,100^BY3 ^BSN,100,Y,N ^FD12345^FS ^XZ

UPC/EAN 5-DIGIT BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

Care should be taken in positioning the UPC/EAN extension with respect to the UPC-A or UPC-E code to ensure the resulting composite code is within the UPC specification.

For UPC codes, with a module width of **2** (default), the field origin offsets for the extension are:

Example • This is an example of a UPC-A:

	Supplement Origin X - Offset	Adjustment Y - Offset
Normal	209 Dots	21 Dots
Rotated	0	209 Dots

This is an example of a UPC-E:

	Supplement Origin X - Offset	Adjustment Y - Offset
Normal	122 Dots	21 Dots
Rotated	0	122 Dots

Additionally, the bar code height for the extension should be 27 dots (0.135 inches) shorter than that of the primary code. A primary UPC code height of 183 dots (0.900 inches) requires an extension height of 155 dots (0.765 inches).

Example • This example illustrates how to create a normal UPC-A bar code for the value 7000002198 with an extension equal to 04414:

UPC-A BAR CODE WITH EXTENSION	ZPL II CODE
	^XA ^FO100,100^BY3 ^BUN,137 ^FD07000002198^FS ^FO400,121 ^BSN,117 ^FD04414^FS ^XZ

^BT

TLC39 bar code

Description The ^BT bar code is the standard for the TCIF can tag telecommunications equipment.

The TCIF CLEI code, which is the Micro-PDF417 bar code, is always four columns. The firmware must determine what mode to use based on the number of characters to be encoded.

Format ^BTo,w1,r1,h1,w2,h2

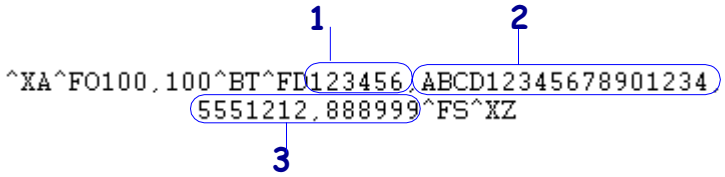
This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated I = inverted B = bottom up
w1 = width of the Code 39 bar code	*Accepted Value (in dots):* 1 to 10 *Default Value (600 dpi printers):* 4 *Default Value (200- and 300 dpi printer):* 2
r1 = wide to narrow bar width ratio the Code 39 bar code	*Accepted Values:* 2.0 to 3.0(increments of 0.1) *Default Value:* 2.0
h1 = height of the Code 39 bar code	*Accepted Values (in dots):* 1 to 9999 *Default Value (600 dpi printer):* 120 *Default Value (300 dpi printer):* 60 *Default Value (200 dpi printer):* 40
h2 = row height of the Micro-PDF417 bar ode	*Accepted Values (in dots):* 1 to 255 *Default Value (600 dpi printer):* 8 *Default Value (200- and 300 dpi printers):* 4
w2 = narrow bar width of the Micro-PDF417 bar code	*Accepted Values (in dots):* 1 to 10 *Default Value (600 dpi printer):* 4 *Default Value (200- and 300 dpi printers):* 2

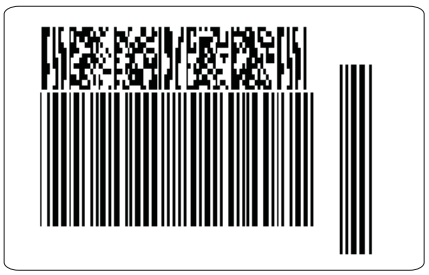
Example • TLC39 Bar Code

This is an example on how to print TLC39 bar code. The callouts identify the key components and are followed by a detailed description below:

Note • Use the command defaults to get results that are in compliance with TCIF industry standards; regardless of printhead density.



1	**ECI Number.** If the seventh character is **not** a comma, only Code 39 prints. This means if more than 6 digits are present, Code 39 prints for the first six digits (and no Micro-PDF symbol is printed). • Must be 6 digits. • Firmware generates invalid character error if the firmware sees anything but 6 digits. • This number is not padded.
2	**Serial number.** The serial number can contain up to 25 characters and is variable length. The serial number is stored in the Micro-PDF symbol. If a comma follows the serial number, then additional data is used below. • If present, must be alphanumeric (letters and numbers, no punctuation). This value is used if a comma follows the ECI number.
3	**Additional data.** If present, it is used for things such as a country code. **Note** • Data cannot exceed 150 bytes. This includes serial number commas. • Additional data is stored in the Micro-PDF symbol and appended after the serial number. A comma must exist between each maximum of 25 characters in the additional fields. • Additional data fields can contain up to 25 alphanumeric characters per field. The result is:

ZPL II CODE	GENERATED LABEL
^XA^FO100, 100^BT^FD123456, ABCd12345678901234, 5551212, 88899 ^FS^XZ	

^BU

UPC-A Bar Code

Description The ^BU command produces a fixed length, numeric symbology. It is primarily used in the retail industry for labeling packages. The UPC-A bar code has 11 data characters. The 6 dot/mm, 12 dot/mm, and 24 dot/mm printheads produce the UPC-A bar code (UPC/EAN symbologies) at 100 percent size. However, an 8 dot/mm printhead produces the UPC/EAN symbologies at a magnification factor of 77 percent.

- ^BU supports a fixed print ratio.
- Field data (^FD) is limited to exactly 11 characters. ZPL II automatically truncates or pads on the left with zeros to achieve required number of characters.

Format ^BUo,h,f,g,e

Important • If additional information about the UPC-A bar code is required, go to www.aimglobal.org.

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 9999 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
e = print check digit	*Accepted Values:* Y (yes) and N (no) *Default Value:* Y

The font style of the interpretation line depends on the modulus (width of narrow bar) selected in ^BY:

6 dot/mm printer: a modulus of 2 dots or greater prints with an OCR-B interpretation line; a modulus of 1 dot prints font A.

8 dot/mm printer: a modulus of 3 dots or greater prints with an OCR-B interpretation line; a modulus of 1 or 2 dots prints font A.

12 dot/mm printer: a modulus of 5 dots or greater prints with an OCR-B interpretation line; a modulus of 1, 2, or 3 dots prints font A.

24 dot/mm printer: a modulus of 9 dots or greater prints with an OCR-B interpretation line; a modulus of 1 to 8 dots prints font A.

Example • This is an example of a UPC-A bar code with extension:

UPC-A BAR CODE WITH EXTENSION	ZPL II CODE
	^XA ^FO100,100^BY3 ^BUN,137 ^FD07000002198^FS ^FO400,121 ^BSN,117 ^FD04414^FS ^XZ

Comments The UPC-A bar code uses the Mod 10 check digit scheme for error checking. For further information on Mod 10, see Appendix C in *ZPL II Programming Guide Volume Two*.

^BX

Data Matrix Bar Code

Description The ^BX command creates a two-dimensional matrix symbology made up of square modules arranged within a perimeter finder pattern.

The ability to create a rectangular Datamatrix bar code is not available as a ZPL coding option.

Format ^BXo,h,s,c,r,f

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = dimensional height of individual symbol elements	*Accepted Values:* 1 to the width of the label The individual elements are square — this parameter specifies both module and row height. If this parameter is zero (or not given), the h parameter (bar height) in ^BY is used as the approximate symbol height.
s = quality level	*Accepted Values:* 0, 50, 80, 100, 140, 200 *Default Value:* 0 *Quality* refers to the amount of data that is added to the symbol for error correction. The AIM specification refers to it as the ECC value. ECC 50, ECC 80, ECC 100, and ECC 140 use convolution encoding; ECC 200 uses Reed-Solomon encoding. For new applications, ECC 200 is recommended. ECC 000-140 should be used only in closed applications where a single party controls both the production and reading of the symbols and is responsible for overall system performance.
c = columns to encode	*Accepted Values:* 9 to 49 Odd values only for quality 0 to 140 (10 to 144); even values only for quality 200.

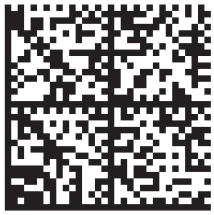
Parameters	Details
r = rows to encode	*Accepted Values:* 9 to 49 Odd values only for quality 0 to 140 (10 to 144); even values only for quality 200. The number of rows and columns in the symbol is automatically determined. You might want to force the number of rows and columns to a larger value to achieve uniform symbol size. In the current implementation, quality 0 to 140 symbols are square, so the larger of the rows or columns supplied are used to force a symbol to that size. If you attempt to force the data into too small of a symbol, no symbol is printed. If a value greater than 49 is entered, the rows or columns value is set to zero and the size is determined normally. If an even value is entered, it generates INVALID-P (invalid parameter). If a value less than 9 but not 0, or if the data is too large for the forced size, no symbol prints; if ^CV is active, INVALID-L prints.
f = format ID (0 to 6) — not used with quality set at 200	*Accepted Values:* 1 = field data is numeric + space (0..9,") – No \&" 2 = field data is uppercase alphanumeric + space (A..Z,'') – No \&'' 3 = field data is uppercase alphanumeric + space, period, comma, dash, and slash (0..9,A..Z,".-/") 4 = field data is upper-case alphanumeric + space (0..9,A..Z,'') – no \&'' 5 = field data is full 128 ASCII 7-bit set 6 = field data is full 256 ISO 8-bit set *Default Value:* 6

Table 10 • Maximum Field Sizes

ECC LEVEL	ID = 1	ID = 2	ID = 3	ID = 4	ID = 5	ID = 6
0	596	452	394	413	310	271
50	457	333	291	305	228	200
80	402	293	256	268	201	176
100	300	218	190	200	150	131
140	144	105	91	96	72	63

Maximum Field Sizes

Example • This is an example of a Data Matrix bar code:

DATA MATRIX BAR CODE	ZPL II CODE
	^XA ^FO100,100 ^BXN,10,200 ^FDZEBRA TECHNOLOGIES CORP 333 CORPORATE WOODS PKWY VERNON HILLS, ILLINOIS 60061-3109^FS ^XZ

Effects of ^BY on ^BX

w = module (no effect — see dimensions of individual symbol elements)

r = ratio (no effect)

h = height of symbol

If the dimensions of individual symbol elements are not specified in the ^BD command, the height of symbol value is divided by the required rows/columns, rounded, limited to a minimum value of one, and used as the dimensions of individual symbol elements.

Field Data (^FD) for ^BX

Quality 000 to 140

- The \& and || can be used to insert carriage returns, line feeds, and the backslash, similar to the PDF417. Other characters in the control character range can be inserted only by using ^FH. Field data is limited to 596 characters for quality *0* to *140*. Excess field data causes no symbol to print; if ^CV is active, INVALID-L prints. The field data must correspond to a user-specified format ID or no symbol prints; if ^CV is active, INVALID-C prints.
- The maximum field sizes for quality *0* to *140* symbols are shown in the table in the **g** parameter.

Quality 200

- If more than 3072 characters are supplied as field data, it is truncated to 3072 characters. This limits the maximum size of a numeric Data Matrix symbol to less than the 3116 numeric characters that the specification would allow. The maximum alphanumeric capacity is 2335 and the maximum 8-bit byte capacity is 1556.
- If ^FH is used, field hexadecimal processing takes place before the escape sequence processing described below.
- The underscore is the default escape sequence control character for quality 200 field data. A different escape sequence control character can be selected by using parameter g in the ^BX command.

The input string escape sequences can be embedded in quality 200 field data using the ASCII 95 underscore character (_) or the character entered in parameter g:

- _X is the shift character for control characters (e.g., _@=NUL,_G=BEL,_0 is PAD)
- _1 to _3 for FNC characters 1 to 3 (explicit FNC4, upper shift, is not allowed)
- FNC2 (Structured Append) must be followed by nine digits, composed of three three-digit numbers with values between 1 and 254, that represent the symbol sequence and file identifier (for example, symbol 3 of 7 with file ID 1001 is represented by 2214001001)
- 5NNN is code page NNN where NNN is a three-digit code page value (for example, Code Page 9 is represented by _5009)
- _dNNN creates ASCII decimal value NNN for a code word (must be three digits)
- _ in data is encoded by __ (two underscores)

^BY

Bar Code Field Default

Description The ^BY command is used to change the default values for the module width (in dots), the wide bar to narrow bar width ratio and the bar code height (in dots). It can be used as often as necessary within a label format.

Format ^BYw,r,h

This table identifies the parameters for this format:

Parameters	Details
w = module width (in dots)	*Accepted Values:* 1 to 10 *Initial Value at power-up:* 2
r = wide bar to narrow bar width ratio	*Accepted Values:* 2.0 to 3.0, in 0.1 increments This parameter has no effect on fixed-ratio bar codes.
h = bar code height (in dots)	*Accepted Values:* 2.0 to 3.0, in 0.1 increments *Initial Value at power-up:* 10

For parameter r, the actual ratio generated is a function of the number of dots in parameter w, module width. See the table on the next page.

Example • Set module width (w) to 9 and the ratio (r) to 2.4. The width of the narrow bar is 9 dots wide and the wide bar is 9 by 2.4, or 21.6 dots. However, since the printer rounds out to the nearest dot, the wide bar is actually printed at 22 dots.

This produces a bar code with a ratio of 2.44 (22 divided by 9). This ratio is as close to 2.4 as possible, since only full dots are printed.

Module width and height (w and h) can be changed at anytime with the ^BY command, regardless of the symbology selected.

Table 11 • Shows module width ratios in dots

Ratio Selected (r)	Module Width in Dots (w)									
	1	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**
2.0	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2:1
2.1	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2:1	2.1:1
2.2	2:1	2:1	2:1	2:1	2.2:1	2.16:1	2.1:1	2.12:1	2.1:1	2.2:1
2.3	2:1	2:1	2.3:1	2.25:1	2.2:1	2.16:1	2.28:1	2.25:1	2.2:1	2.3:1
2.4	2:1	2:1	2.3:1	2.25:1	2.4:1	2.3:1	2.28:1	2.37:1	2.3:1	2.4:1
2.5	2:1	2.5:1	2.3:1	2.5:1	2.4:1	2.5:1	2.4:1	2.5:1	2.4:1	2.5:1
2.6	2:1	2.5:1	2.3:1	2.5:1	2.6:1	2.5:1	2.57:1	2.5:1	2.5:1	2.6:1
2.7	2:1	2.5:1	2.6:1	2.5:1	2.6:1	2.6:1	2.57:1	2.65:1	2.6:1	2.7:1
2.8	2:1	2.5:1	2.6:1	2.75:1	2.8:1	2.6:1	2.7:1	2.75:1	2.7:1	2.8:1
2.9	2:1	2.5:1	2.6:1	2.75:1	2.8:1	2.8:1	2.85:1	2.87:1	2.8:1	2.9:1
3.0	3:1	3:1	3:1	3:1	3:1	3:1	3:1	3:1	3:1	3:1

Comments Once a ^BY command is entered into a label format, it stays in effect until another ^BY command is encountered.

^BZ

POSTNET Bar Code

Description The POSTNET bar code is used to automate the handling of mail. POSTNET uses a series of five bars, two tall and three short, to represent the digits 0 to 9.

- ^BZ supports a print ratio of 2.0:1 to 3.0:1.
- Field data (^FD) is limited to the width (or length, if rotated) of the label.

Format ^BZo,h,f,g

Important • If additional information about the POSTNET bar code is required, go to www.aimglobal.org, or contact the United States Postal Service and ask for Publication 25 — Designing Letter Mail, which includes a full specification for POSTNET. You can also download Publication 25 from:

http://pe.usps.gov/cpim/ftp/pubs/pub25/pub25.pdf

This table identifies the parameters for this format:

Parameters	Details
o = orientation	*Accepted Values:* N = normal R = rotated 90 degrees (clockwise) I = inverted 180 degrees B = read from bottom up, 270 degrees *Default Value:* current ^FW value
h = bar code height (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* value set by ^BY
f = print interpretation line	*Accepted Values:* Y (yes) or N (no) *Default Value:* N
g = print interpretation line above code	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Important • This is an example of a POSTNET bar code:

POSTNET BAR CODE	ZPL II CODE
 12345	^XA ^FO100,100^BY3 ^BZN,40,Y,N ^FD12345^FS ^XZ

POSTNET BAR CODE CHARACTERS
0 1 2 3 4 5 6 7 8 9

^CC

Change Carets

Description The ^CC command is used to change the format command prefix. The default prefix is the caret (^).

Format ^CCx

This table identifies the parameters for this format:

Parameters	Details
x = caret character change	*Accepted Values:* any ASCII character *Default Value:* a parameter is required. If a parameter is not entered, the next character received is the new prefix character.

Example • This is an example of how to change the format prefix to from a ^ to a /:

```
^XA
^CC/
/XZ
```

Comments In the above example, the forward slash (/) is set at the new prefix. Note the /XZ ending tag uses the new designated prefix character (/).

~CC

Change Carets

Description The ~CC command is used to change the format command prefix. The default prefix is the caret (^).

Format ~CCx

This table identifies the parameters for this format:

Parameters	Details
x = caret character change	*Accepted Values:* any ASCII character *Default Value:* a parameter is required. If a parameter is not entered, the next character received is the new prefix character.

Example • This is an example of how to change the command prefix from ~ to a /:

```
~CC/
/XA/JUS/XZ
```

^CD ~CD

Change Delimiter

Description The ^CD and ~CD commands are used to change the delimiter character. This character is used to separate parameter values associated with several ZPL II commands. The default delimiter is a comma (,).

Format ^CDa or ~CDa

This table identifies the parameters for this format:

Parameters	Details
a = delimiter character change	*Accepted Values:* any ASCII character *Default Value:* a parameter is required. If a parameter is not entered, the next character received is the new prefix character.

Example • This shows how to change the character delimiter to a period (.) :

```
^XA
^CD.
^XZ
```

- To save, the JUS command is required. Here is an example using JUS:

```
~CD.
^XA^JUS^XZ
```

^CF

Change Alphanumeric Default Font

Description The ^CF command sets the default font used in your printer. You can use the ^CF command to simplify your programs.

Format ^CFf,h,w

This table identifies the parameters for this format:

Parameters	Details
f = specified default font	*Accepted Values:* A through Z and 0 to 9 *Initial Value at power-up:* A
h = individual character height (in dots)	*Accepted Values:* 0 to 32000 *Initial Value at power-up:* 9
w = individual character width (in dots)	*Accepted Values:* 0 to 32000 *Initial Value at power-up:* 5 or last permanent saved value

Parameter f specifies the default font for every alphanumeric field. Parameter h is the default height for every alphanumeric field, and parameter w is the default width value for every alphanumeric field.

The default alphanumeric font is A. If you do not change the alphanumeric default font and do not use any alphanumeric field command (^AF) or enter an invalid font value, any data you specify prints in font A.

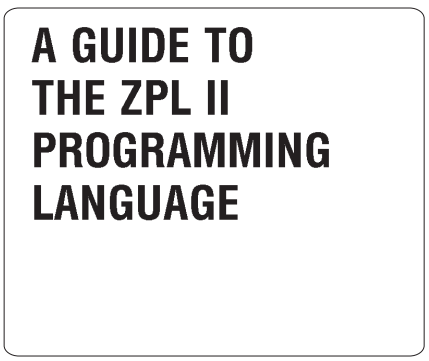
Defining only the height or width forces the magnification to be proportional to the parameter defined. If neither value is defined, the last ^CF values given or the default ^CF values for height and width are used.

Example • This is an example of ^CF code and the result of the code:

ZPL II CODE

```
^XA
^CF0,89
^FO20,50
^FDA GUIDE TO^FS
^FO20,150
^FDTHE ZPL II^FS
^FO20,250
^FDPROGRAMMING^FS
^FO20,350
^FDLANGUAGE^FS
^XZ
```

GENERATED LABEL



Comments Any font in the printer, including downloaded fonts, EPROM stored fonts, and fonts A through Z and 0 to 9, can also be selected with ^CW.

^CI

Change International Font

Description Zebra printers can print fonts using international character sets: U.S.A.1, U.S.A.2, UK, Holland, Denmark/Norway, Sweden/Finland, Germany, France 1, France 2, Italy, Spain, and several other sets.

ZPL II follows the ISO standards for international characters.

The ^CI command enables you to call up the international character set you want to use for printing. You can mix character sets on a label.

This command allows character remapping. Any character within a font can be remapped to a different numerical position.

Format ^CIa,s1,d1,s2,d2,...

This table identifies the parameters for this format:

Parameters	Details
a = desired character set	*Accepted Values:* 0 = U.S.A. 1 1 = U.S.A. 2 2 = U.K. 3 = Holland 4 = Denmark/Norway 5 = Sweden/Finland 6 = Germany 7 = France 1 8 = France 2 9 = Italy 10 = Spain 11 = miscellaneous 12 = Japan (ASCII with Yen symbol) 13 = IBM Code Page 850 (see page 34) 14 = 16-bit (Unicode) encoded scalable fonts * 15 = Shift-JIS for scalable Japanese fonts ** 16 = EUC-Kanji for scalable fonts 17 = Unicode (for Unicode-encoded fonts) 18 to 23 = Reserved 24 = 8-bit access to Unicode-encoded fonts 25 = Reserved 26 = Asian fonts with ASCII Transparency *** *Initial Value at power-up:* 0
s1 = source 1 (character output image)	*Accepted Values:* decimals 0 to 255
d1 = destination 1 (character input)	*Accepted Values:* decimals 0 to 255
s2 = source 2 (character output image)	*Accepted Values:* decimals 0 to 255
d2 = destination 2 (character input)	*Accepted Values:* decimals 0 to 255
... = continuation of pattern	Up to 256 source and destination pairs can be entered in this command.

The encoding is controlled by the conversion table (.DAT). The table generated by ZTools™ is the TrueType font's internal encoding (Unicode).

**Shift-JIS encoding converts Shift-JIS to JIS and then looks up the JIS conversion in JIS.DAT. This table must be present for Shift-JIS to function.

***Now supports ASCII transparency for Asian fonts. 7F and less are treated as single byte characters. 80 to FE is treated as the first byte of a 2 byte character 8000 to FEFF in the encoding table for Unicode. Entire range is available for double byte characters.

Example • This example remaps the Euro symbol (36) decimal to the dollar sign value (21) decimal. When the dollar sign character is sent to the printer, the Euro symbol prints.

```
^CI0,36,21
```

The font selected determines the shape and resolution of the printed symbol.

International Character Sets

Hex	23	30	40	5B	5C	5D	5E	60	7B	7C	7D	7E
CI0	#	0	@	[	¢	]	^	`	{	¦	}	~
CI1	#	0	@	⅓	¢	⅔	^	`	¼	½	¾	~
CI2	£	0	@	[	¢	]	^	`	{	¦	}	~
CI3	ƒ	0	§	[	IJ	]	^	`	{	ij	}	~
CI4	#	0	@	Æ	Ø	Å	^	`	æ	ø	å	~
CI5	Ü	0	É	Ä	Ö	Å	Ü	é	ä	ö	å	ü
CI6	#	0	§	Ä	Ö	Ü	^	`	ä	ö	ü	ß
CI7	£	0	à	[	ç	]	^	`	é	¦	ù	è
CI8	#	0	à	â	ç	ê	î	ô	é	ù	è	û
CI9	£	0	§	[	ç	é	^	ù	à	ò	è	ì
CI10	#	0	§	¡	Ñ	¿	^	`	{	ñ	ç	~
CI11	£	0	É	Ä	Ö	Ü	^	ä	ë	ï	ö	ü
CI12	#	0	@	[	¥	]	^	`	{	¦	}	~
CI13	#	0	@	[	\	]	^	`	{	¦	}	~

Note • `^CI 13 =` US keyboard

Comments The *space* character cannot be remapped for any font.

^CM

Change Memory Letter Designation

Description The ^CM command allows you to reassign a letter designation to the printer's memory devices. If a format already exists, you can reassign the memory device to the corresponding letter without forcing, altering, or recreating the format itself.

Using this command affects every subsequent command that refers to specific memory locations.

Format ^CMa,b,c,d

This table identifies the parameters for this format:

Parameters	Details
a = memory alias letter designation	*Accepted Values:* B:, E:,R:, A:, and NONE *Default Value:* B:
b = memory alias letter designation	*Accepted Values:* B:, E:,R:, A:, and NONE *Default Value:* E:
c = memory alias letter designation	*Accepted Values:* B:, E:,R:, A:, and NONE *Default Value:* R:
d = memory alias letter designation	*Accepted Values:* B:, E:,R:, A:, and NONE *Default Value:* A:

Comments If two or more parameters specify the same letter designator, all letter designators are set to their default values.

If any of the parameters are out of specification, the command is ignored.

Example • This example designates letter E: to point to the B: memory device, and the letter B: to point to the E: memory device.

```
^XA
^CME,B,R,A
^JUS
^XA
```

Comments It is recommended that after entering the ^CM command, ^JUS is entered to save changes to EEPROM. Any duplicate parameters entered reset the letter designations back to the default.

^CO

Cache On

Description The ^CO command is used to change the size of the character cache. By definition, a *character cache* (referred to as cache) is a portion of the DRAM reserved for storing scalable characters. All printers have a default 22K cache that is always turned on. The maximum single character size that can be stored, without changing the size of the cache, is 450 dots by 450 dots.

There are two types of fonts used in Zebra printers: bitmapped and scalable. Letters, numbers, and symbols in a bitmapped font have a fixed size (for example: 10 points, 12 points, 14 points). By comparison, scalable fonts are not fixed in size.

Because their size is fixed, bitmapped fonts can be moved quickly to the label. In contrast, scalable fonts are much slower because each character is built on an as-needed basis before it is moved to the label. By storing scaled characters in a cache, they can be recalled at a much faster speed.

The number of characters that can be stored in the cache depends on two factors: the size of the cache (memory) and the size of the character (in points) being saved. The larger the point size, the more space in the cache it uses. The default cache stores every scalable character that is requested for use on a label. If the same character, with the same rotation and size is used again, it is quickly retrieved from cache.

It is possible that after a while the print cache could become full. Once the cache is full, space for new characters is obtained by eliminating an existing character from the print cache. Existing characters are eliminated by determining how often they have been used. This is done automatically. For example, a 28-point *Q* that was used only once would be a good candidate for elimination from the cache.

Maximum size of a single print cache character is 1500 dots by 1500 dots. This would require a cache of 300K.

When the cache is too small for the desired style, smaller characters might appear but larger characters do not. If possible, increase the size of the cache.

Format ^COa,b,c

This table identifies the parameters for this format:

Parameters	Details
a = cache on	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y
b = amount of additional memory to be added to cache (in K)	*Accepted Values:* any size up to total memory available *Default Value:* 40
c = cache type	*Accepted Values:* 0 = cache buffer (normal fonts) 1 = internal buffer (recommended for Asian fonts) *Default Value:* 0

Example • To resize the print cache to 62K, assuming a 22K existing cache:

```
^COY,40
```

To resize the print cache to 100K, assuming a 22K existing cache:

```
^COY,78
```

Print Cache Performance

For printing large characters, memory added to the cache by the ^CO command is not physically added to the 22K cache already in the printer. In the second example above, the resulting 100K cache is actually two separate blocks of memory, 22K and 78K.

Because large characters need contiguous blocks of memory, a character requiring a cache of 90K would not be completely stored because neither portion of the 100K cache is big enough. Therefore, if large characters are needed, the ^CO command should reflect the actual size of the cache you need.

Increasing the size of the cache improves the performance in printing scalable fonts. However, the performance decreases if the size of the cache becomes large and contains too many characters. The performance gained is lost because of the time involved searching cache for each character.

Comments The cache can be resized as often as needed. Any characters in the cache when it is resized are lost. Memory used for the cache reduces the space available for label bitmaps, graphic, downloaded fonts, et cetera.

Some Asian fonts require an internal working buffer that is much larger than the normal cache. Since most fonts do not require this larger buffer, it is now a selectable configuration option. Printing with the Asian fonts greatly reduces the printer memory available for labels, graphics, fonts, and formats.

^CT ~CT

Change Tilde

Description The ^CT and ~CT commands are used to change the control command prefix. The default prefix is the tilde (~).

Format ^CTa or ~CTa

This table identifies the parameters for this format:

Parameters	Details
a = change control command character	*Accepted Values:* any ASCII character *Default Value:* a parameter is required. If a parameter is not entered, the next character received is the new control command character.

Example • This is an example of how to change the control command prefix from a ^, to a ~:

```
^XA
^CT+
^XZ
+DGR:GRAPHIC.GRF,04412,010
```

^CV

Code Validation

Description The ^CV command acts as a switch to turn the code validation function on and off. When this command is turned on, all bar code data is checked for these error conditions:

- character not in character set
- check-digit incorrect
- data field too long (too many characters)
- data field too short (too few characters)
- parameter string contains incorrect data or missing parameter

When invalid data is detected, an error message and code is printed in reverse image in place of the bar code. The message reads INVALID - X where X is one of these error codes:

C = character not in character set

E = check-digit incorrect

L = data field too long

S = data field too short

P = parameter string contains incorrect data
(occurs only on select bar codes)

Once turned on, the ^CV command remains active from format to format until turned off by another ^CV command or the printer is turned off. The command is not permanently saved.

Format ^CVa

This table identifies the parameters for this format:

Parameters	Details
a = code validation	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • The examples below show the error labels ^CVY generates when incorrect field data is entered. Compare the letter following *INVALID* – to the listing on the previous page.

ZPL II CODE	GENERATED LABEL
^XA ^CVY ^FO50,50 ^BEN,100,Y,N ^FD97823456 890^FS ^XZ	INVALID - C
^XA ^CVY ^FO50,50 ^BEN,100,Y,N ^FD9782345678907^FS ^XZ	INVALID - E
^XA ^CVY ^FO50,50 ^BEN,100,Y,N ^FD97823456789081^FS ^XZ	INVALID - L
^XA ^CVY ^FO50,50 ^BEN,100,Y,N ^FD97823456789^FS ^XZ	INVALID - S
^XA ^CVY ^FO50,50 ^BQN2,3 ^FDHM,BQRCODE-22^FS ^XZ	INVALID - P

Comments If more than one error exists, the first error detected is the one displayed.

The ^CV command tests the integrity of the data encoded into the bar code. It is not used for (or to be confused with) testing the scan-integrity of an image or bar code.

^CW

Font Identifier

Description All built-in fonts are referenced using a one-character identifier. The ^CW command assigns a single alphanumeric character to a font stored in DRAM, memory card, EPROM, or Flash.

If the assigned character is the same as that of a built-in font, the downloaded font is used in place of the built-in font. The new font is printed on the label wherever the format calls for the built-in font. If used in place of a built-in font, the change is in effect only until power is turned off.

If the assigned character is different, the downloaded font is used as an additional font. The assignment remains in effect until a new command is issued or the printer is turned off.

Format ^CWa,d:o.x

This table identifies the parameters for this format:

Parameters	Details
a = letter of existing font to be substituted, or new font to be added	*Accepted Values:* A through Z and 0 to 9 *Default Value:* a one-character entry is required
d = device to store font in (optional)	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = name of the downloaded font to be substituted for the built-in, or as an additional font	*Accepted Values:* any name up to 8 characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .FNT

Example • These examples show how to use:

- MYFONT.FNT stored in DRAM whenever a format calls for Font A:

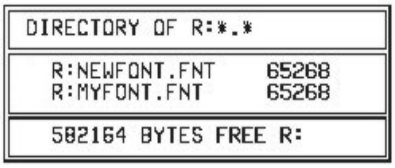
```
^XA
^CWA,R:MYFONT.FNT
^XZ
```

- MYFONT.FNT stored in DRAM as additional Font Q:

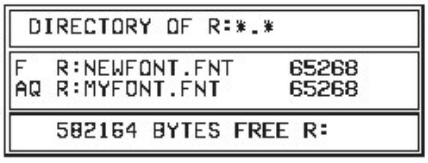
```
^XA
^CWQ,R:MYFONT.FNT
^XZ
```

- NEWFONT.FNT stored in DRAM whenever a format calls for font F:

```
^XA
^CWF,R:NEWFONT.FNT
^XZ
```



Label Listing Before Assignment



Label Listing After Assignment

~DB

Download Bitmap Font

Description The ~DB command sets the printer to receive a downloaded bitmap font and defines native cell size, baseline, space size, and copyright.

This command consists of two portions, a ZPL II command defining the font and a structured data segment that defines each character of the font.

Format ~DBd:o.x,a,h,w,base,space,#char,©,data

This table identifies the parameters for this format:

Parameters	Details
d = drive to store font	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = name of font	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .FNT
a = orientation of native font	*Fixed Value:* normal
h = maximum height of cell (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* a value must be specified
w = maximum width of cell (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* a value must be specified
base = dots from top of cell to character baseline	*Accepted Values:* 0 to 32000 *Default Value:* a value must be specified
space = width of space or non-existent characters	*Accepted Values:* 0 to 32000 *Default Value:* a value must be specified
#char = number of characters in font	*Accepted Values:* 1 to 256 (must match the characters being downloaded) *Default Value:* a value must be specified

Parameters	Details
© = copyright holder	*Accepted Values:* 1 to 63 alphanumeric characters *Default Value:* a value must be specified

data = structured ASCII data that defines each character in the font	The # symbol signifies character code parameters, which are separated with periods. The character code is from 1 to 4 characters to allow for large international character sets to be downloaded to the printer.
	The data structure is:
	#xxxx.h.w.x.y.i.data
	#xxxx = character code
	h = bitmap height (in dot rows)
	w = bitmap width (in dot rows)
	x = x-offset (in dots)
	y = y-offset (in dots)
	i = typesetting motion displacement (width, including inter character gap of a particular character in the font)
	data = hexadecimal bitmap description

Example • This is an example of how to use the ~DB command. It shows the first two characters of a font being downloaded to DRAM.

```
~DBR:TIMES.FNT,N,5,24,3,10,2,ZEBRA 1992,
#0025.5.16.2.5.18.
OOFF
OOFF
FFOO
FFOO
FFFF
#0037.4.24.3.6.26.
OOFFOO
OFOOFO
OFOOFO
OOFFOO
```

~DE

Download Encoding

Description The standard encoding for TrueType Windows® fonts is always Unicode. The ZPL II field data must be converted from some other encoding to Unicode that the Zebra printer understands. The required translation tables are provided with ZTools and downloaded with the ~DE command.

Format ~DEd:o.x,s,data

This table identifies the parameters for this format:

Parameters	Details
d = location of table	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = name of table	*Accepted Values:* any valid name, up to 8 characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .DAT
s = table size	*Accepted Values:* the number of memory bytes required to hold the Zebra downloadable format of the font *Default Value:* if an incorrect value or no value is entered, the command is ignored
data = data string	*Accepted Values:* a string of ASCII hexadecimal values *Default Value:* if no data is entered, the command is ignored

Example • This is an example of how to download the required translation table:

```
~DER:JIS.DAT,27848,300021213001...
```

(27848 two-digit hexadecimal values)

Comments For more information on ZTools, see the program documentation included with the software.

^DF

Download Format

Description The ^DF command saves ZPL II format commands as text strings to be later merged using ^XF with variable data. The format to be stored might contain field number (^FN) commands to be referenced when recalled.

While use of stored formats reduces transmission time, no formatting time is saved—this command saves ZPL II as text strings formatted at print time.

Enter the ^DF stored format command immediately after the ^XA command, then enter the format commands to be saved.

Format ^DFd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = device to store image	*Accepted Value:* R:, E:, B:, and A: *Default Value:* R:
o = image name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .ZPL

~DG

Download Graphics

Description The ~DG command downloads an ASCII Hex representation of a graphic image. If .GRF is not the specified file extension, .GRF is automatically appended.

For more saving and loading options when downloading files, see *~DY* on page 131.

Format ~DGd:o.x,t,w,data

This table identifies the parameters for this format:

Parameters	Details
d = device to store image	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = image name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .GRF
t = total number of bytes in graphic	See the formula below.
w = number of bytes per row	See the formula below.
data = ASCII hexadecimal string defining image	The data string defines the image and is an ASCII hexadecimal representation of the image. Each character represents a horizontal nibble of four dots.

Example • The key for these examples is as follows:

x = width of the graphic in millimeters
y = height of the graphic in millimeters
z = dots/mm = print density of the printer being programmed
8 = bits/byte

Example • The t parameter can be determined using this formula:

$$\frac{xz}{8} \times yz = totalbytes$$

Example • For example, to determine the correct t parameter for a graphic 8 mm wide, 16 mm high, and a print density of 8 dots/mm, the formula works this way:

$$\left(\frac{8 \times 8}{8}\right) \times (16 \times 8) = \text{bytes}$$
$$8 \times 128 = 1024$$
$$t = 1024$$

Raise any portion of a byte to the next whole byte.

Use this formula to determine the w parameter (the width in terms of bytes per row):

$$\left(\frac{xz}{8}\right) = \text{totalbytes/row}$$
$$w = 8$$

For example, to determine the correct w parameter for a graphic 8 mm wide and a print density of 8 dots/mm, the formula works this way:

$$\left(\frac{8 \times 8}{8}\right) = 8\text{bytes}$$
$$w = 8$$

Raise any portion of a byte to the next whole byte.

Parameter w is the first value in the t calculation.

The data parameter is a string of hexadecimal numbers sent as a representation of the graphic image. Each hexadecimal character represents a horizontal nibble of four dots. For example, if the first four dots of the graphic image are white and the next four black, the dot-by-dot binary code is 00001111. The hexadecimal representation of this binary value is 0F. The entire graphic image is coded in this way, and the complete graphic image is sent as one continuous string of hexadecimal values.

Comments Do not use spaces or periods when naming your graphics. Always use different names for different graphics.

If two graphics with the same name are sent to the printer, the first graphic is erased and replaced by the second graphic.

~DN

Abort Download Graphic

Description After decoding and printing the number of bytes in parameter `t` of the `~DG` command, the printer returns to normal Print Mode. Graphics Mode can be aborted and normal printer operation resumed by using the `~DN` command.

Format `~DN`

Comments If you need to stop a graphic from downloading, you should abort the transmission from the host device. To clear the `~DG` command, however, you must send a `~DN` command.

~DS

Download Scalable Font

Description The ~DS command is used to set the printer to receive a downloadable scalable font and defines the size of the font in bytes. ~DS is used for downloading Intellifont data to the printer. For downloading TrueType fonts, see *~DT* on page 129.

The ~DS command, and its associated parameters, is the result of converting a vendor-supplied font for use on a Zebra printer. To convert this font use the ZTools utility.

Format ~DSd:o.x,s,data

This table identifies the parameters for this format:

Parameters	Details
d = device to store image	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = image name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .FNT
s = size of font in bytes	*Fixed Value:* this number is generated by ZTools and should not be changed
data = ASCII hexadecimal string that defines font	*Fixed Value:* this number is generated by ZTools and should not be changed

Example • This example shows the first three lines of a scalable font that was converted using the ZTools program and is ready to be downloaded to the printer. If necessary, the destination and object name can be changed.

```
~DSB:CGTIMES.FNT,37080,
OOFFOOFFOOFFOOFF
FFOAECB28FFFOOFF
```

Comments Downloaded scalable fonts are not checked for integrity. If they are corrupt, they cause unpredictable results at the printer.

~DT

Download TrueType Font

Description Use ZTools to convert a TrueType font to a Zebra-downloadable format. ZTools creates a downloadable file that includes a ~DT command. For information on converting and downloading Intellifont information, see the ~DS command on page 128.

Format ~DTd:o.x,s,data

This table identifies the parameters for this format:

Parameters	Details
d = font location	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = font name	*Accepted Values:* any valid TrueType name, up to 8 characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .DAT
s = font size	*Accepted Values:* the number of memory bytes required to hold the Zebra-downloadable format of the font *Default Value:* if an incorrect value or no value is entered, the command is ignored
data = data string	*Accepted Values:* a string of ASCII hexadecimal values (two hexadecimal digits/byte). The total number of two-digit values must match parameter s. *Default Value:* if no data is entered, the command is ignored

Example • This is an example of how to download a true type font:

```
~DTR:FONT,52010,00AF01B0C65E...
```

(52010 two-digit hexadecimal values)

~DU

Download Unbounded TrueType Font

Description Some international fonts, such as Asian fonts, have more than 256 printable characters. These fonts are supported as *large TrueType fonts* and are downloaded to the printer with the ~DU command. Use ZTools to convert the large TrueType fonts to a Zebra-downloadable format.

The Field Block (^FB) command cannot support the large TrueType fonts.

Format ~DUd:o.x,s,data

This table identifies the parameters for this format:

Parameters	Details
d = font location	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = font name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .FNT
s = font size	*Accepted Values:* the number of memory bytes required to hold the Zebra-downloadable format of the font *Default Value:* if no data is entered, the command is ignored
data = data string	*Accepted Values:* a string of ASCII hexadecimal values (two hexadecimal digits/byte). The total number of two-digit values must match parameter s. *Default Value:* if no data is entered, the command is ignored

Example • This is an example of how to download an unbounded true type font:

```
~DUR:KANJI,86753,60CA017B0CE7...
```

(86753 two-digit hexadecimal values)

~DY

Download Graphics

Description ~DY downloads to the printer graphic objects in any supported format. This command can be used in place of ~DG for more saving and loading options.

Format ~DYf,b,x,t,w,data

This table identifies the parameters for this format:

Parameters	Details
d = font location	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
f = font name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
b = format downloaded in data field (f)	*Accepted Values:* A = uncompressed bitmap (.GRF, ASCII) B = uncompressed bitmap (.GRF, binary) C = AR-compressed bitmap (.GRF, compressed binary—used only by Zebra's BAR-ONE® software) P = PNG image (.PNG) *Default Value:* a value must be specified
x = extension of stored graphic	*Accepted Values:* G = raw bitmap (.GRF) P = store as compressed (.PNG) *Default Value:* .GRF, unless parameter b is set to P (.PNG)
t = total number of bytes in graphic	.GRF images: the size after decompression into memory .PNG images: the size of the .PNG file
w = total number of bytes per row	.GRF images: number of bytes per row .PNG images: value ignored—data is encoded directly into .PNG data
data = data	ASCII hexadecimal encoding, ZB64, or binary data, depending on b. a, p = ASCII hexadecimal or ZB64 b, c = binary When binary data is sent, all control prefixes and flow control characters are ignored until the total number of bytes needed for the graphic format is received.

Comments For more information on ZB64 encoding and compression, see Appendix I in *ZPL II Programming Guide Volume Two*.

~EF

Erase Stored Formats

Description The ~EF command erases all stored formats.

Format ~EF

Comments **The ~EF command is no longer recommended for use.** It is recommended that the ^ID (Object Delete) command be used to selectively delete stored formats.

~EG

Erase Download Graphics

See *^ID* on page 177.

^FB

Field Block

Description The ^FB command allows you to print text into a defined *block type* format. This command formats an ^FD or ^SN string into a block of text using the origin, font, and rotation specified for the text string. The ^FB command also contains an automatic word-wrap function.

Format ^FBa,b,c,d,e

This table identifies the parameters for this format:

Parameters	Details
a = width of text block line (in dots)	*Accepted Values:* 0 to the width of the label (or 9999) *Default Value:* 0 If the value is less than font width or not specified, text does not print.
b = maximum number of lines in text block	*Accepted Values:* 1 to 9999 *Default Value:* 1 Text exceeding the maximum number of lines overwrites the last line. Changing the font size automatically increases or decreases the size of the block.
c = add or delete space between lines (in dots)	*Accepted Values:* -9999 to 9999 *Default Value:* 0 Numbers are considered to be positive unless preceded by a minus sign. Positive values add space; negative values delete space.
d = text justification	*Accepted Values:* L (left), C (center), R (right), J (justified) *Default Value:* L Last line is left-justified if J is used.
e = hanging indent (in dots) of the second and remaining lines	*Accepted Values:* 0 to 9999 *Default Value:* 0

Example • These are examples of how the ^FB command affects field data.

ZPL II CODE	GENERATED LABEL
^XA ^CF0,30,30^FO25,50 ^FB250,4,, ^FDFD command that IS preceded by an FB command.^FS ^XZ	FD command that IS preceded by an FB command.
^XA ^CF0,30,30^FO25,50 ^FDFD command that IS NOT preceded by an FB command.^FS ^XZ	FD command that IS NOT preceded by an FB cor

Comments on the ^FB Command

This scheme can be used to facilitate special functions:

\& = carriage return/line feed

\(*) = soft hyphen (word break with a dash)

\\ = backslash (\)

Item 1: ^CI13 must be selected to print a backslash (\).

Item 2: If a soft hyphen is placed near the end of a line, the hyphen is printed. If it is not placed near the end of the line, it is ignored.

(*) = any alphanumeric character

- If a word is too long to print on one line by itself (and no soft hyphen is specified), a hyphen is automatically placed in the word at the right edge of the block. The remainder of the word is on the next line. The position of the hyphen depends on word length, not a syllable boundary. Use a soft hyphen within a word to control where the hyphenation occurs.
- Maximum data-string length is 3K, including control characters, carriage returns, and line feeds.
- Normal carriage returns, line feeds, and *word spaces* at line breaks are discarded.
- When using ^FT (Field Typeset), ^FT uses the baseline origin of the last possible line of text. Increasing the font size causes the text block to increase in size from bottom to top. This could cause a label to print past its top margin.
- When using ^FO (Field Origin), increasing the font size causes the text block to increase in size from top to bottom.
- If ^SN is used instead of ^FD, the field does not print.
- ^FS terminates an ^FB command. Each block requires its own ^FB command.

^FC

Field Clock (for Real-Time Clock)

Description The ^FC command is used to set the clock-indicators (delimiters) and the clock mode for use with the Real-Time Clock hardware. This command must be included within each label field command string each time the Real-Time Clock values are required within the field.

Format ^FCa,b,c

This table identifies the parameters for this format:

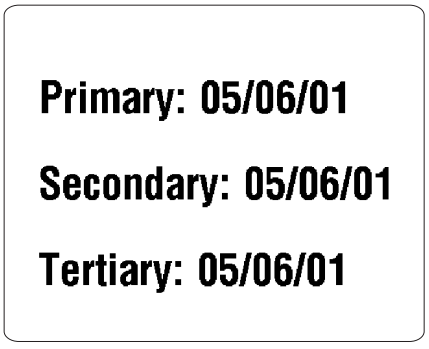
Parameters	Details
a = primary clock indicator character	*Accepted Values:* any ASCII character *Default Value:* %
b = secondary clock indicator character	*Accepted Values:* any ASCII character *Default Value:* none—this value cannot be the same as a or c
c = tertiary clock indicator character	*Accepted Values:* any ASCII character *Default Value:* none—this value cannot be the same as a or b

Example • Entering these ZPL sets the primary clock indicator to %, the secondary clock indicator to {, and the tertiary clock indicator to #. The results are printed on a label with Primary, Secondary, and Tertiary as field data.

ZPL II CODE

```
^XA
^FO10,100^A0N,50,50
^FC%,{,#
^FDPrimary: %m/%d/%y^FS
^FO10,200^A0N,50,50
^FC%,{,#
^FDSecondary: {m/{d/{y^FS
^FO10,300^A0N,50,50
^FC%,{,#
^FDTertiary: #m/#d/#y^FS
^XZ
```

GENERATED LABEL



Comments The ^FC command is ignored if the Real-Time Clock hardware is not present.

^FD

Field Data

Description The ^FD command defines the data string for the field. The field data can be any printable character except those used as command prefixes (^ and ~).

Format ^FDa

This table identifies the parameters for this format:

Parameters	Details
a = data to be printed	*Accepted Values:* any ASCII string up to 3072 characters *Default Value:* none—a string of characters must be entered

Comments The ^ and ~ characters can be printed by changing the prefix characters—see the *~CC* on page 106 and *^CT ~CT* on page 116 commands. The new prefix characters cannot be printed.

Characters with codes above 127, or the ^ and ~ characters, can be printed using the ^FH and ^FD commands.

- ^CI13 must be selected to print a backslash (\).

^FH

Field Hexadecimal Indicator

Description The ^FH command allows you to enter the hexadecimal value for any character directly into the ^FD statement. The ^FH command must precede each ^FD command that uses hexadecimals in its field.

Within the ^FD statement, the hexadecimal indicator must precede each hexadecimal value. The default hexadecimal indicator is _ (underscore). There must be a minimum of two characters designated to follow the underscore. The a parameter can be added when a different hexadecimal indicator is needed.

This command can be used with any of the commands that have field data (that is ^FD, ^FV (Field Variable), and ^SN (Serialized Data)).

Valid hexadecimal characters are:

0 1 2 3 4 5 6 7 8 9 A B C D E F a b c d e f

Format ^FHa

This table identifies the parameters for this format:

Parameters	Details
a = hexadecimal indicator	*Accepted Values:* any character except current format and control prefix (^ and ~ by default) *Default Value:* _ (underscore)

Example • This is an example of how to enter a hexadecimal value directly into a ^FD statement:

ZPL II CODE	GENERATED LABEL
^XA ^FO100,100 ^AD^FH ^FDTilde _7e used for HEX^FS ^XZ	Tilde ~ used for HEX
^XA ^FO100,100 ^AD^FH\ ^FDTilde \7E used for HEX^FS ^XZ	Tilde ~ used for HEX

^FM

Multiple Field Origin Locations

Description The ^FM command allows you to control the placement of bar code symbols.

It designates field locations for the PDF417 (^B7) and Micro-PDF417 (^BF) bar codes when the structured append capabilities are used. This allows printing multiple bar codes from the same set of text information.

The structured append capability is a way of extending the text printing capacity of both bar codes. If a string extends beyond what the data limitations of the bar code are, it can be printed as a series: 1 of 3, 2 of 3, 3 of 3. Scanners read the information and reconcile it into the original, unsegmented text.

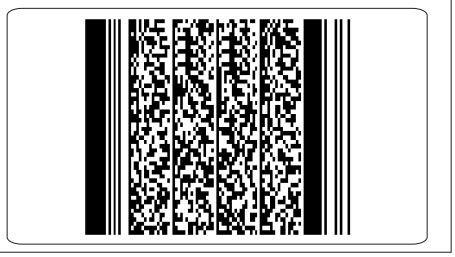
The ^FM command triggers multiple bar code printing on the same label with ^B7 and ^BF only. When used with any other commands, it is ignored.

Format ^FMx1,y1,x2,y2,...

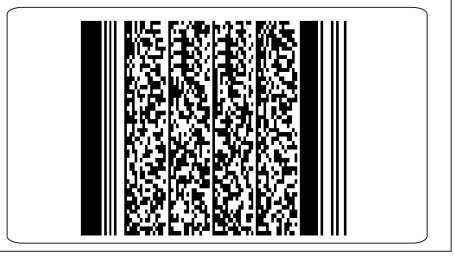
This table identifies the parameters for this format:

Parameters	Details
x1 = x-axis location of first symbol (in dots)	*Accepted Values:* 0 to 32000 e = exclude this bar code from printing *Default Value:* a value must be specified
y1 = y-axis location of first symbol (in dots)	*Accepted Values:* 0 to 32000 e = exclude this bar code from printing *Default Value:* a value must be specified
x2 = x-axis location of second symbol (in dots)	same as x1 parameter
y2 = y-axis location of second symbol (in dots)	same as y1 parameter
... = continuation of X,Y pairs	*Maximum number of pairs:* 60

Example • This example assumes a maximum of three bar codes:

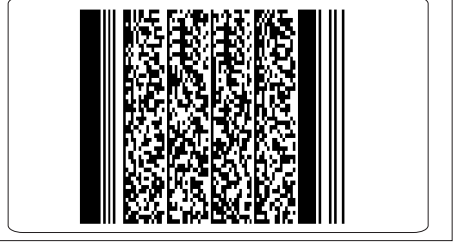
ZPL II CODE	GENERATED LABEL
^XA ^FM100,100,200,200,300,300 ^B7N,5,5,,50,N^FD<data>^FS ^XZ	

Example • This example assumes a maximum of three bar codes, with bar code 2 of 3 omitted:

ZPL II CODE	GENERATED LABEL
^XA ^FM100,100,e,e,300,300 ^B7N,5,5,,50,N^FD<data>^FS ^FS	

If `e` is entered for any of the x, y values, the bar code does not print.

Example • Symbol 2 of 3 in this example is still excluded:

ZPL II CODE	GENERATED LABEL
^XA ^FM100,100,200,200,300,300 ^B7N,5,5,,50,N^FD<data>^FS ^XZ	

Comments Subsequent bar codes print once the data limitations of the previous bar code have been exceeded. For example, bar code 2 of 3 prints once 1 of 3 has reached the maximum amount of data it can hold. Specifying three fields does not ensure that three bar codes print; enough field data to fill three bar code fields has to be provided.

The number of the `x,y` pairs can exceed the number of bar codes generated. However, if too few are designated, no symbols print.

^FN

Field Number

Description The ^FN command numbers the data fields. This command is used in both ^DF (Store Format) and ^XF (Recall Format) commands.

In a stored format, use the ^FN command where you would normally use the ^FD (Field Data) command. In recalling the stored format, use ^FN in conjunction with the ^FD command.

Format ^FN#

This table identifies the parameters for this format:

Parameters	Details
# = number to be assigned to the field	*Accepted Values:* 0 to 9999 *Default Value:* 0

Example • This example recalls the format (^XF) saved with ^DF and inserts field number data.

ZPL II CODE	GENERATED LABEL
^XA ^XFR:STOREFMT.ZPL^FS ^FN1^FDZEBRA^FS ^FN2^FDLABEL^FS ^XZ	ZEBRA LABEL BUILT BY ZEBRA

Comments

- The same ^FN value can be stored with several different fields.
- If a label format contains a field with ^FN and ^FD, the data in that field prints for any other field containing the same ^FN value.

^FO

Field Origin

Description The ^FO command sets a field origin, relative to the label home (^LH) position. ^FO sets the upper-left corner of the field area by defining points along the x-axis and y-axis independent of the rotation.

Format ^FOx,y

This table identifies the parameters for this format:

Parameters	Details
x = x-axis location (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* 0
y = y-axis location (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* 0

Comments If the value entered for the x or y parameter is too high, it could position the field origin completely off the label.

^FP

Field Parameter

Description The ^FP command allows vertical formatting of the font field, commonly used for printing Asian fonts.

Format ^FPd,g

This table identifies the parameters for this format:

Parameters	Details
d = direction	*Accepted Values:* H = horizontal printing (left to right) V = vertical printing (top to bottom) R = reverse printing (right to left) *Default Value:* H
g = additional inter-character gap (in dots)	*Accepted Values:* 0 to 9999 *Default Value:* 0 if no value is entered

Example • This is an example of how to implement reverse and vertical print:

ZPL II CODE	GENERATED LABEL
^XA ^FO100,50 ^FPV,10 ^AV ^FDvertical^FS ^XZ	v e r t i c a l
^XA ^FO350,50 ^FPR,10 ^AV ^FDreverse^FS ^XZ	esrever

Comments When using reverse printing, the origin specified in ^FT is the lower- left corner of the right-most text character.

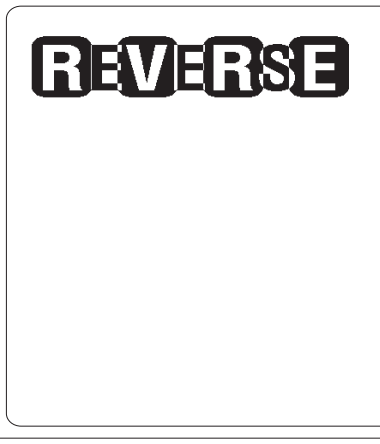
^FR

Field Reverse Print

Description The ^FR command allows a field to appear as white over black or black over white. When printing a field and the ^FR command has been used, the color of the output is the reverse of its background.

Format ^FR

Example • In this example, the ^GB command creates areas of black allowing the printing to appear white:

ZPL II CODE	GENERATED LABEL
^XA ^PR1 ^FO100,100 ^GB70,70,70,,3^FS ^FO200,100 ^GB70,70,70,,3^FS ^FO300,100 ^GB70,70,70,,3^FS ^FO400,100 ^GB70,70,70,,3^FS ^FO107,110^CF0,70,93 ^FR^FDREVERSE^FS ^XZ	

Comments The ^FR command applies to only one field and has to be specified each time. When multiple ^FR commands are going to be used, it might be more convenient to use the ^LR command.

^FS

Field Separator

Description The ^FS command denotes the end of the field definition. Alternatively, ^FS command can also be issued as a single ASCII control code SI (Control-O, hexadecimal 0F).

Format ^FS

^FT

Field Typeset

Description The ^FT command also sets the field position, relative to the home position of the label designated by the ^LH command. The typesetting origin of the field is fixed with respect to the contents of the field and does not change with rotation.

Format ^FTx,y

This table identifies the parameters for this format:

Parameters	Details
x = x-axis location (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* position after last formatted text field
y = y-axis location (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* position after last formatted text field

Text The origin is at the start of the character string, at the baseline of the font. Normally the baseline is the bottom of most characters, except for those with descenders, such as g, y, et cetera.

Bar Codes The origin is at the base of the bar code, even when an interpretation is present below the bar code, or if the bar code has guard bars.

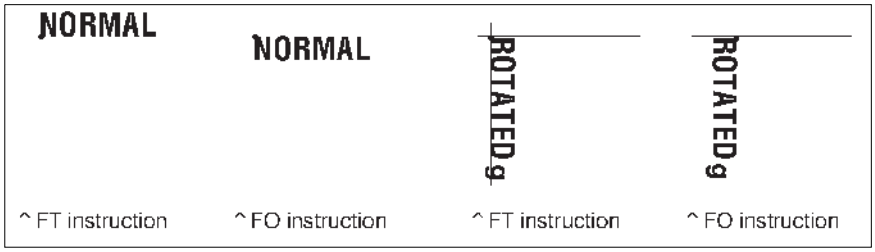
Graphic Boxes The origin is at the bottom-left corner of the box.

Images The origin is at the bottom-left corner of the rectangular image area.

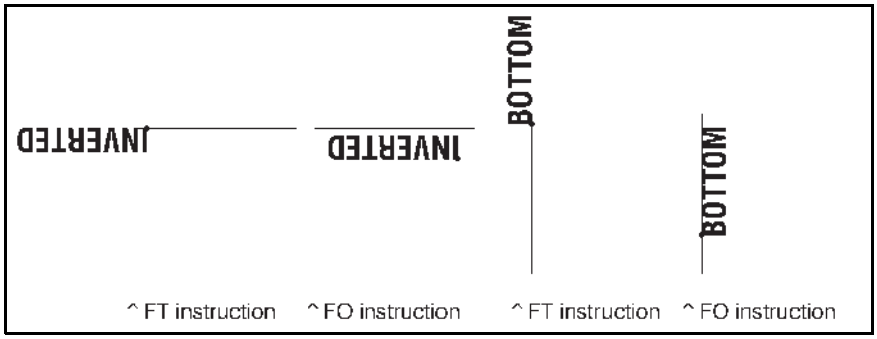
Example • The example below shows the differences in font orientation when using ^FT and ^FO relative to their ^LH position. The origin point of the font when using the ^FT command is always at the left of the baseline position of the first element or character in the field.

In normal orientation, all characters rest on the baseline. In rotated orientation, all characters are drawn to the right of the label from the baseline. In inverted orientation, all characters draw down from the baseline and print to the left. In bottom orientation, all characters draw towards the left of the label from the baseline and printer to the right. The *dot* shows the origin point for both the ^FT and ^FO font orientations.

Scalable Normal Font Orientation, Scalable Rotated Font Orientation



Scalable Inverted Font Orientation, Scalable Bottom-up Font Orientation



Example • This is an example of the ^FT command and concatenation:

ZPL II CODE	GENERATED LABEL
^XA ^FT10,200^A0N,30,20^FDACME ^FS ^FT^GS^FDC^FS ^FT^A0N,30,20^FDSummer ^FS ^FT^A0N,60,50^FDClearance ^FS ^FT^A0N,120,100^FDSale ^FS ^XZ	ACME ™ Summer Clearance Sale

Comments When a coordinate is missing, the position following the last formatted field is assumed. This *remembering* simplifies field positioning with respect to other fields. Once the first field is positioned, other fields follow automatically.

There are several instances where using the ^FT command without specifying x and y parameters is not recommended:

- when positioning the first field in a label format
- at any time with the ^FN (Field Number) command
- following an ^SN (Serialization Data) command

^FV

Field Variable

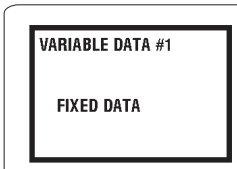
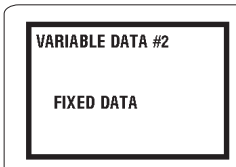
Description ^FV replaces the ^FD (field data) command in a label format when the field is variable.

Format ^FVa

This table identifies the parameters for this format:

Parameters	Details
a = variable field data to be printed	*Accepted Values:* 0 to 3072 character string *Default Value:* if no data is entered, the command is ignored

Example • This is an example of how to use the ^MC and ^FV command:

ZPL II CODE	GENERATED LABEL
^XA ^FO40,40 ^GB300,203,8^FS ^FO55,60^CF0,25 ^FVVARIABLE DATA #1^FS ^FO80,150 ^FDFIXED DATA^FS ^MCN ^XZ	
^XA ^FO55,60^CF0,25 ^FVVARIABLE DATA #2^FS ^MCY ^XZ	

Comments ^FV fields are always cleared after the label is printed. ^FD fields are not cleared.

^FW

Field Orientation

Description The ^FW command sets the default orientation for all command fields that have an orientation (rotation) parameter. Fields can be rotated 0, 90, 180, or 270 degrees clockwise by using this command.

The ^FW command affects only fields that follow it. Once you have issued a ^FW command, the setting is retained until you turn off the printer or send a new ^FW command to the printer.

Format ^FWr

This table identifies the parameters for this format:

Parameters	Details
r = rotate field	*Accepted Value:* N = normal R = rotated 90 degrees I = inverted 180 degrees B = bottom-up 270 degrees, read from bottom up *Initial Value at Power-up:* N

Comments If the ^FW command is entered with the r parameter missing, the command is ignored.

^FW affects only the orientation in commands where the rotation parameter has not been specifically set. If a command has a specific rotation parameter, that value is used.

^FX

Comment

Description The ^FX command is useful when you want to add *non-printing* informational comments or statements within a label format. Any data after the ^FX command up to the next caret (^) or tilde (~) command does not have any effect on the label format. Therefore, you should avoid using the caret (^) or tilde (~) commands within the ^FX statement.

Format ^FXc

This table identifies the parameters for this format:

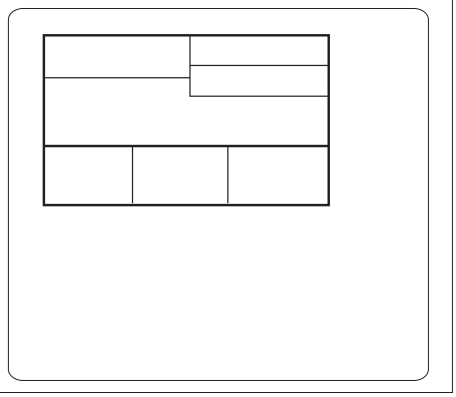
Parameters	Details
c = non printing comment	Creates a non-printable comment.

Example • This is an example of how to use the ^FX command effectively.

ZPL II CODE

```
^XA
^LH100,100^FS
^FXSHIPPING LABEL^FS
^FO10,10^GB470,280,4^FS
^FO10,190^GB470,4,4^FS
^FO10,80^GB240,2,2^FS
^FO250,10^GB2,100,2^FS
^FO250,110^GB226,2,2^FS
^FO250,60^GB226,2,2^FS
^FO156,190^GB2,95,2^FS
^FO312,190^GB2,95,2^FS
^XZ
```

GENERATED LABEL



Comments Correct usage of the ^FX command includes following it with the ^FS command.

^GB

Graphic Box

Description The ^GB command is used to draw boxes and lines as part of a label format. Boxes and lines are used to highlight important information, divide labels into distinct areas, or to improve the appearance of a label. The same format command is used for drawing either boxes or lines.

Format ^GBw,h,t,c,r

This table identifies the parameters for this format:

Parameters	Details
w = box width (in dots)	*Accepted Values:* value of t to 32000 *Default Value:* value used for thickness (t) or 1
h = box height (in dots)	*Accepted Values:* value of t to 32000 *Default Value:* value used for thickness (t) or 1
t = border thickness (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* 1
c = line color	*Accepted Values:* B (black) or W (white) *Default Value:* B
r = degree of corner-rounding	*Accepted Values:* 0 (no rounding) to 8 (heaviest rounding) *Default Value:* 0

For the w and h parameters, keep in mind that printers have a default of 6, 8, 12, or 24 dots/millimeter. This comes out to 153, 203, 300, or 600 dots per inch. To determine the values for w and h, calculate the dimensions in millimeters and multiply by 6, 8, 12, or 24.

If the width and height are not specified, you get a solid box with its width and height as specified by value t.

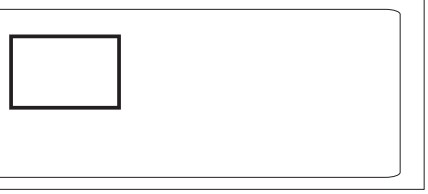
The roundness-index is used to determine a rounding-radius for each box. Formula:

rounding-radius = (rounding-index / 8) * (shorter side / 2)

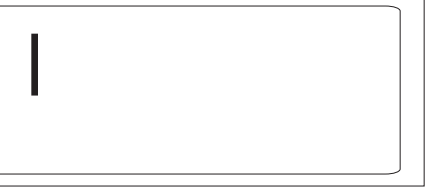
where the shorter side is the lesser of the width and height (after adjusting for minimum and default values).

Example • Here are a few examples:

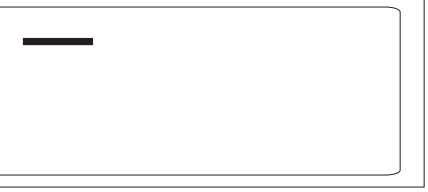
Width: 1.5 inch; Height: 1 inch; Thickness: 10; Color: default; Rounding: default

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^GB300,200,10^FS ^XZ	

Width: 0 inch; Height: 1 inch; Thickness: 20; Color: default; Rounding: default:

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^GB0,203,20^FS ^XZ	

Width: 1 inch; Height: 0 inch; Thickness: 30; Color: default; Rounding: default

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^GB203,0,20^FS ^XZ	

Width: 1.5 inch; Height: 1 inch; Thickness: 10; Color: default; Rounding: 5

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^GB300,200,10,,5^FS ^XZ	

^GC

Graphic Circle

Description The ^GC command produces a circle on the printed label. The command parameters specify the diameter (width) of the circle, outline thickness, and color. Thickness extends inward from the outline.

Format ^GCd,t,c

This table identifies the parameters for this format:

Parameters	Details
d = circle diameter (in dots)	*Accepted Values:* 3 to 4095 (larger values are replaced with 4095) *Default Value:* 3
t = border thickness (in dots)	*Accepted Values:* 2 to 4095 *Default Value:* 1
c = line color	*Accepted Values:* B (black) or W (white) *Default Value:* B

Example • This is an example of how to create a circle on the printed label:

ZPL II CODE	GENERATED LABEL
^XA ^FO50,50 ^GC250,10,B^FS ^XZ	

^GD

Graphic Diagonal Line

Description The ^GD command produces a straight diagonal line on a label. This can be used in conjunction with other graphic commands to create a more complex figure.

Format ^GDw,h,t,c,o

This table identifies the parameters for this format:

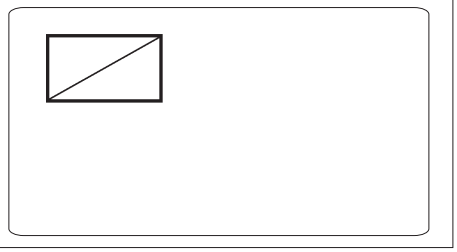
Parameters	Details
w = box width (in dots)	*Accepted Values:* 3 to 32000 *Default Value:* value of t (thickness) or 1
h = box height (in dots)	*Accepted Values:* 3 to 32000 *Default Value:* value of t (thickness) or 1
t = border thickness (in dots)	*Accepted Values:* 1 to 32000 *Default Value:* 1
c = line color	*Accepted Values:* B (black) or W (white) *Default Value:* B
o = orientation (direction of the diagonal)	*Accepted Values:* R (or /) = right-leaning diagonal L (or \) = left-leaning diagonal *Default Value:* R

Example • This is an example of how to create a diagonal line connecting one corner with the opposite corner of a box on a printed label:

ZPL II CODE

```
^XA
^FO150,100
^GB350,203,10^FS
^FO155,110
^GD330,183,10,,R^FS
^XZ
```

GENERATED LABEL



^GE

Graphic Ellipse

Description The ^GE command produces an ellipse in the label format.

Format ^GEw,h,t,c

This table identifies the parameters for this format:

Parameters	Details
w = ellipse width (in dots)	*Accepted Values:* 3 to 4095 (larger values are replaced with 4095) *Default Value:* value used for thickness (t) or 1
h = ellipse height (in dots)	*Accepted Values:* 3 to 4095 *Default Value:* value used for thickness (t) or 1
t = border thickness (in dots)	*Accepted Values:* 2 to 4095 *Default Value:* 1
c = line color	*Accepted Values:* B (black) or W (white) *Default Value:* B

Example • This is an example of how to create a ellipse on a printed label:

ZPL II CODE	GENERATED LABEL
^XA ^FO100,100 ^GE300,100,10,B^FS ^XZ	

^GF

Graphic Field

Description The ^GF command allows you to download graphic field data directly into the printer's bitmap storage area. This command follows the conventions for any other field, meaning a field orientation is included. The graphic field data can be placed at any location within the bitmap space.

Format ^GFa,b,c,d,data

This table identifies the parameters for this format:

Parameters	Details
a = compression type	*Accepted Values:* A = ASCII hexadecimal (follows the format for other download commands) B = binary (data sent after the c parameter is strictly binary) C = compressed binary (data sent after the c parameter is in compressed binary format. The data is compressed on the host side using Zebra's compression algorithm. The data is then decompressed and placed directly into the bitmap.) *Default Value:* A
b = binary byte count	*Accepted Values:* 1 to 99999 This is the total number of bytes to be transmitted for the total image or the total number of bytes that follow parameter d. For ASCII download, the parameter should match parameter c. Out-of-range values are set to the nearest limit. *Default Value:* command is ignored if a value is not specified
c = graphic field count	*Accepted Values:* 1 to 99999 This is the total number of bytes comprising the graphic format (width x height), which is sent as parameter d. Count divided by bytes per row gives the number of lines in the image. This number represents the size of the image, not necessarily the size of the data stream (see d). *Default Value:* command is ignored if a value is not specified

Parameters	Details
d = bytes per row	*Accepted Values:* 1 to 99999 This is the number of bytes in the downloaded data that comprise one row of the image. *Default Value:* command is ignored if a value is not specified
data = data	*Accepted Values:* ASCII hexadecimal data: 00 to FF A string of ASCII hexadecimal numbers, two digits per image byte. CR and LF can be inserted as needed for readability. The number of two-digit number pairs must match the above count. Any numbers sent after count is satisfied are ignored. A comma in the data pads the current line with *00* (white space), minimizing the data sent. ~DN or any caret or tilde character prematurely aborts the download. **Binary data:** Strictly binary data is sent from the host. All control prefixes are ignored until the total number of bytes needed for the graphic format is sent.

Example • This example downloads 8,000 total bytes of data and places the graphic data at location 100,100 of the bitmap. The data sent to the printer is in ASCII form.

```
^FO100,100^GFA,8000,8000,80,ASCII data
```

Example • This example downloads 8,000 total bytes of data and places the graphic data at location 100,100 of the bitmap. The data sent to the printer is in binary form.

```
^FO100,100^GFB,8000,8000,80,Binary data
```

^GS

Graphic Symbol

Description The ^GS command enables you to generate the registered trademark, copyright symbol, and other symbols.

Format ^GSo,h,w

This table identifies the parameters for this format:

Parameters	Details
o = font orientation	*Accepted Values:* N = normal R = rotate 90 degrees clockwise I = inverted 180 degrees B = bottom-up, 270 degrees *Default Value:* N or last ^FW value
h = character height proportional to width (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* last ^CF value
w = character width proportional to height (in dots)	*Accepted Values:* 0 to 32000 *Default Value:* last ^CF value

Example • Use the ^GS command followed by ^FD and the appropriate character (A through E) within the field data to generate the desired character:

ZPL II CODE	GENERATED LABEL
^XA^CFD ^FO50,50 ^FDZEBRA PROGRAMMING^FS ^FO50,75 ^FDLANGUAGE II (ZPL II)^FS ^FO280,75 ^GS^FDC^FS ^XZ	ZEBRA PROGRAMMING LANGUAGE II (ZPL II™)

A = ® (Registered Trade Mark)

B = © (Copyright)

C = ™ (Trade Mark)

D = (Underwriters Laboratories approval)

E = (Canadian Standards Association approval)

~HB

Battery Status

Description When the ~HB command is sent to the printer, a data string is sent back to the host. The string starts with an <STX> control code sequence and terminates by an <ETX><CR><LF> control code sequence.

Format ~HB

Parameters: when the printer receives the command, it returns:

<STX>bb.bb,hh.hh,bt<ETX><CR><LF>

<STX>	=	ASCII start-of-text character
bb.bb	=	current battery voltage reading to the nearest 1/4 volt
hh.hh	=	current head voltage reading to the nearest 1/4 volt
bt	=	battery temperature in Celsius
<ETX>	=	ASCII end-of-text character
<CR>	=	ASCII carriage return
<LF>	=	ASCII line feed character

Comments This command is used for the power-supply battery of the printer and should not be confused with the battery backed-up RAM.

~HD

Head Temperature Information

Description The ~HD command echoes printer status information that includes the power supply and head temperature using the terminal emulator.

Format ~HD

Example • This is an example of the ~HD command:

```
Head Temp = 29
Ambient Temp = 00
Head Test = Passed
Darkness Adjust = 23
Print Speed = 2
Slew Speed = 6
Backfeed Speed = 2
Static_pitch_length = 0521
Dynamic_pitch_length = 0540
Max_dynamic_pitch_length = 0540
Min_dynamic_pitch_length = 0537
COMMAND PFX = ~ : FORMAT PFX = ^ : DELIMITER = ,
P30 INTERFACE = None
P31 INTERFACE = None
P32 INTERFACE = Front Panel              Revision 5
P33 INTERFACE = None
P34 INTERFACE = None
P35 INTERFACE = None
Dynamic_top_position = 0008

No ribbon A/D = 0000
```

^HF

Graphic Symbol

Description The ^HF command sends an object of ZPL format instructions to the host, in ~DF.

Format ^HF,o,h,w

This table identifies the parameters for this format:

Parameters	Details
o = font orientation	*Accepted Values:* N = normal R = rotate I = inverted 180 degrees B = bottom-up, 270 degrees *Default Value:* N or last ^FW value.
h = character height proportional to width (in dots)	*Accepted Value:* 0 to 32000 *Default Value:* last ^CF value
w = character width proportional to height (in dots)	*Accepted Value:* 0 to 32000 *Default Value:* last ^CF value

^HG

Host Graphic

Description The ^HG command is used to upload graphics to the host. The graphic image can be stored for future use, or it can be downloaded to any Zebra printer.

Format ^HGd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = device location of object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* search priority
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .GRF

Comments For more information on uploading graphics, see *^HY* on page 174.

^HH

Configuration Label Return

Description The ^HH command echoes printer configuration back to the host, using a terminal emulator.

Format ^HH

Example • This is an example of the ^HH command:

```
+10                     DARKNESS
+000                    TEAR OFF
TEAR OFF                PRINT MODE
NON-CONTINUOUS          MEDIA TYPE
WEB                     SENSOR TYPE
DIRECT-THERMAL          PRINT METHOD
050 6/8 MM              PRINT  WIDTH
0622                    LABEL LENGTH
22.0IN    557MM         MAXIMUM LENGTH
9600                    BAUD
8 BITS                  DATA BITS
NONE                    PARITY
XON/XOFF                HOST HANDSHAKE
NONE                    PROTOCOL
000                     NETWORK ID
NORMAL MODE             COMMUNICATIONS
<~>  7EH                CONTROL PREFIX
<^>  5EH                FORMAT PREFIX
<,>  2CH                DELIMITER CHAR
ZPL II                  ZPL MODE
NO MOTION               MEDIA POWER UP
NO MOTION               HEAD CLOSE
DEFAULT                 BACKFEED
+000                    LABEL TOP
+0000                   LEFT POSITION
026                     WEB S.
068                     MEDIA S.
050                     MARK S.
001                     MARK MED S.
CS                      MODES ENABLED
..                      MODES DISABLED
864 8/MM FULL           RESOLUTION
V32.10.2    <-          FIRMWARE
V2.2.6.98.A             HARDWARE ID
CUSTOMIZED              CONFIGURATION
1024.............R:     RAM
8192.............B:     MEMORY CARD
0768.............E:     ONBOARD FLASH
NONE                    FORMAT CONVERT
NONE                    OPTION
05/14/03                RTC DATE
02:23                   RTC TIME
DYNAMIC                 IP RESOLUTION
ALL                     IP PROTOCOL
010.003.005.090         IP ADDRESS
255.255.255.000         SUBNET MASK
010.003.005.001         DEFAULT GATEWAY
```

~HI

Host Identification

Description The ~HI command is designed to be sent from the host to the Zebra printer to retrieve information. Upon receipt, the printer responds with information on the model, software version, dots-per-millimeter setting, memory size, and any detected objects.

Format ~HI

When the printer receives this command, it returns:

XXXXXX,V1.0.0,dpm,000KB,X

XXXXXX = model of Zebra printer

V1.0.0 = version of software

dpm = dots/mm

6, 8, 12, or 24 dots/mm printheads

000KB = memory

512KB = 1/2 MB
1024KB = 1 MB
2048KB = 2 MB
4096KB = 4 MB
8192KB = 8 MB

x = recognizable objects

only options specific to printer are shown (cutter, options, et cetera.)

~HM

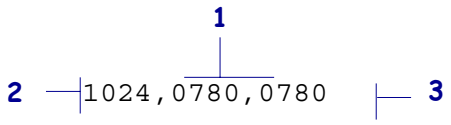
Host RAM Status

Description Sending ~HM to the printer immediately returns a memory status message to the host. Use this command whenever you need to know the printer's RAM status.

When ~HM is sent to the Zebra printer, a line of data containing information on the total amount, maximum amount, and available amount of memory is sent back to the host.

Format ~HM

Example • This example shows when the ~HM is sent to the printer, a line of data containing three numbers are sent back to the host. Each set of numbers is identified and explained in the table that follows:



1	The total amount of RAM (in kilobytes) installed in the printer. In this example, the printer has 1024K RAM installed.
2	The maximum amount of RAM (in kilobytes) available to the user. In this example, the printer has a maximum of 780K RAM available.
3	The amount of RAM (in kilobytes) currently available to the user. In this example, there is 780K of RAM in the printer currently available to the user.

Comments Memory taken up by bitmaps is included in the currently available memory value (due to ^MCN).

Downloading a graphic image, fonts, or saving a bitmap affects only the amount of RAM. The total amount of RAM and maximum amount of RAM does not change after the printer is turned on.

~HS

Host Status Return

Description When the host sends ~HS to the printer, the printer sends three data strings back. Each string starts with an <STX> control code and is terminated by an <ETX><CR><LF> control code sequence. To avoid confusion, the host prints each string on a separate line.

String 1

<STX>aaa,b,c,dddd,eee,f,g,h,iii,j,k,l<ETX><CR><LF>

aaa	=	communication (interface) settings*
b	=	paper out flag (1 = paper out)
c	=	pause flag (1 = pause active)
dddd	=	label length (value in number of dots)
eee	=	number of formats in receive buffer
f	=	*buffer full* flag (1 = receive buffer full)
g	=	*communications diagnostic mode* flag (1 = diagnostic mode active)
h	=	*partial format* flag (1 = partial format in progress)
iii	=	unused (always 000)
j	=	*corrupt RAM* flag (1 = configuration data lost)
k	=	temperature range (1 = under temperature)
l	=	temperature range (1 = over temperature)

* This string specifies the printer's baud rate, number of data bits, number of stop bits, parity setting, and type of handshaking. This value is a three-digit decimal representation of an eight-bit binary number. To evaluate this parameter, first convert the decimal number to a binary number.

The nine-digit binary number is read according to this table:

aaa = a^8 a^7 a^6 a^5 a^4 a^3 a^2 a^1 a^0	
a^7 = Handshake 0 = Xon/Xoff 1 = DTR	a^8 a^2 a^1 a^0 = Baud 0 000 = 110 0 001 = 300 0 010 = 600 0 011 = 1200 0 100 = 2400 0 101 = 4800 0 110 = 9600 0 111 = 19200 1 000 = 28800 *(available only on certain printer models)* 1 001 = 38400 *(available only on certain printer models)* 1 010 = 57600 *(available only on certain printer models)* 1 011 = 14400
a^6 = Parity Odd/Even 0 = Odd 1 = Even	
a^5 = Disable/Enable 0 = Disable 1 = Enable	
a^4 = Stop Bits 0 = 2 Bits 1 = 1 Bit	
a^3 = Data Bits 0 = 7 Bits 1 = 8 Bits	

String 2

```
<STX>mmm,n,o,p,q,r,s,t,uuuuuuuu,v,www<ETX><CR><LF>
```

mmm	=	function settings*
n	=	unused
o	=	*head up* flag (1 = head in up position)
p	=	*ribbon out* flag (1 = ribbon out)
q	=	*thermal transfer mode* flag (1 = Thermal Transfer Mode selected)
r	=	Print Mode 0 = Rewind 1 = Peel-Off 2 = Tear-Off 3 = Cutter 4 = Applicator
s	=	print width mode
t	=	*label waiting* flag (1 = label waiting in Peel-off Mode)
uuuuuuuu	=	labels remaining in batch
v	=	*format while printing* flag (always 1)
www	=	number of graphic images stored in memory

* This string specifies the printer's media type, sensor profile status, and communication diagnostics status. As in String 1, this is a three-digit decimal representation of an eight-bit binary number. First, convert the decimal number to a binary number.

The eight-digit binary number is read according to this table:

mmm = m7 m6 m5 m4 m3 m2 m1 m0	
m7 = Media Type 0 = Die-Cut 1 = Continuous	m4 m3 m2 m1 = Unused 0 = Off 1 = On
m6 = Sensor Profile 0 = Off	m0 = Print Mode 0 = Direct Thermal 1 = Thermal Transfer
m5 = Communications Diagnostics 0 = Off 1 = On	

String 3

```
<STX>xxxx,y<ETX><CR><LF>
```

xxxx	=	password
y	=	0 (static RAM not installed)
		1 (static RAM installed)

~HU

Return ZebraNet Alert Configuration

Description This command returns the table of configured ZebraNet Alert settings to the host.

Format ~HU

Example • If the ~HU command is sent to the printer with existing Alert messages set to go to e-mail and SNMP traps, the data returned would look something like the information below. See ^SX on page 267 for complete information on the individual parameter settings.

```
B,C,Y,Y,ADMIN@COMPANY.COM,0
J,F,Y,Y,,0
C,F,Y,Y,,0
D,F,Y,Y,,0
E,F,Y,N,,0
F,F,Y,N,,0
H,C,Y,N,ADMIN@COMPANY.COM,0
N,C,Y,Y,ADMIN@COMPANY.COM,0
O,C,Y,Y,ADMIN@COMPANY.COM,0
P,C,Y,Y,ADMIN@COMPANY.COM,0
```

The first line indicates that condition B (ribbon out) is routed to destination C (e-mail address).

The next two characters, Y and Y, indicate that the *condition set* and *condition clear* options have been set to yes.

The following entry is the destination that the Alert e-mail should be sent to; in this example it is admin@company.com.

The last figure seen in the first line is 0, which is the port number.

Each line shows the settings for a different Alert condition as defined in the ^SX command.

^HV

Host Verification

Description This command is used to return data from specified fields, along with an optional ASCII header, to the host computer. The command can be used with any field that has been assigned a number with the ^RT command or the ^FN and ^RF commands.

Format ^HV#,n,h

This table identifies the parameters for this format:

Parameters	Details
# = field number specified with another command	The value assigned to this parameter should be the same as the one used in another command. *Accepted values:* 0 to 9999 *Default value:* 0
n = number of bytes to be returned	*Accepted values:* 1 to 256 *Default value:* 64
h = header	Header (in uppercase ASCII characters) to be returned with the data. *Acceptable values:* 0 to 3072 characters *Default value:* no header

^HW

Host Directory List

Description ^HW is used to transmit a directory listing of objects in a specific memory area (storage device) back to the host device. This command returns a formatted ASCII string of object names to the host.

Each object is listed on a line and has a fixed length. The total length of a line is also fixed. Each line listing an object begins with the asterisk (*) followed by a blank space. There are eight spaces for the object name, followed by a period and three spaces for the extension. The extension is followed by two blank spaces, six spaces for the object size, two blank spaces, and three spaces for option flags (reserved for future use). The format looks like this:

```
<STX><CR><LF>
DIR R: <CR><LF>
*Name.ext(2sp.)(6 obj. sz.)(2sp.)(3 option flags)
*Name.ext(2sp.)(6 obj. sz.)(2sp.)(3 option flags)
<CR><LF>
-xxxxxxx bytes free
<CR><LF>
<ETX>
<STX> = start of text
<CR><LR> = carriage return/line feed
<ETX> = end on text
```

The command might be used in a stand-alone file to be issued to the printer at any time. The printer returns the directory listing as soon as possible, based on other tasks it might be performing when the command is received.

This command, like all ^ (caret) commands, is processed in the order that it is received by the printer.

Format ^HWd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = location to retrieve object listing	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* asterisk (*). A question mark (?) can also be used.
x = extension	*Accepted Values:* any extension conforming to Zebra conventions *Default Value:* asterisk (*). A question mark (?) can also be used.

Example • Listed is an example of the ^HW command to retrieve from information R:

```
^XA
^HWR:*.*
^XZ
```

Example • The printer returned this information as the Host Directory Listing:-DIR R:*.*

```
*R:ARIALN1.FNT 49140
*R:ARIALN2.FNT 49140
*R:ARIALN3.FNT 49140
*R:ARIALN4.FNT 49140
*R:ARIALN.FNT 49140
*R:ZEBRA.GRF 8420
-794292 bytes free R:RAM
```

^HY

Upload Graphics

Description The ^HY command is an extension of the ^HG command. ^HY is used to upload graphic objects from the printer in any supported format.

Format ^HYd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = location of object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* search priority
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* an object name must be specified
x = extension	*Accepted Values:* G = .GRF (raw bitmap format) P = .PNG (compressed bitmap format) *Default Value:* format of stored image

Comments The image is uploaded in the form of a ~DY command. The data field of the returned ~DY command is always encoded in the ZB64 format.

^HZ

Display Description Information

Description The ^HZ command is used for returning printer description information in XML format. The printer returns information on format parameters, object directories, individual object data, and print status information. For more information, see Chapter 6 (XML: Super Host Status) in *ZPL II Programming Guide Volume Two*.

Format ^HZb

This table identifies the parameters for this format:

Parameters	Details
b = display description to return	*Accepted Values:* a = display all information f = display printer format setting information l = display object directory listing information o = display individual object data information r = display printer status information *Default Value:* if the value is missing or invalid, the command is ignored

Format ^HZO,d:o.x,l

This table identifies the parameters for this format:

Parameters	Details
d = location of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = object name	*Accepted Values:* 1 to 8, or 1 to 16 alphanumeric characters based on parameter **l**. *Default Value:* if a name is not specified, UNKNOWN is used.

Parameters	Details
x = extension	Supported extensions for objects (parameter o) include: . FNT — font . GRF — graphic . PNG — compressed graphic . ZPL — stored format . DAT — encoding table . ZOB — downloadable object . STO — Alert data file . BAZ — security for all ZBI executables For more details on . BAZ, see *Encrypting ZBI Files* on page 321.
l = long filename support	*Accepted Values:* Y = Yes If Y, the object data stores the filename as 16 characters. The data is only compatible with firmware V60.13.0.5 and above. N = No If N, the object data stores the filename as 8 characters. The data is forward and backward compatible with all versions of firmware. *Default Value:* N

Example • This example shows the object data information for the object SAMPLE.GRF located on R:.

```
^XA
^HZO,R:SAMPLE.GRF
^XZ
```

^ID

Object Delete

Description The ^ID command deletes objects, graphics, fonts, and stored formats from storage areas. Objects can be deleted selectively or in groups. This command can be used within a printing format to delete objects before saving new ones, or in a stand-alone format to delete objects.

The image name and extension support the use of the asterisk (*) as a wild card. This allows you to easily delete a selected groups of objects.

Format ^IDd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = location of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = object name	*Accepted Values:* any 1 to 8 character name *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Accepted Values:* any extension conforming to Zebra conventions *Default Value:* .GRF

Example • To delete stored formats from DRAM:

```
^XA
^IDR:*.ZPL^FS
^XZ
```

Example • To delete formats and images named SAMPLE from DRAM, regardless of the extension:

```
XA
^IDR:SAMPLE.*^FS
^XZ
```

Example • To delete the image SAMPLE1.GRF prior to storing SAMPLE2.GRF:

```
^XA
^FO25,25^AD,18,10
^FDDelete^FS
^FO25,45^AD,18,10
^FDthen Save^FS
^IDR:SAMPLE1.GRF^FS
^ISR:SAMPLE2.GRF^FS^XZ
```

Example • In this , the * is a wild card, indicating that all objects with the .GRF extension are deleted:

```
^XA
^IDR:*.GRF^FS
^XZ
```

Comments When an object is deleted from R:, the object can no longer be used and memory is available for storage. This applies only to R: memory.

The ^ID command also frees up the uncompressed version of the object in DRAM.

If the name is specified as *.ZOB, all downloaded bar code fonts (or other objects) are deleted.

If the named downloadable object cannot be found in the R:, E:, B:, and A: device, the ^ID command is ignored.

^IL

Image Load

Description The ^IL command is used at the beginning of a label format to load a stored image of a format and merge it with additional data. The image is always positioned at ^FO0,0.

Important • See *^IS* on page 182.

Using this technique to overlay the image of constant information with variable data greatly increases the throughput of the label format.

Format ^ILd:o.x

This table identifies the parameters for this format:

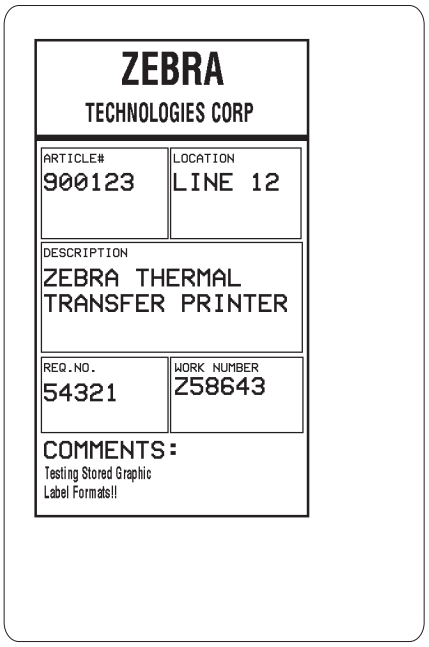
Parameters	Details
d = location of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .GRF

Example • This example recalls the stored image `SAMPLE2.GRF` from DRAM and overlays it with the additional data. The graphic was stored using the `^IS` command. For the stored label format, see the *^IS* on page 182 command.

ZPL II CODE

```
^XA
^ILR:SAMPLE2.GRF^FS
^CFD,36,20
^FO15,210
^FD900123^FS
^FO218,210
^FDLINE 12^FS
^FO15,360^AD
^FDZEBRA THERMAL^FS
^FO15,400^AD
^FDTRANSFER PRINTER^FS
^FO15,540
^FD54321^FS
^FO220,530
^FDZ58643^FS
^FO15,670^A0,27,18
^FDTesting Stored Graphic^FS
^FO15,700^A0,27,18
^FDLabel Formats!!^FS
^XZ
```

GENERATED LABEL



^IM

Image Move

Description The ^IM command performs a direct move of an image from storage area into the bitmap. The command is identical to the ^XG command (Recall Graphic), except there are no sizing parameters.

Format ^IMd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = location of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* search priority
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Fixed Value:* .GRF

Example • This example moves the image SAMPLE.GRF from DRAM and prints it in several locations in its original size.

```
^XA
^FO100,100^IMR:SAMPLE.GRF^FS
^FO100,200^IMR:SAMPLE.GRF^FS
^FO100,300^IMR:SAMPLE.GRF^FS
^FO100,400^IMR:SAMPLE.GRF^FS
^FO100,500^IMR:SAMPLE.GRF^FS
^XZ
```

Comments By using the ^FO command, the graphic image can be positioned anywhere on the label.

The difference between ^IM and ^XG: ^IM does not have magnification, and therefore might require less formatting time. However, to take advantage of this, the image must be at a 8-, 16-, or 32-bit boundary.

^IS

Image Save

Description The ^IS command is used within a label format to save that format as a graphic image, rather than as a ZPL II script. It is typically used toward the end of a script. The saved image can later be recalled with virtually no formatting time and overlaid with variable data to form a complete label.

Using this technique to overlay the image of constant information with the variable data greatly increases the throughput of the label format.



Important • See *^IL* on page 179.

Format ^ISd:o.x,p

This table identifies the parameters for this format:

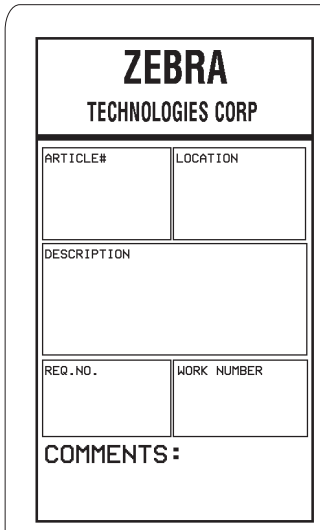
Parameters	Details
d = location of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = object name	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension	*Accepted Values:* .GRF or .PNG *Default Value:* .GRF
p = print image after storing	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y

Example • This is an example of using the ^IS command to save a label format to DRAM. The name used to store the graphic is SAMPLE2.GRF.

ZPL II CODE

```
^XA
^LH10,15^FWN^BY3,3,85^CFD,36
^GB430,750,4^FS
^FO10,170^GB200,144,2^FS
^FO10,318^GB410,174,2^FS
^FO212,170^GB206,144,2^FS
^FO10,498^GB200,120,2^FSR
^FO212,498^GB209,120,2^FS
^FO4,150^GB422,10,10^FS
^FO135,20^A0,70,60
^FDZEBRA^FS
^FO80,100^A0,40,30
^FDTECHNOLOGIES CORP^FS
^FO15,180^CFD,18,10^FS
^FDARTICLE#^FS
^FO218,180
^FDLOCATION^FS
^FO15,328
^FDDESCRIPTION^FS
^FO15,508
^FDREQ.NO.^FS
^FO220,508
^FDWORK NUMBER^FS
^FO15,630^AD,36,20
^FDCOMMENTS:^FS
^ISR:SAMPLE2.GRF,Y
^XZ
```

GENERATED LABEL



~JA

Cancel All

Description The ~JA command cancels all format commands in the buffer. It also cancels any batches that are printing.

The printer stops after the current label is finished printing. All internal buffers are cleared of data and the DATA LED turn off.

Submitting this command to the printer scans the buffer and deletes only the data before the ~JA in the input buffer — it does not scan the remainder of the buffer for additional ~JA commands.

Format ~JA

^JB

Initialize Flash Memory

Description The ^JB command is used to initialize the two types of Flash memory available in the Zebra printers.

Format ^JBa

This table identifies the parameters for this format:

Parameters	Details
a = device to initialize	*Acceptable Values:* B = Flash card (PCMCIA) E = internal Flash memory *Default Value:* a device must be specified

Example • This is an example of initializing the different types of flash memory:

- ^JBB – initializes the optional Flash card when installed in the printer.
- ^JBE – initializes the optional Flash memory when installed in the printer.
- ^JBA – initializes initial Compact Flash memory when installed in the printer.

~JB

Reset Optional Memory

Description The ~JB command is used for these conditions:

- The ~JB command must be sent to the printer if the battery supplying power to the battery powered memory card fails and is replaced. A bad battery shows a *battery dead* condition on the Printer Configuration Label.
- The ~JB command can also be used to intentionally clear (reinitialize) the B: memory card. The card must **not** be write protected.

Format ~JB

Comments If the battery is replaced and this command is not sent to the printer, the memory card cannot function.

~JC

Set Media Sensor Calibration

Description The ~JC command is used to force a label length measurement and adjust the media and ribbon sensor values.

Format ~JC

Comments In Continuous Mode, only the media and ribbon sensors are calibrated.

~JD

Enable Communications Diagnostics

Description The ~JD command initiates Diagnostic Mode, which produces an ASCII printout (using current label length and full width of printer) of all characters received by the printer. This printout includes the ASCII characters, the hexadecimal value, and any communication errors.

Format ~JD

~JE

Disable Diagnostics

Description The ~JE command cancels Diagnostic Mode and returns the printer to normal label printing.

Format ~JE

~JF

Set Battery Condition

Description There are two low battery voltage levels sensed by the *PA/PT400*™ printers. When battery voltage goes below the first level, the green LED begins flashing as a warning but printing continues. When this warning occurs, it is recommended to recharge the battery.

As printing continues, a second low voltage level is reached. At this point, both green and orange LEDs flash as a warning, and printing automatically pauses.

When pause on low voltage is active (~JFY) and the battery voltage level falls below the second *low voltage* level, printing pauses and an error condition is displayed as an indication that the printer should be plugged into the battery charger. By pressing FEED, printing continues on a label-by-label basis, but there is a high risk of losing label format information due to the continued decrease of battery voltage.

When pause on low voltage is not active (~JFN), and the battery voltage level falls below the second *low voltage* level, printing continues and the orange LED remains off. If the battery voltage continues to decrease, label information could be lost and cause the printer to stop operating. This option should be selected only when the printer is connected to the Car Battery Adapter. From time to time the printer might sense that battery voltage is below the first *low voltage* level, but due to the continuous recharging of the car battery, further loss of battery voltage is not a concern and printing continues.

If this option is not selected when using the Car Battery Adapter, you might need to press FEED to take the printer out of Pause Mode and print each label.

Format ~JFp

This table identifies the parameters for this format:

Parameters	Details
p = pause on low voltage	*Accepted Values:* Y (pause on low voltage) or N (do not pause) N is suggested when the printer is powered by the Car Battery Adapter. *Default Value:* Y

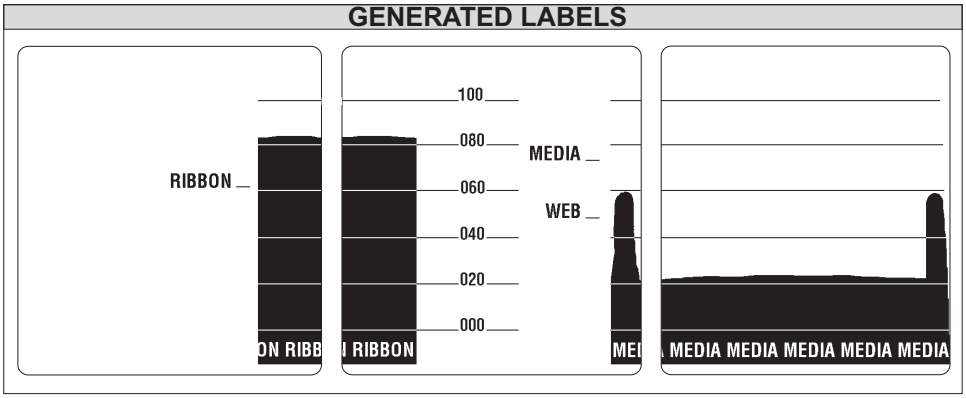
~JG

Graphing Sensor Calibration

Description The ~JG command is used to force a label length measurement, recalibrate the media and ribbon sensors, and print a graph (media sensor profile) of the sensor values.

Format ~JG

Example • Sending the ~JG command to the printer produces a series of labels resembling this image:



^JI

Start ZBI (Zebra BASIC Interpreter)

Description ^JI works much like the ~JI command. Both commands are sent to the printer to initialize the Zebra BASIC Interpreter.

In interactive mode, ^JI can be sent through one of the communication ports (serial, parallel, or Ethernet) to initialize the printer to receive ZBI commands. This command can be sent from one of the Zebra software utilities, such as ZTools, or from a standard PC program, such as Hyper terminal.

When the command is received, the printer responds by sending a ZBI header back to the console, along with the program version number. This indicates that the interpreter is active.

Format ^JId:o.x,b,c,d

This table identifies the parameters for this format:

Parameters	Details
d = location of program to run after initialization	*Acceptable Values:* R:, E:, B:, and A: *Default Value:* location must be specified
o = name of program to run after initialization	*Accepted Values:* any valid program name *Default Value:* name must be specified
x = extension of program to run after initialization	*Fixed Value:* .BAS
b = console control	*Accepted Values:* Y (console on) or N (console off) *Default Value:* Y
c = echoing control	*Accepted Values:* Y (echo on) or N (echo off) *Default Value:* Y
d = memory allocation for ZBI	*Accepted Values:* 20K to 1024K *Default Value*: 50K

Comments When the printer is turned on, it can receive ZPL II commands and label formats. However, for the printer to recognize ZBI commands and programs, it must be initialized using ^JI or ~JI.

Only one ZBI interpreter can be active in the printer at a time. If a second ^JI or ~JI command is received while the interpreter is running, the command is ignored.

The interpreter is deactivated by entering one of two commands:

ZPL at the ZBI prompt

~JQ at an active ZPL port

~JI

Start ZBI (Zebra BASIC Interpreter)

Description ~JI works much like the ^JI command. Both commands are sent to the printer to initialize the Zebra BASIC Interpreter.

In interactive mode, ~JI can be sent through one of the communication ports (serial, parallel, or Ethernet) to initialize the printer to receive ZBI commands. This command can be sent from one of the Zebra software utilities, such as ZTools, or from a standard PC program, such as Hyper terminal.

When the command is received, the printer responds by sending a ZBI header back to the console, along with the program version number. This indicates that the interpreter is active.

Format ~JI

Comments While receiving commands, the printer *echoes* the received characters back to the source. This can be toggled on and off with the ZBI ECHO command.

When the printer is turned on, it can receive ZPL II commands and label formats. However, for the printer to recognize ZBI commands and formats, it must be initialized using ^JI or ~JI.

Only one ZBI interpreter can be active in the printer at a time. If a second ~JI or ^JI command is received while the interpreter is running, the command is ignored.

The interpreter is deactivated by entering one of these commands:

ZPL at the ZBI prompt

~JQ at an active ZPL port

^JJ

Set Auxiliary Port

Description The ^JJ command allows you to control an online verifier or applicator device.

Format ^JJa,b,c,d,e,f

This table identifies the parameters for this format:

Parameters	Details
a = Operational Mode for auxiliary port	*Accepted Values:* 0 = off 1 = reprint on error—the printer stops on a label with a verification error. When PAUSE is pressed, the label reprints (if ^JZ is set to reprint). If a bar code is near the upper edge of a label, the label feeds out far enough for the bar code to be verified and then backfeeds to allow the next label to be printed and verified. 2 = maximum throughput—the printer stops when a verification error is detected. The printer starts printing the next label while the verifier is still checking the previous label. This mode provides maximum throughput, but does not allow the printer to stop immediately on a label with a verification error. *Default Value:* 0
b = Application Mode	*Accepted Values:* 0 = off 1 = End Print signal normally high, and low only when the printer is moving the label forward. 2 = End Print signal normally low, and high only when the printer is moving the label forward. 3 = End Print signal normally high, and low for 20 ms when a label has been printed and positioned. 4 = End Print signal normally low, and high for 20 ms when a label has been printed and positioned. *Default Value:* 0
c = Application Mode start signal print	*Accepted Values:* p = Pulse Mode – Start Print signal must be de-asserted before it can be asserted for the next label. l = Level Mode – Start Print signal does not need to be de-asserted to print the next label. As long as the Start Print signal is low and a label is formatted, a label prints. *Default Value:* 0

Parameters	Details
d = Application Label Error Mode	*Accepted Values:* e = error mode—the printer asserts the *Service Required* signal (svce_req - pin 10) on the application port, enters into Pause Mode, and displays an error message on the LCD. f = Feed Mode—a blank label prints when the web is not found where expected to sync the printer to the media. *Default Value:* f
e = Reprint Mode	*Accepted Values:* e = enabled—printer ignores the Reprint signal. d = disabled—the last label reprints after the signal is asserted. If a label is canceled, the label to be reprinted is also canceled. This mode consumes more memory because the last printed label is not released until it reprints. *Default Value:* d
f = Ribbon Low Mode	*Accepted Values:* e = *enabled* – printer warning issued when ribbon low. d = *disabled* – printer warning not issued when ribbon low. *Default Value:* e

~JL

Set Label Length

Description The ~JL command is used to set the label length. Depending on the size of the label, the printer feeds one or more blank labels.

Format ~JL

^JM

Set Dots per Millimeter

Description The ^JM command lowers the density of the print—24 dots/mm becomes 12, 12 dots/mm becomes 6, 8 dots/mm becomes 4, and 6 dots/mm becomes 3. ^JM also affects the field origin (^FO) placement on the label (see example).

When sent to the printer, the ^JM command doubles the format size of the label. Depending on the printhead, normal dot-per-millimeter capabilities for a Zebra printer are 12 dots/mm (304 dots/inch), 8 dots/mm (203 dots/inch) or 6 dots/mm (153 dots/inch).

This command must be entered before the first ^FS command in a format. The effects of ^JM are persistent.

Format ^JMn

This table identifies the parameters for this format:

Parameters	Details
n = set dots per millimeter	*Accepted Values:* A = 24 dots/mm, 12 dots/mm, 8 dots/mm or 6 dots/mm B = 12 dots/mm, 6 dots/mm, 4 dots/mm or 3 dots/mm *Default Value:* A

Example • This example of the affects of alternating the dots per millimeter:

ZPL II CODE	GENERATED LABEL
^XA ^JMA^FS ^FO100,100 ^B2N,50,Y,N,N ^FD1234567890^FS ^XZ	
^XA ^JMB^FS ^FO100,100 ^B2N,50,Y,N,N ^FD1234567890^FS ^XZ	

Comments If ^JMB is used, the UPS Maxicode bar code becomes out of specification.

~JN

Head Test Fatal

Description The ~JN command turns on the head test option. When activated, ~JN causes the printer to halt when a head test failure is encountered.

Once an error is encountered the printer remains in error mode until the head test is turned off (~JO) or power is cycled.

Format ~JN

Comments If the communications buffer is full, the printer is not able to receive data. In this condition, the ~JO command is not received by the printer.

~JO

Head Test Non fatal

Description The ~JO command turns off the head test option. ~JO is the default printhead test condition and overrides a failure of printhead element status check. This state is changed when the printer receives a ~JN (Head Test Fatal) command. The printhead test does not produce an error when ~JO is active.

Format ~JO

~JP

Pause and Cancel Format

Description The ~JP command clears the format currently being processed and places the printer into Pause Mode.

The command clears the next format that would print, or the oldest format from the buffer. Each subsequent ~JP command clears the next buffered format until the buffer is empty. The DATA indicator turns off when the buffer is empty and no data is being transmitted.

Issuing the ~JP command is identical to using CANCEL on the printer, but the printer does not have to be in Pause Mode first.

Format ~JP

~JQ

Terminate Zebra BASIC Interpreter

Description The ~JQ command is used when Zebra BASIC Interpreter is active. Sending ~JQ to the printer terminates the ZBI session.

Format ~JQ

Comments Entering ZPL at the command prompt also terminates a ZBI session.

~JR

Power On Reset

Description The ~JR command resets all of the printer's internal software, performs a power-on self-test (POST), clears the buffer and DRAM, and resets communication parameters and default values. Issuing a ~JR command performs the same function as a manual power-on reset.

Format ~JR

^JS

Sensor Select

Format ~JSb

This table identifies the parameters for this format:

Parameters	Details
b = backfeed order in relation to printing	*Accepted Values:* A = 100 percent backfeed after printing and cutting B = 0 percent backfeed after printing and cutting, and 100 percent before printing the next label N = normal — 90 percent backfeed after label is printed O = off — turn backfeed off completely 10 to 90 = percentage value The value entered must be a multiple of 10. Values not divisible by 10 are rounded to the nearest acceptable value. For example, ~JS55 is accepted as 50 percent backfeed. *Default Value:* N

Comments When using a specific value, the difference between the value entered and 100 percent is calculated before the next label is printed. For example, a value of 40 means 40 percent of the backfeed takes place after the label is cut or removed. The remaining 60 percent takes place before the next label is printed.

The value for this command is also reflected in the Backfeed parameter on the printer configuration label.

For ~JSN — the Backfeed parameter is listed as DEFAULT

For ~JSA — or 100 the Backfeed parameter is listed as AFTER

For ~JSB — or 0 the Backfeed parameter is listed as BEFORE

For ~JS10 — to 90 the Backfeed parameter is listed as the value entered

~JS

Change Backfeed Sequence

Description The ~JS command is used to control the backfeed sequence. This command can be used on printers with or without built-in cutters.

These are the primary applications:

- to allow programming of the *rest point* of the cut edge of continuous media.
- provide immediate backfeed after peel-off when the printer is used in a print/apply application configuration.

This command stays in effect only until the printer is turned off, a new ~JS command is sent, or the setting is changed on the front panel. When a ~JS command is encountered, it overrides the current front panel setting for the Backfeed Sequence.

The most common way of eliminating backfeed is to operate in Rewind Mode. Rewind Mode does not backfeed at all. After a label prints, the leading edge of the next label is placed at the print line. This eliminates the need to backfeed and does not introduce a non printable area at the leading edge or bottom of the label. It also does not allow the label to be taken from the printer because it is not fed out from under the printhead.

Running in another mode with backfeed turned off allows the label to be removed and eliminates the time-reduction of the backfeed sequence.

Format ~JSb

This table identifies the parameters for this format:

Parameters	Details
b = backfeed order in relation to printing	*Accepted Values:* A = 100 percent backfeed after printing and cutting B = 0 percent backfeed after printing and cutting, and 100 percent before printing the next label N = normal — 90 percent backfeed after label is printed O = off — turn backfeed off completely 10 to 90 = percentage value The value entered must be a multiple of 10. Values not divisible by 10 are rounded to the nearest acceptable value. For example, ~JS55 is accepted as 50 percent backfeed. *Default Value:* N

Comments When using a specific value, the difference between the value entered and 100 percent is calculated before the next label is printed. For example, a value of 40 means 40 percent of the backfeed takes place after the label is cut or removed. The remaining 60 percent takes place before the next label is printed.

The value for this command is also reflected in the Backfeed parameter on the printer configuration label.

For ~JSN — the Backfeed parameter is listed as DEFAULT

For ~JSA — or 100 the Backfeed parameter is listed as AFTER

For ~JSB — or 0 the Backfeed parameter is listed as BEFORE

For ~JS10 — to 90 the Backfeed parameter is listed as the value entered

^JT

Head Test Interval

Description The ^JT command allows you to change the printhead test interval from every 100 labels to any desired interval. With the ^JT command, the printer is allowed to run the test after printing a label. When a parameter is defined, the printer runs the test after printing a set amount of labels.

The printer's default head test state is off. Parameters for running the printhead test are defined by the user.

Format ^JT####,a,b,c

This table identifies the parameters for this format:

Parameters	Details
#### = four-digit number of labels printed between head tests	*Accepted Values:* 0000 to 9999 If a value greater than 9999 is entered, it is ignored. *Default Value:* 0000 (off)
a = manually select range of elements to test	*Accepted Values:* Y (yes) or N (no) *Initial Value at Power-up:* N
b = first element to check when parameter a is Y	*Accepted Values:* 0 to 9999 *Initial Value at Power-up:* 0
c = last element to check when parameter a is Y	*Accepted Values:* 0 to 9999 *Initial Value at Power-up:* 9999

Comments The ^JT command supports testing a range of print elements. The printer automatically selects the test range by tracking which elements have been used since the previous test.

^JT also turns on Automatic Mode to specify the first and last elements for the head test. This makes it possible to select any specific area of the label or the entire print width.

If the last element selected is greater than the print width selected, the test stops at the selected print width.

Whenever the head test command is received, a head test is performed on the next label unless the count is set to 0 (zero).

^JU

Configuration Update

Description The ^JU command sets the active configuration for the printer.

Format ^JUa

This table identifies the parameters for this format:

Parameters	Details
a = active configuration	*Accepted Values:* F = reload factory values These values are lost at power-off if not saved with ^JUS. R = recall last saved values S = save current settings These values are used at power-on. *Default Value:* a value must be specified

^JW

Set Ribbon Tension

Description ^JW sets the ribbon tension for the printer it is sent to.

Format ^JWt

This table identifies the parameters for this format:

Parameters	Details
t = tension	*Accepted Values:* L = low M = medium H = high *Default Value:* a value must be specified

Comments ^JW is used only for *PAX* series printers.

~JX

Cancel Current Partially Input Format

Description The ~JX command cancels a format currently being sent to the printer. It does not affect any formats currently being printed, or any subsequent formats that might be sent.

Format ~JX

^JZ

Reprint After Error

Description The ^JZ command reprints a partially printed label caused by a Ribbon Out, Media Out, or Head Open error condition. The label is reprinted as soon as the error condition is corrected.

This command remains active until another ^JZ command is sent to the printer or the printer is turned off.

Format ^JZa

This table identifies the parameters for this format:

Parameters	Details
a = reprint after error	*Accepted Values:* Y (yes) or N (no) *Initial Value at Power-up:* Y

Comments ^JZ sets the error mode for the printer. If ^JZ changes, only labels printed after the change are affected.

If the parameter is missing or incorrect, the command is ignored.

~KB

Kill Battery (Battery Discharge Mode)

Description To maintain performance of the rechargeable battery in the portable printers, the battery must be fully discharged and recharged regularly. The ~KB command places the printer in battery discharge mode. This allows the battery to be drained without actually printing.

Format ~KB

Comments While the printer is in Discharge Mode, the green power LED flashes in groups of three flashes.

Discharge Mode might be terminated by sending a printing format to the printer or by pressing either of the front panel keys.

If the battery charger is plugged into the printer, the battery is automatically recharged once the discharge process is completed.

^KD

Select Date and Time Format (for Real Time Clock)

Description The ^KD command selects the format that the Real-Time Clock's date and time information presents as on a configuration label. This is also displayed on the *Printer Idle* LCD front panel display, and displayed while setting the date and time.

Format ^KDa

This table identifies the parameters for this format:

Parameters	Details
a = value of date and time format	*Accepted Values:* 0 = normal, displays *Version Number* of firmware 1 = MM/DD/YY (24-hour clock) 2 = MM/DD/YY (12-hour clock) 3 = DD/MM/YY (24-hour clock) 4 = DD/MM/YY (12-hour clock) *Default Value:* 0

Comments If the Real-Time Clock hardware is not present, Display Mode is set to 0 (Version Number).

If Display Mode is set to 0 (Version Number) and the Real-Time Clock hardware is present, the date and time format on the configuration label is presented in format 1.

If Display Mode is set to 0 (Version Number) and the Real-Time Clock hardware is present, the date and time format on the front panel display is presented in format 1.

^KL

Define Language

Description The ^KL command selects the language displayed on the front panel.

Format ^KLa

This table identifies the parameters for this format:

Parameters	Details
a = language	*Accepted Values:* 1 = English 2 = Spanish 3 = French 4 = German 5 = Italian 6 = Norwegian 7 = Portuguese 8 = Swedish 9 = Danish 10 = Spanish2 11 = Dutch 12 = Finnish 13 = Japanese *Default Value:* 1

^KN

Define Printer Name

Description The printer's network name and description can be set using the ^KN command. ^KN is designed to make your Zebra printer easy for users to identify. The name the administrator designates is listed on the configuration label and on the Web page generated by the printer.

Format ^KNa,b

This table identifies the parameters for this format:

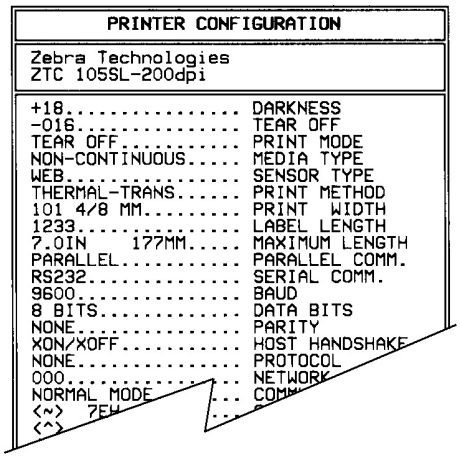
Parameters	Details
a = printer name	*Accepted Values:* up to 16 alphanumeric characters *Default Value:* if a value is not entered, the parameter is ignored If more than 16 characters are entered, only the first 16 are used.
b = printer description	*Accepted Values:* up to 35 alphanumeric characters *Default Value:* if a value is not entered, the parameter is ignored If more than 35 characters are entered, only the first 35 are used.

Example • This is an example of how to change the printer's network name an description:

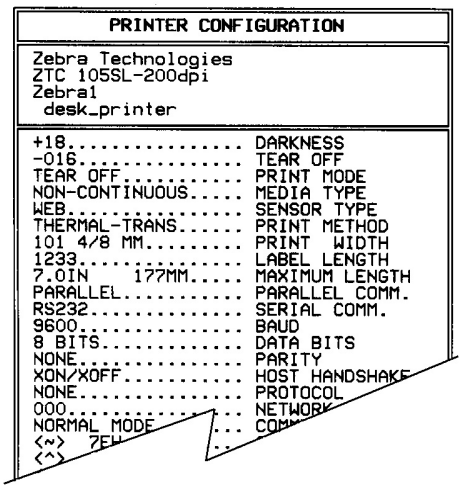
This shows how a configuration looks before using this command and after using this command:

```
^XA
^KNZebra1,desk_printer
^XZ
```

Before using this command:



After using this command:



^KP

Define Password

Description The ^KP command is used to define the password that must be entered to access the front panel switches and LCD Setup Mode.

Format ^KP####

This table identifies the parameters for this format:

Parameters	Details
#### = mandatory four-digit password	*Accepted Values:* any four-digit numeric sequence *Default Value:* 1234

Example • This is an example of how to set a new front panel password:

```
^XA
^KP5678
^XZ
```

Comments If you forget your password, the printer can be returned to a default Setup Mode and the default password *1234* is valid again. Caution should be used, however — this also sets the printer configuration values back to their defaults.

To return the printer to the default factory settings using ZPL, send this:

```
^XA
^JUF
^XZ
```

To return the printer to the default factory settings using the control panel keys, see your printer's User Guide for the procedure.

^LH

Label Home

Description The ^LH command sets the label home position.

The default home position of a label is the upper-left corner (position 0,0 along the x and y axis). This is the axis reference point for labels. Any area below and to the right of this point is available for printing. The ^LH command changes this reference point. For instance, when working with preprinted labels, use this command to move the reference point below the preprinted area.

This command affects only fields that come after it. It is recommended to use ^LH as one of the first commands in the label format.

Format ^LHx,y

This table identifies the parameters for this format:

Parameters	Details
x = x-axis position (in dots)	*Accepted Values:* 0 to 32000 *Initial Value at Power-up:* 0 or last permanently saved value
y = y-axis position (in dots)	*Accepted Values:* 0 to 32000 *Initial Value at Power-up:* 0 or last permanently saved value

Depending on the printhead used in your printer, use one of these when figuring the values for x and y:

6 dots = 1 mm, 152 dots = 1 inch
8 dots = 1 mm, 203 dots = 1 inch
11.8 dots = 1 mm, 300 dots = 1 inch
24 dots = 1 mm, 608 dots = 1 inch

Comments To be compatible with existing printers, this command must come before the first ^FS (Field Separator) command. Once you have issued an ^LH command, the setting is retained until you turn off the printer or send a new ^LH command to the printer.

^LL

Label Length

Description The ^LL command defines the length of the label. This command is necessary when using continuous media (media not divided into separate labels by gaps, spaces, notches, slots, or holes).

To affect the current label and be compatible with existing printers, ^LL must come before the first ^FS (Field Separator) command. Once you have issued ^LL, the setting is retained until you turn off the printer or send a new ^LL command.

Format ^LLy

This table identifies the parameters for this format:

Parameters	Details
y = y-axis position (in dots)	*Accepted Values:* 1 to 32000, not to exceed the maximum label size. While the printer accepts any value for this parameter, the amount of memory installed determines the maximum length of the label. *Default Value:* typically set through the LCD (if applicable), or to the maximum label length capability of the printer.

Comments These formulas can be used to determine the value of y:

For 6 dot/mm printheads...	Label length in inches x 152.4 (dots/inch) = y
For 8 dot/mm printheads...	Label length in inches x 203.2 (dots/inch) = y
For 12 dot/mm printheads...	Label length in inches x 304.8 (dots/inch) = y
For 24 dot/mm printheads...	Label length in inches x 609.6 (dots/inch) = y

Values for y depend on the memory size. If the entered value for y exceeds the acceptable limits, the bottom of the label is cut off. The label also shifts down from top to bottom.

If multiple ^LL commands are issued in the same label format, the last ^LL command affects the next label unless it is prior to the first ^FS.

^LR

Label Reverse Print

Description The ^LR command reverses the printing of all fields in the label format. It allows a field to appear as white over black or black over white.

Using the ^LR is identical to placing an ^FR command in all current and subsequent fields.

Format ^LRa

This table identifies the parameters for this format:

Parameters	Details
a = reverse print all fields	*Accepted Values:* Y (yes) or N (no) *Initial Value at Power-up:* N or last permanently saved value

Example • This is an example that shows printing white over black and black over white:

ZPL II CODE	GENERATED LABEL
^XA^LRY ^FO100,50 ^GB195,203,195^FS ^FO180,110^CFG ^FDLABEL^FS ^FO130,170 ^FDREVERSE^FS ^XZ	

Comments The ^LR setting remains active unless turned off by ^LRN or the printer is turned off.

Note • ^GB needs to be used together with ^LR.

Only fields following this command are affected.

^LS

Label Shift

Description The ^LS command allows for compatibility with Z-130 printer formats that are set for less than full label width. It is used to shift all field positions to the left so the same commands used on a Z-130 or Z-220 Printer can be used on other Zebra printers.

To determine the value for the ^LS command, use this formula:

Z-130 and Z-220 values for ^LHx + ^FOx
(distance from edge of label) = printer value for ^LSa

If the print position is less than 0, set ^LS to 0.

Format ^LSa

Important • The ability to save the ^LS command depends on the version of firmware.

This table identifies the parameters for this format:

Parameters	Details
a = shift left value (in dots)	*Accepted Values:* -9999 to 9999 *Initial Value at Power-up:* 0

Comments When entering positive values, it is not necessary to use the + sign. The value is assumed to be positive unless preceded by a negative sign (-).

To be compatible with existing Zebra printers, this command must come before the first ^FS (Field Separator) command. Once you have issued an ^LS command, the setting is retained until you turn off the printer or send a new ^LS command to the printer.

^LT

Label Top

Description The ^LT command moves the entire label format a maximum of 120 dot rows up or down from its current position, in relation to the top edge of the label. A negative value moves the format towards the top of the label; a positive value moves the format away from the top of the label.

This command can be used to fine-tune the position of the finished label without having to change any of the existing parameters.

Important • For some printer models, it is possible to request a negative value large enough to cause the media to backup into the printer and become unthreaded from the platen. This condition can result in a printer error or unpredictable results.

Format ^LTx

This table identifies the parameters for this format:

Parameters	Details
x = label top (in dot rows)	*Accepted Values:* -120 to 120 *Default Value:* a value must be specified or the command is ignored

Comments The Accepted Value range for x might be smaller depending on the printer platform.

The ^LT command does not change the media rest position.

^MC

Map Clear

Description In normal operation, the bitmap is cleared after the format has been printed. The ^MC command is used to retain the current bitmap. This applies to current and subsequent labels until cleared with ^MCY.

Format ^MCa

Important • To produce a label template, ^MC must be used with ^FV.

This table identifies the parameters for this format:

Parameters	Details
a = map clear	*Accepted Values:* Y (clear bitmap) or N (do not clear bitmap) *Initial Value at Power-up:* Y

Comments The ^MC command retains the image of the current label after formatting. It appears in the background of the next label printed.

^MD

Media Darkness

Description The ^MD command adjusts the darkness relative to the current darkness setting.

Format ^MDa

This table identifies the parameters for this format:

Parameters	Details
a = media darkness level	*Accepted Values:* -30 to 30, depending on current value *Initial Value at Power-up:* 0 If no value is entered, this command is ignored.

Example • These examples show setting the printer to different darkness levels:

- If the current value (value on configuration label) is 16, entering the command ^MD-9 decreases the value to 7.
- If the current value (value on configuration label) is 1, entering the command ^MD15 increases the value to 16.
- If the current value (value on configuration label) is 25, entering the command ^MD10 increases only the value to 30, which is the maximum value allowed.

Each ^MD command is treated separately in relation to the current value as printed on the configuration label.

Important • The darkness setting range for the *Xi*III*Plus* is 0 to 30 in increments of .1

The firmware is setup so that the 2 Darkness ZPL commands (^MD, ~SD) accepts that range of settings.

Example • These are examples of the *Xi*III*Plus* Darkness Setting:

```
^MD8.3
~SD8.3
```

Example • For example, this is what would happen if two ^MD commands were received:

Assume the current value is 15. An ^MD-6 command is received that changes the current value to 9. Another command, ^MD2, is received. The current value changes to 17.

The two ^MD commands are treated individually in relation to the current value of 15.

Comments The ~SD command value, if applicable, is added to the ^MD command.

^MF

Media Feed

Description The ^MF command dictates what happens to the media at power-up and at head-close after the error clears.

Format ^MFp,h

This table identifies the parameters for this format:

Parameters	Details
p = feed action at power-up	*Accepted Values:* F = feed to the first web after sensor C = (see ~*JC* on page 187 definition) L = (see ~*JL* on page 196 definition) N = no media feed *Default Value:* platform-dependent
h = feed action after closing printhead	*Accepted Values:* F = feed to the first web after sensor C = (see ~*JC* on page 187 definition) L = (see ~*JL* on page 196 definition) N = no media feed *Default Value:* platform-dependent

Comments It is important to remember that if you choose the N setting, the printer assumes that the media and its position relative to the printhead are the same as before power was turned off or the printhead was opened. Use the ^JU command to save changes.

^ML

Maximum Label Length

Description The ^ML command lets you adjust the maximum label length.

Format ^MLa,b,c,d

This table identifies the parameters for this format:

Parameters	Details
a = maximum label length (in dot rows)	*Accepted Values:* 0 to maximum length of label *Default Value:* last permanently saved value
b = maximum logical paper out counter	*Accepted Values:* must be greater than the actual label length or the printer indicates a **paper out** error after each label *Default Value:* set to one inch longer than twice the label length
c = maximum physical paper out counter	*Accepted Values:* must be greater than the actual notch or hole or the printer indicates a **paper out** condition after each label *Default Value:* is set to one half an inch
d = maximum ribbon out counter	*Accepted Values:* allows for the ribbon sensors to occasionally get an incorrect ribbon reading without causing an error condition *Default Value:* set to one half of a millimeter **Important** • The printer ignores ribbon indications that are less than one-half of a millimeter in length.

Comments For calibration to work properly, you must set the maximum label length equal to or greater than your actual label length.

^MM

Print Mode

Description The ^MM command determines the action the printer takes after a label or group of labels has printed. This bulleted list identifies the different modes of operation:

- Tear-off — after printing, the label advances so the web is over the tear bar. The label, with liner attached, can be torn off manually.
- Peel-off — after printing, the label moves forward and activates a Label Available Sensor. Printing stops until the label is manually removed from the printer.

  Power Peel – liner automatically rewinds using an optional internal rewind spindle.

  Value Peel – liner feeds down the front of the printer and is manually removed.

  Prepeel – after each label is manually removed, the printer feeds the next label forward to prepeel a small portion of the label away from the liner material. The printer then backfeeds and prints the label. The prepeel feature assists in the proper peel operation of some media types.
- Rewind — the label and backing are rewound on an (optional) external rewind device. The next label is positioned under the printhead (no backfeed motion).
- Applicator — when used with an application device, the label move far enough forward to be removed by the applicator and applied to an item.
- Cutter — after printing, the media feeds forward and is automatically cut into predetermined lengths.

Format ^MMa,b

This table identifies the parameters for this format:

Parameters	Details
a = desired mode	*Accepted Values:* T = Tear-off P = Peel-off (not available on *S*-300) R = Rewind A = Applicator (depends on printer model) C = Cutter *Default Value:* T The values available for parameter a are dependent on the printer being used and whether it supports the option.
b = prepeel select	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y The command is ignored if parameters are missing or invalid. The current value of the command remains unchanged.

Comments Be sure to select the appropriate value for the print mode being used to avoid unexpected results.

^MN

Media Tracking

Description The ^MN command relays to the printer what type of media is being used (continuous or non-continuous) for purposes of tracking. This bulleted list shows the types of media associated with this command:

- Continuous Media – this media has no physical characteristic (web, notch, perforation, mark, et cetera) to separate labels. Label length is determined by the ^LL command.
- Non-continuous Media – this media has some type of physical characteristic (web, notch, perforation, mark, et cetera) to separate the labels.

Format ^MNa

This table identifies the parameters for this format:

Parameters	Details
a = media being used	*Accepted Values:* N = continuous media *Y = non-continuous media web sensing *W = non-continuous media web sensing M = non-continuous media mark sensing *Default Value:* a value must be entered or the command is ignored

* provides the same result.

^MP

Mode Protection

Description The ^MP command is used to disable the various mode functions on the front panel. Once disabled, the settings for the particular mode function can no longer be changed and the LED associated with the function does not light.

Because this command has only one parameter, each mode must be disabled with an individual ^MP command.

Format ^MPa

This table identifies the parameters for this format:

Parameters	Details
a = mode to protect	*Accepted Values:* D = disable Darkness Mode P = disable Position Mode C = disable Calibration Mode E = enable all modes S = disable all mode saves (modes can be adjusted but values are not saved) W = disable Pause F = disable Feed X = disable Cancel M = disable menu changes *Default Value:* a value must be entered or the command is ignored

Example • This example disables these modes, D and C.

```
^XA
^MPD
^MPC
^XZ
```

^MT

Media Type

Description The ^MT command selects the type of media being used in the printer. There are two choices for this command:

- Thermal Transfer Media – this media uses a high-carbon black or colored ribbon. The ink on the ribbon is bonded to the media.
- Direct Thermal Media – this media is heat sensitive and requires no ribbon.

Format ^MTa

This table identifies the parameters for this format:

Parameters	Details
a = media type used	*Accepted Values:* T = thermal transfer media D = direct thermal media *Default Value:* a value must be entered or the command is ignored

^MU

Set Units of Measurement

Description The ^MU command sets the units of measurement the printer uses. ^MU works on a field-by-field basis. Once the mode of units is set, it carries over from field to field until a new mode of units is entered.

^MU also allows for printing at lower resolutions — 600 dpi printers are capable of printing at 300, 200, and 150 dpi; 300 dpi printers are capable of printing at 150 dpi.

Format ^MUa,b,c

This table identifies the parameters for this format:

Parameters	Details
a = units	*Accepted Values:* D = dots I = inches M = millimeters *Default Value:* D
b = format base in dots per inch	*Accepted Values:* 150, 200, 300 *Default Value:* a value must be entered or the command is ignored
c = desired dots-per-inch conversion	*Accepted Values:* 300, 600 *Default Value:* a value must be entered or the command is ignored

Example • This is an example of Setting Units:

Assume 8 dot/millimeter (203 dot/inch) printer.

Field based on dots:

```
^MUd^FO100,100^GB1024,128,128^FS
```

Field based on millimeters:

```
^MUm^FO12.5,12.5^GB128,16,16^FS
```

Field based on inches:

```
^MUi^FO.493,.493^GB5.044,.631,.631^FS
```

Example • This is an example of Converting dpi Values.

Convert a 150 dpi format to a 300 dpi format with a base in dots:

```
^MUd,150,300
```

Convert a 150 dpi format to a 600 dpi format with a base in dots:

```
^MUd,150,600
```

Convert a 200 dpi format to a 600 dpi format with a base in dots:

```
^MUd,200,600
```

To reset the conversion factor to the original format, enter matching values for parameters `b` *and* `c`*:*

```
^MUd,150,150
^MUd,200,200
^MUd,300,300
^MUd,600,600
```

Comments This command should appear at the beginning of the label format to be in proper ZPL II format.

To turn the conversion off, enter matching values for parameter `b` and `c`.

^MW

Modify Head Cold Warning

Description The ^MW command allows you to set the head cold warning indicator based on the operating environment.

Format ^MWy

This table identifies the parameters for this format:

Parameters	Details
a = enable head cold warning	*Accepted Values:* y = enable head cold warning n = disable head cold warning



Important • When a parameter is **not** given, the instruction is *ignored*.

~NC

Network Connect

Description The ~NC command is used to connect a particular printer to a network by calling up the printer's network ID number.

Format ~NC###

This table identifies the parameters for this format:

Parameters	Details
### = network ID number assigned (must be a three-digit entry)	*Accepted Values:* 001 to 999 *Default Value:* 000 (none)

Comments Use this command at the beginning of any label format to specify which printer on the network is going to be used. Once the printer is established, it continues to be used until it is changed by another ~NC command. This command must be included in the label format to *wake up the printer.*

The commands ^MW, ~NC, ^NI, ~NR, and ~NT are used only with ZNET RS-485 printer networking.

^NI

Network ID Number

Description The ^NI command is used to assign a network ID number to the printer. This must be done before the printer can be used in a network.

Format ^NI###

This table identifies the parameters for this format:

Parameters	Details
### = network ID number assigned (must be a three-digit entry)	*Accepted Values:* 001 to 999 *Default Value:* 000 (none)

Comments The last network ID number set is the one recognized by the system.

The commands ~NC, ^NI, ~NR, and ~NT are used only with ZNET RS-485 printer networking.

~NR

Set All Network Printers Transparent

Description The ~NR command sets all printers in the network to be transparent, regardless of ID or current mode.

Format ~NR

Comments The commands ~NC, ^NI, ~NR, and ~NT are used only with ZNET RS-485 printer networking.

^NS

Change Networking Settings

Description The ^NS command is used to change network settings.

Format ^NSa,b,c,d

This table identifies the parameters for this format:

Parameters	Details
a = network setting	*Accepted Values:* IP Resolution. a (ALL) , b (BOOTP), c (DHCP and BOOTP), d (DHCP), g (GLEANING ONLY), r (RARP), p (permanent)
b = IP Address	*Accepted Values:* 0 to 255
c = Subnet Mask	*Accepted Values:* 0 to 255
d = Default Gateway	*Accepted Values:* 0 to 255

~NT

Set Currently Connected Printer Transparent

Description The ~NT command sets the currently connected network printer to be transparent.

Format ~NT

Comments With Z Series® printers, the ~NT command functions the same as the ~NR command. All Z Series printers on a network receive the transmission.

The commands ~NC, ^NI, ~NR, and ~NT are used only with ZNET RS-485 printer networking.

^PF

Slew Given Number of Dot Rows

Description The ^PF command causes the printer to slew labels (move labels at a high speed without printing) a specified number of dot rows from the bottom of the label. This allows faster printing when the bottom portion of a label is blank.

Format ^PF#

This table identifies the parameters for this format:

Parameters	Details
# = number of dots rows to slew	*Accepted Values:* 0 to 32000 *Default Value:* a value must be entered or the command is ignored

^PH ~PH

Slew to Home Position

Description The ^PH or ~PH command causes the printer to feed one blank label.

The ~PH command feeds one label after the format currently being printed is done or when the printer is placed in pause.

The ^PH command feeds one blank label after the current format prints.

Format ^PH or ~PH

^PM

Printing Mirror Image of Label

Description The ^PM command prints the entire printable area of the label as a mirror image. This command flips the image from left to right.

Format ^PMa

This table identifies the parameters for this format:

Parameters	Details
a = print mirror image of entire label	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

Example • This is an example of printing a mirror image on a label:

ZPL II CODE	GENERATED LABEL
^XA^PMY ^FO100,100 ^CFG ^FDMIRROR^FS ^FO100,160 ^FDIMAGE^FS ^XZ	

Comments If the parameter is missing or invalid, the command is ignored. Once entered, the ^PM command remains active until ^PMN is received or the printer is turned off.

^PO

Print Orientation

Description The ^PO command inverts the label format 180 degrees. The label appears to be printed upside down. If the original label contains commands such as ^LL, ^LS, ^LT and ^PF, the inverted label output is affected differently.

Format ^POa

This table identifies the parameters for this format:

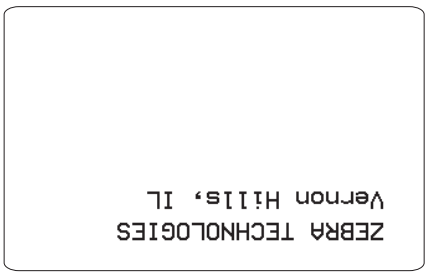
Parameters	Details
a = invert label 180 degrees	*Accepted Values:* N (normal) or I (invert) *Default Value:* N

Example • This is an example of printing a label at 180 degrees:

ZPL II CODE

```
^XA^CFD
^POI
^LH330,10
^FO50,50
^FDZEBRA TECHNOLOGIES^FS
^FO50,75
^FDVernon Hills, IL^FS
^XZ
```

GENERATED LABEL



The ^POI command inverts the x, y coordinates. All image placement is relative to these inverted coordinates. Therefore, a different ^LH (Label Home) can be used to move the print back onto the label.

Comments If multiple ^PO commands are issued in the same label format, only the last command sent to the printer is used.

Once the ^PO command is sent, the setting is retained until another ^PO command is received or the printer is turned off.

^PP ~PP

Programmable Pause

Description The ~PP command stops printing after the current label is complete (if one is printing) and places the printer in Pause Mode.

The ^PP command is not immediate. Therefore, several labels might print before a pause is performed. This command pauses the printer after the current format prints.

The operation is identical to pressing PAUSE on the front panel of the printer. The printer remains paused until PAUSE is pressed or a ~PS (Print Start) command is sent to the printer.

Format ^PP or ~PP

^PQ

Print Quantity

Description The ^PQ command gives control over several printing operations. It controls the number of labels to print, the number of labels printed before printer pauses, and the number of replications of each serial number.

Format ^PQq,p,r,o

This table identifies the parameters for this format:

Parameters	Details
q = total quantity of labels to print	*Accepted Value:* 1 to 99,999,999 *Default Value:* 1
p = pause and cut value (labels between pauses)	*Accepted Value:* 1 to 99,999,999 *Default Value:* 0 (no pause)
r = replicates of each serial number	*Accepted Value:* 0 to 99,999,999 replicates *Default Value:* 0 (no replicates)
o = override pause count	*Accepted Value:* Y (yes) or N (no) *Default Value:* N

If the o parameter is set to Y, the printer cuts but does not pause, and the printer does **not** pause after every group count of labels has been printed. With the o parameter set to N (default), the printer pauses after every group count of labels has been printed.

Example • This example shows the control over print operations:

^PQ50,10,1,Y: This example prints a total of 50 labels with one replicate of each serial number. It prints the total quantity in groups of 10, but does not pause after every group.

^PQ50,10,1,N: This example prints a total of 50 labels with one replicate of each serial number. It prints the total quantity in groups of 10, pausing after every group.

^PR

Print Rate

Description The ^PR command determines the media and slew speed (feeding a blank label) during printing.

The printer operates with the selected speeds until the setting is reissued or the printer is turned off.

The print speed is application-specific. Because print quality is affected by media, ribbon, printing speeds, and printer operating modes, it is very important to run tests for your applications.



Important • Some models go to default print speed when power is turned off.

Format ^PRp,s,b

This table identifies the parameters for this format:

Parameters	Details
p = print speed	*Accepted Values:* A or 2 = 50.8 mm/sec. (2 inches/sec.) B or 3 = 76.2 mm/sec. (3 inches/sec.) C or 4 = 101.6 mm/sec. (4 inches/sec.) 5 = 127 mm/sec.(5 inches/sec.) D or 6 = 152.4 mm/sec. (6 inches/sec.) E or 8 = 203.2 mm/sec. (8 inches/sec.) 9 = 220.5 mm/sec. 9 inches/sec.) 10 = 245 mm/sec.(10 inches/sec.) 11 = 269.5 mm/sec.(11 inches/sec.) 12 = 304.8 mm/sec. 12 inches/sec.) *Default Value:* A
s = slew speed	*Accepted Values:* A or 2 = 50.8 mm/sec. (2 inches/sec.) B or 3 = 76.2 mm/sec. (3 inches/sec.) C or 4 = 101.6 mm/sec. (4 inches/sec.) 5 = 127 mm/sec. 5 inches/sec.) D or 6 = 152.4 mm/sec. (6 inches/sec.) E or 8 = 203.2 mm/sec. (8 inches/sec.) 9 = 220.5 mm/sec. (9 inches/sec.) 10 = 245 mm/sec. (10 inches/sec.) 11 = 269.5 mm/sec. 11 inches/sec.) 12 = 304.8 mm/sec. 12 inches/sec.) *Default Value:* D
b = backfeed speed	*Accepted Values:* A or 2 = 50.8 mm/sec. (2 inches/sec.) B or 3 = 76.2 mm/sec. (3 inches/sec.) C or 4 = 101.6 mm/sec. (4 inches/sec.) 5 = 127 mm/sec.(5 inches/sec.) D or 6 = 152.4 mm/sec. (6 inches/sec.) E or 8 = 203.2 mm/sec. (8 inches/sec.) 9 = 220.5 mm/sec. 9 inches/sec.) 10 = 245 mm/sec. 10 inches/sec.) 11 = 269.5 mm/sec. 11 inches/sec.) 12 = 304.8 mm/sec. 12 inches/sec.) *Default Value:* A

Comments The speed setting for p, s, and b is dependent on the limitations of the printer. If a particular printer is limited to a rate of 6 ips (inches per second), a value of 12 can be entered but the printer performs only at a 6 ips rate. See your printer's users guide for specifics on performance.

~PR

Applicator Reprint

Description The ~PR command is supported only by the *PAX* and *PAX*2-Series printers. If the ~PR command is enabled (see ^*JJ* on page 194), the last label printed reprint, similar to the applicator asserting the Reprint signal on the applicator port.

Format ~PR

Comments Pressing PREVIOUS on the front panel also causes the last label to reprint.

~PS

Print Start

Description The ~PS command causes a printer in Pause Mode to resume printing. The operation is identical to pressing PAUSE on the front panel of the printer when the printer is already in Pause Mode.

Format ~PS

^PW

Print Width

Description The ^PW command allows you set the print width.

Format ^PWa

This table identifies the parameters for this format:

Parameters	Details
a = label width (in dots)	*Accepted Values:* 2, to the width of the label If the value exceeds the width of the label, the width is set to the label's maximum size. *Default Value:* last permanently saved value

Limitation Not all Zebra printers support the ^PW command.

~RO

Reset Advanced Counter

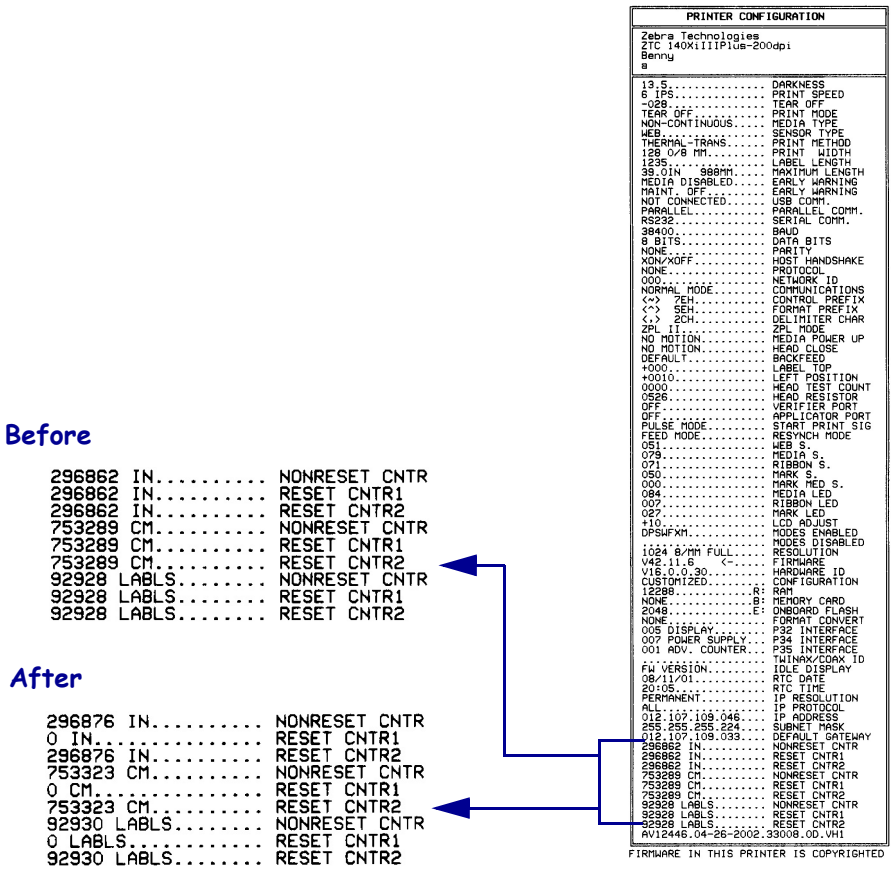
Description The ~RO command resets the advanced counters used by the printer to monitor label generation in inches, centimeters, and number of labels. Two resettable counters are available and can be reset.

Format ~ROc

This table identifies the parameters for this format:

Parameters	Details
c = counter number	*Accepted Values:* 1 or 2 *Default Value:* a value must be specified or the command is ignored

Example • This is an example of the ~RO command.



^SC

Set Serial Communications

Description The ^SC command allows you to change the serial communications parameters you are using.

Format ^SCa,b,c,d,e,f

This table identifies the parameters for this format:

Parameters	Details
a = baud rate	*Accepted Values:* 110; 600; 1,200; 2400; 4800; 9600; 14400; 19200; 28800; 38400; or 57600; 115200 *Default Value:* must be specified or the parameter is ignored
b = word length (in data bits)	*Accepted Values:* 7 or 8 *Default Value:* must be specified
c = parity	*Accepted Values:* N (none), E (even), or O (odd) *Default Value:* must be specified
d = stop bits	*Accepted Values:* 1 or 2 *Default Value:* must be specified
e = protocol mode	*Accepted Values:* X (XON/XOFF), D (DTR/DSR), or R (RTS) *Default Value:* must be specified
f = Zebra protocol	*Accepted Values:* A = ACK/NAK N = none Z = Zebra *Default Value:* must be specified

Comments If any of the parameters are missing, out of specification, not supported by a particular printer, or have a ZPL-override DIP switch set, the command is ignored.

A ^JUS command causes the changes in Communications Mode to persist through power-up and software resets.

~SD

Set Darkness

Description The ~SD command allows you to set the darkness of printing. ~SD is the equivalent of the darkness setting parameter on the front panel display.

Format ~SD##

This table identifies the parameters for this format:

Parameters	Details
## = desired darkness setting (two-digit number)	*Accepted Values:* 00 to 30 *Default Value:* last permanently saved value

Example • These are examples of the *Xi*III*Plus* Darkness Setting:

```
^MD8.3
~SD8.3
```

Comments The ^MD command value, if applicable, is added to the ~SD command.

^SE

Select Encoding

Description The ^SE command was created to select the desired ZPL or ZPL II encoding table.

Format ^SEd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = location of encoding table	*Accepted Values:* R:, E:, B:, and A: *Default Value:* R:
o = name of encoding table	*Accepted Value:* 1 to 8 alphanumeric characters *Default Value:* a value must be specified
x = extension	*Fixed Value:* .DAT

^SF

Serialization Field (with a Standard ^FD String)

Description The ^SF command allows you to serialize a standard ^FD string. Fields serialized with this command are right-justified or end with the last character of the string. The increment string is aligned with the mask, starting with the right-most position. The maximum size of the mask and increment string is 3K combined.

Format ^SFa,b

This table identifies the parameters for this format:

Parameters	Details
a = mask string	The mask string sets the serialization scheme. The length of the string mask defines the number of characters in the current ^FD string to be serialized. The mask is aligned to the characters in the ^FD string starting with the right-most position. *Mask String placeholders:* D or d – Decimal numeric 0–9 H or h – Hexadecimal 0–9 plus a-f or A-F O or o – Octal 0–7 A or a – Alphabetic a–z or A–Z N or n – Alphanumeric 0–9 plus a–z or A–Z % – Ignore character or skip
b = increment string	The increment string is the value to be added to the field on each label. The default value is equivalent to a decimal value of one. The string is composed of any characters defined in the serial string. Invalid characters are assumed to be equal to a value of zero in that character position. The increment value for alphabetic strings start with 'A' or 'a' as the zero placeholder. This means to increment an alphabetic character by one, a value of 'B' or 'b' must be in the increment string.

For characters that do not get incremented, the % character needs to be added to the increment string.

Example • This is an example of serializing a ^FD string:

ZPL II CODE	GENERATED LABELS
^XA ^FO100,100 ^CF0,100 ^FD12A^SFnnA,F^FS ^PQ3 ^XZ ***Note: The ZPL II code above will generate three separate labels, seen to the right.***	

This mask has the first characters as alphanumeric (nn = 12) and the last digit as uppercase alphabetic (A). The decimal value of the increment number is equivalent to 5 (F). The number of labels generated depends on the number specified by the ^PQ command.

In a similar instance, the ^FD string could be replaced with either of the ^FD strings below to generate a series of label, determined by ^PQ.

```
^FDBL0000^SFAAdddd,1
```

The print sequence on this series of labels is:

```
BL0000, BL0001,...BL0009, BL0010,...
BL0099, BL0100,...BL9999, BM0000...
```

```
^FDBL00-0^SFAAdd%d,1%1
```

The print sequence on this series of labels is:

```
BL00-0, BL01-1, BL02-2,...BL09-9,
BL11-0, BL12-1...
```

^SL

Set Mode and Language (for Real-Time Clock)

Description The ^SL command is used to specify the Real-Time Clock's mode of operation and language for printing information.

Format ^SLa,b

This table identifies the parameters for this format:

Parameters	Details
a = mode	*Accepted Values:* S = Start Time Mode. This is the time that is read from the Real-Time Clock when label formatting begins (when ^XA is received). The first label has the same time placed on it as the last label. T = Time Now Mode. This is the time that is read from the Real-Time Clock when the label to be printed is placed in print queue. *Time Now* is similar to a serialized time or date field. *Default Value:* S
b = language	*Accepted Values:* 1 = English 2 = Spanish 3 = French 4 = German 5 = Italian 6 = Norwegian 7 = Portuguese 8 = Swedish 9 = Danish 10 = Spanish 2 11 = Dutch 12 = Finnish *Default Value:* the language selected with ^KL or the front panel

^SN

Serialization Data

Description The ^SN command allows the printer to index data fields by a selected increment or decrement value, making the data fields increase or decrease by a specified value each time a label is printed. This can be performed on 100 to 150 fields in a given format and can be performed on both alphanumeric and bar code fields. A maximum of 12 of the right-most integers are subject to indexing. The first integer found when scanning from right to left starts the indexing portion of the data field.

If the alphanumeric field to be indexed ends with an alpha character, the data is scanned, character by character, from right to left until a numeric character is encountered. Serialization takes place using the value of the first number found.

Format ^SNv,n,z

This table identifies the parameters for this format:

Parameters	Details
v = starting value	*Accepted Values:* 12-digits maximum for the portion to be indexed *Default Value:* 1
n = increment or decrement value	*Accepted Values:* 12-digit maximum *Default Value:* 1 To indicate a decrement value, precede the value with a minus (–) sign.
z = add leading zeros (if needed)	*Accepted Values:* Y (yes) or N (no) *Default Value:* N

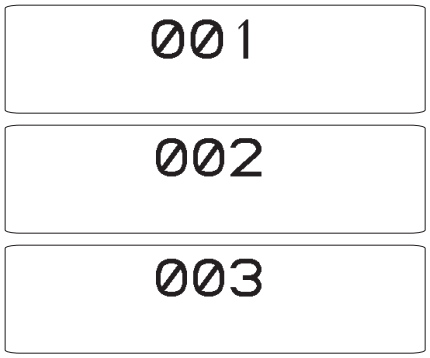
Example • This example shows incrementing by a specified value:

ZPL II CODE

```
^XA
^FO260,110
^CFG
^SN001,1,Y^FS
^PQ3
^XZ
```

Note: The ZPL II code above will generate three separate labels, seen to the right.

GENERATED LABELS



Comments Incrementing and decrementing takes place for each serial-numbered field when all replicates for each serial number have been printed, as specified in parameter r of the ^PQ (print quality) command.

If, during the course of printing serialized labels, the printer runs out of either paper or ribbon, the first label printed (after the media or ribbon has been replaced and calibration completed) has the same serial number as the *partial* label printed before the *out* condition occurred. This is done in case the last label before the *out* condition did not fully print. This is controlled by the ^JZ command.

Using Leading Zeros

In the ^SN command, the z parameter determines if leading zeros are printed or suppressed. Depending on which value is used (Y = print leading zeros; N = do not print leading zeros), the printer either prints or suppresses the leading zeros.

The default value for this parameter is N (do not print leading zeros).

Print Leading Zeros

The starting value consists of the right-most consecutive sequence of digits. The width (number of digits in the sequence) is determined by scanning from right to left until the first non-digit (space or alpha character) is encountered. To create a specific width, manually place leading zeros as necessary.

Suppressing Leading Zeros

The starting value consists of the right-most consecutive sequence of digits, including any leading spaces. The width (number of digits in the sequence) is determined by scanning from right to left until the first alpha character (except a space) is encountered. To create a specific width, manually place leading spaces or zeros as necessary. Suppressed zeros are replaced by spaces. During the serialization process, when the entire number contains all zeros, the last zero is not suppressed.

The ^SN command replaces the Field Data (^FD) command within a label formatting program.

^SO

Set Offset (for Real-Time Clock)

Description The ^SO command is used to set the secondary and the tertiary offset from the primary Real-Time Clock.

Format ^SOa,b,c,d,e,f,g

This table identifies the parameters for this format:

Parameters	Details
a = clock set	*Accepted Values:* 2 (secondary) or 3 (tertiary) *Default Value:* value must be specified
b = months offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0
c = days offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0
d = years offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0
e = hours offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0
f = minutes offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0
g = seconds offset	*Accepted Values:* −32000 to 32000 *Default Value:* 0

^SP

Start Print

Description The ^SP command allows a label to start printing at a specified point before the entire label has been completely formatted. On extremely complex labels, this command can increase the overall throughput of the print.

The command works as follows: Specify the dot row at which the ^SP command is to begin. This creates a label *segment*. Once the ^SP command is processed, all information in that segment prints. During the printing process, all of the commands after the ^SP continue to be received and processed by the printer.

If the segment after the ^SP command (or the remainder of the label) is ready for printing, media motion does not stop. If the next segment is not ready, the printer stops mid-label and wait for the next segment to be completed. Precise positioning of the ^SP command requires a trial-and-error process, as it depends primarily on print speed and label complexity.

The ^SP command can be effectively used to determine the worst possible print quality. You can determine whether using the ^SP command is appropriate for the particular application by using this procedure.

If you send the label format up to the first ^SP command and then wait for printing to stop before sending the next segment, the printed label is a sample of the worst possible print quality. It drops any field that is out of order.

If the procedure above is used, the end of the label format must be:

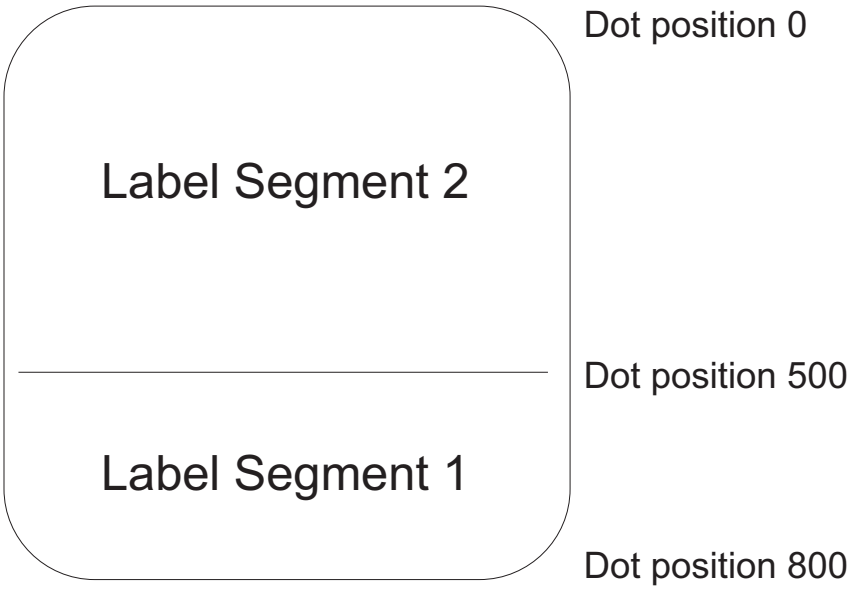
```
^SP#^FS
```

Format ^SPa

This table identifies the parameters for this format:

Parameters	Details
a = dot row to start printing	*Accepted Values:* 0 to 32000 *Default Value:* 0

Example • In this example, a label 800 dot rows in length uses ^SP500. Segment 1 prints while commands in Segment 2 are being received and formatted.



^SQ

Halt ZebraNet Alert

Description The ^SQ command is used to stop the ZebraNet Alert option.

Format ^SQa,b,c

This table identifies the parameters for this format:

Parameters	Details
a = condition type	*Accepted Values:* A = paper out B = ribbon out C = printhead over-temp D = printhead under-temp E = head open F = power supply over-temp G = ribbon-in warning (Direct Thermal Mode) H = rewind full I = cut error J = printer paused K = PQ job completed L = label ready M = head element out N = ZBI (Zebra BASIC Interpreter) runtime error O = ZBI (Zebra BASIC Interpreter) forced error P = power on Q = clean printhead R = media low S = ribbon low T = replace head U = battery low V = all errors
b = destination	*Accepted Values:* A = serial port B = parallel port C = e-mail address D = TCP/IP E = UDP/IP F = SNMP trap * = wild card to stop alerts for all destinations
c = halt messages	*Accepted Values:* Y (halt messages) or N (start messages) *Default Value:* Y

^SR

Set Printhead Resistance

Description The ^SR command allows you to set the printhead resistance.

Format ^SR####

This table identifies the parameters for this format:

Parameters	Details
#### = resistance value (four-digit numeric value)	*Accepted Value:* 0488 to 1175 *Default Value:* last permanently saved value

Comments To avoid damaging the printhead, this value should be less than or equal to the value shown on the printhead being used. Setting a higher value could damage the printhead.



Note • New models automatically set head resistance.

^SS

Set Media Sensors

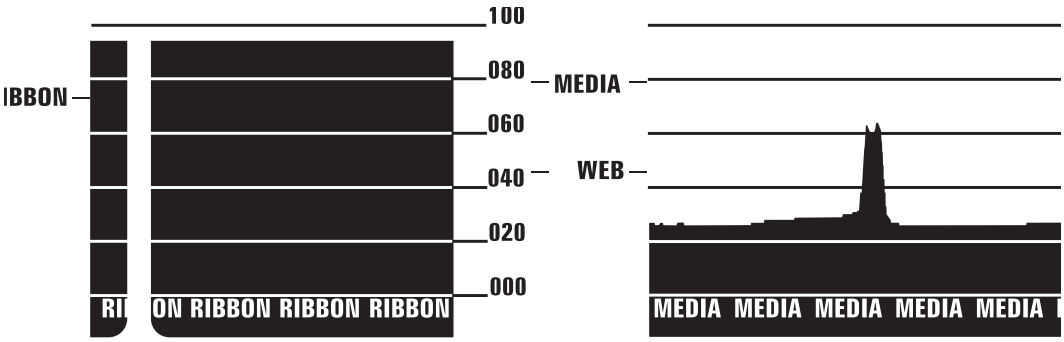
Description The ^SS command is used to change the values for media, web, ribbon, and label length set during the media calibration process. The media calibration process is described in your specific printer's user's guide.

Format ^SSw,m,r,l,m2,r2,a,b,c

This table identifies the parameters for this format:

Parameters	Details
w = web (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value shown on the media sensor profile or configuration label
m = media (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value shown on the media sensor profile or configuration label
r = ribbon (three-digit value)	*Accepted Values:* 001 to 100 *Default Value:* value shown on the media sensor profile or configuration label
l = label length (in dots, four-digit value)	*Accepted Values:* 0001 to 32000 *Default Value:* value calculated in the calibration process
m2 = intensity of media LED (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value calculated in the calibration process
r2 = intensity of ribbon LED (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value calculated in the calibration process
a = mark sensing (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value calculated in the calibration process
b = mark media sensing (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value calculated in the calibration process
c = mark LED sensing (three-digit value)	*Accepted Values:* 000 to 100 *Default Value:* value calculated in the calibration process

Example • Below is an example of a media sensor profile. Notice the numbers from 000 to 100 and where the words WEB, MEDIA, and RIBBON appear in relation to those numbers. Also notice the black vertical spike. This represents where the printer sensed the transition from media-to-web-to-media.



The media and sensor profiles produced vary in appearance from printer to printer.

Comments The `m2` and `r2` parameters have no effect in Stripe® *S*-300 and *S*-500 printers.

Maximum values for parameters depend on which printer platform is being used.

^ST

Set Date and Time (for Real-Time Clock)

Description The ^ST command sets the date and time of the Real-Time Clock.

Format ^STa,b,c,d,e,f,g

This table identifies the parameters for this format:

Parameters	Details
a = month	*Accepted Values:* 01 to 12 *Default Value:* current month
b = day	*Accepted Values:* 01 to 31 *Default Value:* current day
c = year	*Accepted Values:* 1998 to 2097 *Default Value:* current year
d = hour	*Accepted Values:* 00 to 23 *Default Value:* current hour
e = minute	*Accepted Values:* 00 to 59 *Default Value:* current minute
f = second	*Accepted Values:* 00 to 59 *Default Value:* current second
g = format	*Accepted Values:* A = a.m. P = p.m. M = 24-hour military *Default Value:* M

^SX

Set ZebraNet Alert

Description The ^SX command is used to configure the ZebraNet Alert System.

Format ^SXa,b,c,d,e,f

This table identifies the parameters for this format:

Note • The values in this table apply to firmware V48_12_4 and above.

Parameters	Details
a = condition type	*Accepted Values:* A = paper out B = ribbon out C = printhead over-temp D = printhead under-temp E = head open F = power supply over-temp G = ribbon-in warning (Direct Thermal Mode) H = rewind full I = cut error J = printer paused K = PQ job completed L = label ready M = head element out N = ZBI (Zebra BASIC Interpreter) runtime error O = ZBI (Zebra BASIC Interpreter) forced error P = power on Q = clean printhead R = media low S = ribbon low T = replace head U = battery low V = all errors *Default Value:* if the parameter is missing or invalid, the command is ignored
b = destination for route alert	*Accepted Values:* A = serial port B* = parallel port C = e-mail address D = TCP/IP E = UDP/IP F = SNMP trap *Default Value:* if this parameter is missing or invalid, the command is ignored *** Requires** bidirectional communication.

Parameters	Details
c = enable *condition set* alert to this destination	*Accepted Values:* Y (yes) or N (no) *Default Value:* Y or previously configured value
d = enable condition clear alert to this destination	*Accepted Values:* Y (yes) or N (no) *Default Value:* N or previously configured value Parameters e and f are sub-options based on destination. If the sub-options are missing or invalid, these parameters is ignored.
e = destination setting	*Accepted Values:* Internet e-mail address (e.g. user@company.com) IP address (for example, 10.1.2.123) SNMP trap IP or IPX addresses
f = port number	*Accepted Values:* TCP port # (0 to 65535) UPD port # (0 to 65535)

Example • This is an example of the different (b) destinations that you can send for the condition type (a):

Serial: ^SXA,A,Y,Y
Parallel: ^SXA,B,Y,Y
E-Mail: ^SXA,C,Y,Y,admin@company.com
TCP: ^SXA,D,Y,Y,123.45.67.89,1234
UDP: ^SXA,E,Y,Y,123.45.67.89,1234
SNMP Trap: ^SXA,F,Y,Y,255.255.255.255

Comments In the example above for SNMP Trap, entering 255.255.255.255 broadcasts the notification to every SNMP manager on the network. To route the device to a single SNMP manager, enter a specific address (123.45.67.89).

^SZ

Set ZPL

Description The ^SZ command is used to select the programming language used by the printer. This command gives you the ability to print labels formatted in both ZPL and ZPL II.

This command remains active until another ^SZ command is sent to the printer or the printer is turned off.

Format ^SZa

This table identifies the parameters for this format:

Parameters	Details
a = ZPL version	*Accepted Values:* 1 = ZPL 2 = ZPL II *Default Value:* 2

Comments If the parameter is missing or invalid, the command is ignored.

~TA

Tear-off Adjust Position

Description The ~TA command lets you adjust the rest position of the media after a label is printed, which changes the position at which the label is torn or cut.

Format ~TA###

Important • These are some important facts about this command:

- For 600 dpi printers, the step size doubles.
- If the number of characters is **less than** 3, the command is ignored.

This table identifies the parameters for this format:

Parameters	Details
### = change in media rest position **Important •** 3-digit value in dot rows must be used.	*Accepted Values:* -120 to 120 *Default Value:* last permanent value saved

Comments If the parameter is missing or invalid, the command is ignored.

^TO

Transfer Object

Description The ^TO command is used to copy an object or group of objects from one storage device to another. It is similar to the copy function used in PCs.

Source and destination devices must be supplied and must be different and valid for the action specified. Invalid parameters cause the command to be ignored.

The asterisk (*) can be used as a wild card for object names and extensions. For instance, ZEBRA.* or *.GRF are acceptable forms for use with the ^TO command.

At least one source parameter (d, o, or x) and one destination parameter (s, o, or x) must be specified. If only ^TO is entered, the command is ignored.

Format ^TOd:o.x,s:o.x

This table identifies the parameters for this format:

Parameters	Details
d = source device of stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* if a drive is not specified, all objects are transferred to the drive set in parameter s
o = stored object name	*Accepted Values:* any existing object conforming to Zebra conventions *Default Value:* if a name is not specified, * is used — all objects are selected
x = extension	*Accepted Values:* any extension conforming to Zebra conventions *Default Value:* if an extension is not specified, * is used — all extensions are selected
s = destination device of the stored object	*Accepted Values:* R:, E:, B:, and A: *Default Value:* a destination must be specified
o = name of the object at destination	*Accepted Values:* up to 8 alphanumeric characters *Default Value:* if a name is not specified, the name of the existing object is used
x = extension	*Accepted Values:* any extension conforming to Zebra conventions *Default Value:* if an extension is not specified, the extension of the existing object is used

Comments Parameters o, x, and s support the use of the wild card (*).

If the destination device does not have enough free space to store the object being copied, the command is canceled.

Zebra files (Z:*.*) cannot be transferred. These files are copyrighted by Zebra Technologies.

Transferring Objects

These are some examples of using the ^TO command.

To copy the object ZLOGO.GRF from DRAM to an optional Memory Card and rename it ZLOGO1.GRF, write the following format:

```
^XA
^TOR:ZLOGO.GRF,B:ZLOGO1.GRF
^XZ
```

To copy the object SAMPLE.GRF from an optional Memory Card to DRAM and keep the same name, write this format:

```
^XA
^TOB:SAMPLE.GRF,R:SAMPLE.GRF
^XZ
```

Transferring Multiple Objects

The asterisk (*) can be used to transfer multiple object files (except *.FNT) from DRAM to the Memory Card. For example, assume you have several object files that contain logos. These files are named LOGO1.GRF, LOGO2.GRF, and LOGO3.GRF.

To transfer all these files to the memory card using the name NEW instead of LOGO, place an asterisk after the names NEW and LOGO in the transfer command. This copies all files beginning with LOGO in one command.

```
^XA
^TOR:LOGO*.GRF,B:NEW*.GRF
^XZ
```

During a multiple transfer, if a file is too big to be stored on the memory card, that file is skipped. All remaining files attempt to be transferred. All files that can be stored within the space limitations are transferred, while other files are ignored.

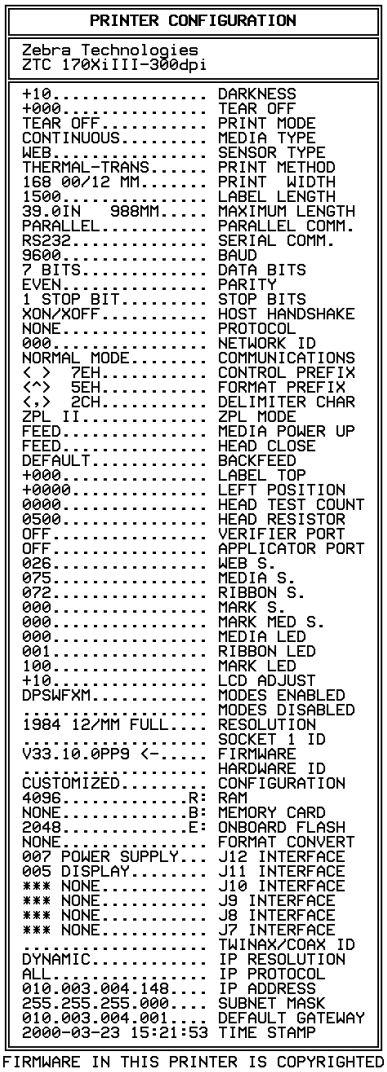
~WC

Print Configuration Label

Description The ~WC command is used to generate a printer configuration label. The printer configuration label contains information about the printer setup, such as sensor type, network ID, ZPL mode, firmware version, and descriptive data on the R:, E:, B:, and A: devices.

Format ~WC

Comments This command works only when the printer is idle.



PRINTER CONFIGURATION

Zebra Technologies
ZTC 170XiIII-300dpi

+10	DARKNESS
+000	TEAR OFF
TEAR OFF	PRINT MODE
CONTINUOUS	MEDIA TYPE
WEB	SENSOR TYPE
THERMAL-TRANS	PRINT METHOD
168 00/12 MM	PRINT WIDTH
1500	LABEL LENGTH
39.0IN 988MM	MAXIMUM LENGTH
PARALLEL	PARALLEL COMM.
RS232	SERIAL COMM.
9600	BAUD
7 BITS	DATA BITS
EVEN	PARITY
1 STOP BIT	STOP BITS
XON/XOFF	HOST HANDSHAKE
NONE	PROTOCOL
000	NETWORK ID
NORMAL MODE	COMMUNICATIONS
< > 7EH	CONTROL PREFIX
<^> 5EH	FORMAT PREFIX
<,> 2CH	DELIMITER CHAR
ZPL II	ZPL MODE
FEED	MEDIA POWER UP
FEED	HEAD CLOSE
DEFAULT	BACKFEED
+000	LABEL TOP
+0000	LEFT POSITION
0000	HEAD TEST COUNT
0500	HEAD RESISTOR
OFF	VERIFIER PORT
OFF	APPLICATOR PORT
026	WEB S.
075	MEDIA S.
072	RIBBON S.
000	MARK S.
000	MARK MED S.
000	MEDIA LED
001	RIBBON LED
100	MARK LED
+10	LCD ADJUST
DPSWFXM	MODES ENABLED
	MODES DISABLED
1984 12/MM FULL	RESOLUTION
	SOCKET 1 ID
V33.10.0PP9 <-	FIRMWARE
	HARDWARE ID
CUSTOMIZED	CONFIGURATION
4096 R:	RAM
NONE B:	MEMORY CARD
2048 E:	ONBOARD FLASH
NONE	FORMAT CONVERT
007 POWER SUPPLY	J12 INTERFACE
005 DISPLAY	J11 INTERFACE
*** NONE	J10 INTERFACE
*** NONE	J9 INTERFACE
*** NONE	J8 INTERFACE
*** NONE	J7 INTERFACE
	TWINAX/COAX ID
DYNAMIC	IP RESOLUTION
ALL	IP PROTOCOL
010.003.004.148	IP ADDRESS
255.255.255.000	SUBNET MASK
010.003.004.001	DEFAULT GATEWAY
2000-03-23 15:21:53	TIME STAMP

FIRMWARE IN THIS PRINTER IS COPYRIGHTED

^WD

Print Directory Label

Description The ^WD command is used to print a label listing bar codes, objects stored in DRAM, or fonts.

For bar codes, the list shows the name of the bar code. For fonts, the list shows the name of the font, the number to use with ^Af command, and size. For objects stored in DRAM, the list shows the name of the object, extension, size, and option flags. All lists are enclosed in a double-line box.

Format ~WDd:o.x

This table identifies the parameters for this format:

Parameters	Details
d = source device — optional	*Accepted Values:* R:, E:, B:, A: and Z: *Default Value:* R:
o =object name — optional	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* * The use of a ? (question mark) is also allowed.
x = extension — optional	*Accepted Values:* any extension conforming to Zebra conventions .FNT = font .BAR = bar code .ZPL = stored ZPL format .GRF = GRF graphic .CO = memory cache .DAT = font encoding .BAS = ZBI program .STO = data storage .PNG = PNG graphic * = all objects *Default Value:* * The use of a ? (question mark) is also allowed.

Example • To print a label listing all objects in DRAM, enter:

```
^XA
^WDR:*.*
^XZ
```

Example • To print a label listing all resident bar codes, enter:

```
^XA
^WDZ:*.BAR
^XZ
```

Example • To print a label listing all resident fonts, enter:

```
^XA
^WDZ:*.FNT
^XZ
```

^XA

Start Format

Description The ^XA command is used at the beginning of ZPL II code. It is the opening bracket and indicates the start of a new label format. This command is substituted with a single ASCII control character STX (control-B, hexadecimal 02).

Format ^XA

Comments Valid ZPL II format requires that label formats should start with the ^XA command and end with the ^XZ command.

^XB

Suppress Backfeed

Description The ^XB command suppresses forward feed of media to tear-off position depending on the current printer mode. Because no forward feed occurs, a backfeed before printing of the next label is not necessary; this improves throughput. When printing a batch of labels, the last label should not contain this command.

Format ^XB

^XB in the Tear-off Mode

Normal Operation: backfeed, print, and feed to rest

^XB *Operation:* print (Rewind Mode)

^XB in Peel-off Mode

Normal Operation: backfeed, print, and feed to rest

^XB *Operation:* print (Rewind Mode)

^XF

Recall Format

Description The ^XF command recalls a stored format to be merged with variable data. There can be multiple ^XF commands in one format, and they can be located anywhere within the code.

When recalling a stored format and merging data using the ^FN (Field Number) function, the calling format must contain the ^FN command to merge the data properly.

While using stored formats reduces transmission time, no formatting time is saved. The ZPL II format being recalled is saved as text strings that need to be formatted at print time.

Format ^XFd:o.x

This table identifies the parameters for this format:

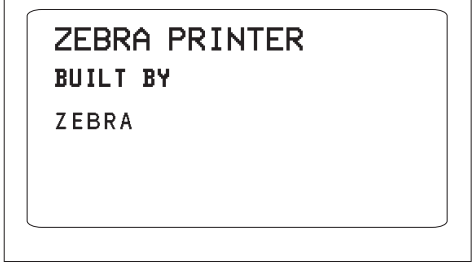
Parameters	Details
d = source device of stored image	*Accepted Values:* R:, E:, B:, and A: *Default Value:* search priority (R:, E:, B:, and A:)
o = name of stored image	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension l	*Fixed Value:* .ZPL

Example • This is an example of using the ^XF command to recall the format STOREFMT.ZPL from DRAM and insert new reference data in the ^FN fields:

ZPL II CODE

```
^XA
^XFR:STOREFMT.ZPL^FS
^FN1^FDZEBRA^FS
^FN2^FDLABEL^FS
^XZ
```

GENERATED LABEL



^XG

Recall Graphic

Description The ^XG command is used to recall one or more graphic images for printing. This command is used in a label format to merge graphics, such as company logos and piece parts, with text data to form a complete label.

An image can be recalled and resized as many times as needed in each format. Other images and data might be added to the format.

Format ^XGd:o.x,mx,my

This table identifies the parameters for this format:

Parameters	Details
d = source device of stored image	*Accepted Values:* R:, E:, B:, and A: *Default Value:* search priority (R:, E:, B:, and A:)
o = name of stored image	*Accepted Values:* 1 to 8 alphanumeric characters *Default Value:* if a name is not specified, UNKNOWN is used
x = extension l	*Fixed Value:* .GRF
mx = magnification factor on the x-axis	*Accepted Values:* 1 to 10 *Default Value:* 1
my = magnification factor on the y-axis	*Accepted Values:* 1 to 10 *Default Value:* 1

Example • This is an example of using the ^XG command to recall the image SAMPLE.GRF from DRAM and print it in five different sizes in five different locations on the same label:

```
^XA
^FO100,100^XGR:SAMPLE.GRF,1,1^FS
^FO100,200^XGR:SAMPLE.GRF,2,2^FS
^FO100,300^XGR:SAMPLE.GRF,3,3^FS
^FO100,400^XGR:SAMPLE.GRF,4,4^FS
^FO100,500^XGR:SAMPLE.GRF,5,5^FS
^XZ
```

^XZ

End Format

Description The ^XZ command is the ending (closing) bracket. It indicates the end of a label format. When this command is received, a label prints. This command can also be issued as a single ASCII control character ETX (Control-C, hexadecimal 03).

Format ^XZ

Comments Label formats must start with the ^XA command and end with the ^XZ command to be in valid ZPL II format.

^ZZ

Printer Sleep

Description The ^ZZ command places the printer in an idle or shutdown mode.

Format ^ZZt,b

This table identifies the parameters for this format:

Parameters	Details
t = number of second (idle time) prior to shutdown	*Accepted Values:* 0 to 999999 – setting 0 disables automatic shutdown *Default Value:* last permanently saved value or 0
b = label status at shutdown	*Accepted Values:* Y = indicates to shutdown when labels are still queued N = indicates all labels must be printed before shutting down *Default Value:* N

Comments The ^ZZ command is only valid on the PA400 and PT400 battery-powered printers.

3

ZBI Commands

This section contains an alphabetical listing of the ZBI commands. The version of firmware you are using can affect if the ZBI code is recognized.

The text in this section is arranged under these headings:

Description Under this heading is a description of how the command is used, what it is capable of, and any defining characteristics it has.

Format The format is how the command is arranged and what parameters it contains. For example, the AUTONUM command starts the auto-numbering option. The format for the command is AUTONUM <A>,<B>. The <A> and <B> are parameters of this command and are replaced with values determined by the user.

Parameters If a command has values that can be defined to make command function more specific, they are listed under this heading. Still using the AUTONUM example, the <A> parameter is defined as:

<A> = number used to start the auto-numbering sequence

Example When a command is best clarified in context, an example of the ZBI code is provided. Text indicating parameters, exact code to be entered, or data returned from the host is printed in the Courier font to be easily recognizable.

Example • An example of AUTONUM code is:

```
AUTONUM 10,5
10 PRINT "HELLO WORLD"
15 GOTO 10
```

In an example, the > symbol indicates a line of code you enter.

Comments This section is reserved for notes that are of value to a programmer, warnings of potential command interactions, or command-specific information that should be taken into consideration. An example comment could be: This is a program command and must be preceded by a line number.

AND

Description The AND statement is a Boolean operator. If both of the expressions are true, the result is true; otherwise, the result is false.

Format `<Boolean expression> AND <Boolean expression>`

Comments This is a Boolean operator that is used in conjunction with Boolean expressions.

Arrays

Description An array is a collection of values used by a program. Arrays share these characteristics:

- arrays are allowed for both integer and string variables.
- indexes of arrays are accessed through parentheses.
- array indexes start at 1 and end at the length of an array (for example, variable 3 returns the value in the third location of the variable array).
- one- and two-dimensional arrays are allowed. Two-dimensional arrays are referenced with two indexes in parentheses, separated by a comma.
- arrays can be re-dimensioned only by a call to DECLARE, which destroys the original array.
- array size is limited only by the size of the memory heap allocated.
- if an array cannot be allocated, an error message displays — `Error: Heap overflow.`
- if you attempt to access an array outside of its limits, an error message shows — `Error: Invalid array access.`

AUTONUM

Description This command automatically generates sequential program line numbers. This feature is disabled by overwriting the current line number and entering the desired interactive mode commands, or leaving the line blank.

Format `AUTONUM A,B`

Parameters:

`A` = the number used to start the auto-numbering sequence

`B` = the automatic increment between the new line numbers

Example • This example shows specifying the starting line number in the increment between new line number:

```
AUTONUM 10,5
10 PRINT "HELLO WORLD"
15 GOTO 10
```

Comments The two lines are automatically started with the AUTONUM parameters; in this case, the first line starts with 10 and each subsequent line in incremented by 5.

This is an interactive command that takes effect as soon as it is received by the printer.

Boolean Expression

Description A Boolean expression holds 0 (zero) as false and non-zero as true.

Formats <string expression> <Boolean compare> <string expression>

<# expression> <Boolean compare> <# expression>

(<Boolean expression>)

Comments # indicates a numeric expression. A numeric expression cannot be compared to a string expression. If attempted, an error message displays — Error: Poorly formed expression.

Numeric expressions can substitute a Boolean expression where a value of 0 (zero) represents false and a non-zero value represents true.

Order of precedence for <, <=, >, >=, = are all the same order followed by NOT, AND, and OR, in that order. Items with the same order of precedence are processed from left to right.

BREAK

Description This command is available only when the DEBUG function has been activated. When DEBUG is on, BREAK halts processing. RUN starts the program from the beginning. RESTART allows the program to continue from where it left off.

Format BREAK

Comments This is a program command that is preceded by a line number.

Channels

Description **I/O** commands can be issued to channels to direct the commands to specific ports. The data-input port is specified through the ZPL commands that start the ZBI interpreter. It is through this port that all the interactive communications take place.

Format #<channel expression>

Parameters:

#<channel expression>

Accepted Values: 0 to 9

Default Value: 0

CLOSE

Description This command is implemented to close specific ports that are in use. If a port is open on a channel and the CLOSE command is entered, the port closes and returns to communicating with the ZPL buffer.

Format CLOSE <channel expression>

Parameters:

<channel expression>

Accepted Values: 0 through 9

Default Value: will not be used

Example • This example shows the closing of channel 1:

```
>CLOSE #1
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

CLRERR

Description This command sends a message to clear the error flag to the printer. The error flag resets if the error is nonpermanent.

Format 10 CLRERR

Comments This is a program command that is preceded by a line number.

CTRL-C

Description Sending 03 to port 0 or using the Ctrl-C keystroke combination terminates any ZBI program currently running.

DEBUG

Description When DEBUG is on, the TRACE and BREAK options are enabled. When DEBUG is off, the TRACE and BREAK options are disabled.

Format DEBUG <ON/OFF>

Parameters:

<ON/OFF> = toggles the debug mode on or off

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

Declaration

Description Some common characteristics of declarations are:

- The names of variables are typically established at the beginning of a ZBI program. When declared, integer variables are initialized to 0. When declared, string variables are initialized to empty strings (“ ”).
- An explicit declaration deletes and reassigns any previously declared variable.
- An implicit declaration uses a variable name that is not previously defined. The value of this variable is undefined.
- Nonarray string and integer variables might be declared implicitly or explicitly.
- Arrays must be explicitly defined.

DECLARE

Description This is the explicit method of declaring a variable and is typically performed at the beginning of a program. Arrays are specified by placing a set of closed parentheses around a numeric expression of the array size to be allocated.

Format `DECLARE <type> <variable name> [,<variable name>]*`

Parameters:

`<type>` = numeric or string

`<variable name>` = the variable being declared, which could be an array

Example • This is an example of declaring a variable:

```
10 DECLARE NUMERIC A ! Declare an integer
20 DECLARE STRING A$ ! Declare a string
30 DECLARE NUMERIC My_Array(10) ! Declare an integer array
of size 10
40 DECLARE STRING My_Str_Array$(10) ! Declare a string
array with 10 strings
50 DECLARE NUMERIC A, B(10), C ! Declare multiple integer
variables
60 DECLARE STRING A$, B$, C$ ! Declare multiple string
variables
70 DECLARE STRING A2D$(5,5) ! Declare A2-D array
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

DELETE

Description This command removes a specified file from the printer’s memory.

Format `DELETE <"filename">`

Parameters <"filename"> = the name of the file to be deleted. Drive location and filename must be in quotation marks.

Example • This is an example of deleting a specified file from printer memory:

```
>DELETE "E:PROGRAM1.BAS"
```

Comments This is an interactive command that takes effect as soon as it is received by the printer.

DIR

Description This command, with no filter included, prompts the printer to list all the ZBI programs residing in all memory locations in the printer.

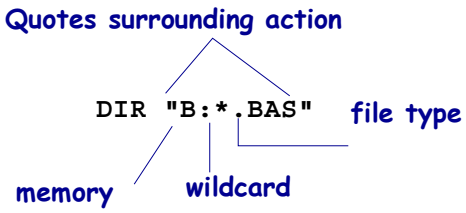
Including a filter signals the printer to limit the search; including a drive location signals the printer to search in only one location.

Asterisks (*) are used as wild cards. A wild card (*) finds every incidence of a particular request. The example here, DIR "B:*.BAS", signals the printer to search for every file with a .BAS extension in B: memory.

Format DIR ["filter"]

Parameters ["filter"] = the name of the file to be accessed (optional). Drive location and file name must be in quotation marks.

Important • Quotes must be around what you are doing. This shows you how to use the wildcard (*) to search for all .BAS files in B: memory:



Comments This is an interactive command that takes effect as soon as it is received by the printer.

DO-LOOP

Description Processing of the loop is controlled by a <WHILE/UNTIL> expression located on the DO or LOOP line.

Processing a WHILE statement is the same on either the DO or LOOP lines. The Boolean expression is evaluated and if the statement is true, the LOOP continues at the line after the DO statement. Otherwise, the line after the corresponding LOOP is the next in line to be processed.

Processing an UNTIL statement is the same on either the DO or LOOP lines. The Boolean expression is evaluated and if the statement is false, the LOOP continues at the line after the DO statement. Otherwise, the line after the corresponding LOOP is the next to be processed.

If <WHILE/UNTIL> is on the LOOP line, the BODY of the loop is executed before the Boolean expression is evaluated.

If neither the DO or LOOP line has a <WHILE/UNTIL> statement, the loop continues indefinitely.

Some notes about DO-LOOPs:

- can be nested
- cannot overlap
- have two formats.

Format 1

```
DO <WHILE/UNTIL> <Boolean expression>
~~BODY~~
LOOP
```

Example • This is an example of how to use the DO-LOOP command:

```
10 DO WHILE A$="70"
20 INPUT A$
30 LOOP
```

Format 2

```
DO
~~BODY~~
LOOP <WHILE/UNTIL> <Boolean expression>
```

Example • This is an example of how to use the DO UNTIL LOOP command:

```
10 DO
20 INPUT A$
30 LOOP UNTIL A$="EXIT"
```

Comments This is a program command that is preceded by a line number.

ECHO

Description When Console Mode is enabled, this command controls whether the printer echoes the characters back to the communications port. If `ECHO ON` is entered, keystroke results return to the screen. If `ECHO OFF` is entered, keystroke results do not return to the screen.

Format `ECHO <ON/OFF>`

Parameters `<ON/OFF> = toggles the ECHO command on or off`

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

END

Description The `END` command terminates any program currently running. When the `END` command is received, the interpreter returns to Interactive Mode.

Format `END`

Example • This is an example of how to use the `END` command:

```
10 PRINT "THIS PROGRAM WILL TERMINATE"
20 PRINT "WHEN THE END COMMAND IS RECEIVED"
30 END
```

Comments This is a program command and is preceded by a line number.

! (EXCLAMATION MARK)

Description The exclamation mark is the marker for adding comments to the end of numbered programming lines. Any text following the ! is ignored when the line or command is processed.

Format `![comment text]`

Example • This is an example of how to use the `!` (comments) command:

```
10 LET A=10 ! Indicates number of labels to print
```

EXIT

Description This command is used to exit the `DO` and `FOR` loops.

Format EXIT <DO/FOR>

Comments This is a program command that is preceded by a line number.

FOR-LOOP

Description The FOR loop assigns the numeric variable to the value of the first expression. The NEXT line increments the numeric variable by the STEP amount.

If the numeric variable is less than the second expression after being incremented by the NEXT line, the program resumes running at the line following the FOR line.

When the STEP portion is omitted, the STEP value is 1. However, if the second expression is greater than the first expression, it defaults to -1.

Format

```
FOR <# variable> = <# expression> TO <# expression>
STEP <# expression>]
~~BODY~~
NEXT <# variable>
```

Parameters

<# variable> = indicates a numeric variable is used

<# expression> = indicates a numeric expression is used

Example • This is an example of how to use the FOR LOOP command:

```
10 FOR X=1 TO 10 STEP 1
20 PRINT X; ":ZBI IS FUN"
30 NEXT X
```

Comments FOR-LOOPs could be nested but cannot overlap. Variables cannot be reused by the nested loops. This is a program command that is preceded by a line number.

Functions

Description Functions built into this interpreter can be used in expressions only. The function names are not case sensitive.

If input parameters exist, they are enclosed in parentheses. If no parameters exist, no parentheses are used.

Variables referenced in the functions could be substituted by functions or expressions of the same type. If the function name ends with a $, it returns a string value. Otherwise, it returns a numeric value.

Specifying the wrong type of parameter returns either Syntax Error or Poorly formed expression error.

Integer Functions

The integer functions listed below return a numeric value:

DATE

This function returns the current date in YYYYDDD format, where YYYY is the year and DDD is the number of days since the beginning of the year. If the Real-Time Clock is not installed, 0 is returned.

Example • This example assumes the current date is January 1, 2003:

```
10 PRINT DATE
RUN
```

The result is:

```
2003001
```

DATAREADY(A)

This function returns the numeral 1 if data is ready on port A, and returns 0 if there is no data available.

Example • This is an example of how to check if there is a data on a port:

```
10 PRINT DATAREADY(0)
RUN
```

The result, assuming no data is waiting, is:

```
0
```

LEN(A$)

This function returns the length of the string A$.

Example • This example identifies the length of a string in characters. Hello World is 10 characters, as follows:

```
10 LET A$="Hello World"
20 PRINT LEN(A$)
RUN
```

The result is:

```
11
```

MAX(X,Y)

This function returns the maximum algebraic value of X or Y. If X is greater than Y, the value of X is returned. Otherwise, the value of Y is returned.

Example • This is an example of how to use the MAX(X,Y) command:

```
10 LET A=-2
20 LET B=1
30 PRINT MAX(A,B)
RUN
```

The result is:

```
1
```

MAXLEN(V$)

This function returns the maximum length for the string V$, which is always 255. This value is independent of V$ but remains for compatibility with the ANSI specification.

Example • This is an example of how to use the MAXLEN(V$) command:

```
10 LET A$="Hello"
20 PRINT MAXLEN(A$)
RUN
```

The result is:

```
255
```

MAXNUM

This function returns the largest number represented by this machine: 2,147,483,647.

Example • This is an example of how to use the MAXNUM command:

```
10 PRINT MAXNUM
RUN
```

The result is:

```
2147483647
```

MIN(X,Y)

This function returns the minimum algebraic value of X or Y. If X is less than Y, the value of X is returned. Otherwise, the value of Y is returned.

Example • This is an example of how to use the MIN(X,Y) command:

```
10 LET A=-2
20 LET B=0
30 PRINT MIN(A,B)
RUN
```

The result returned is:

```
-2
```

MOD(X,Y)

This function returns X Modulo Y (same as the remainder of X/Y).

Example • This is an example of how to use the MOD(X,Y) command:

```
10 LET A=9
20 LET B=2
30 LET C=-2
40 PRINT MOD(A,B)
50 PRINT MOD(C,A)
RUN
```

The result is:

```
1
-2
```

ORD(A$)

This function returns the ASCII value of the first character of string A$.

Example • This is an example of how to use the ORD(A$) command:

```
10 LET A$="ABC"
20 PRINT ORD(A$)
RUN
```

The result is:

```
65
POS(A$,B$)
```

This function returns the location of the first occurrence of B$ in A$. If there is no occurrence, the result is 0.

Example • This is an example of how to use the ORD(A$) command:

```
10 LET A$="ABCDD"
20 LET B$="D"
30 PRINT POS(A$,B$)
RUN
```

The result is:

```
4
```

POS(A$,B$,M)

This function returns the location of the first occurrence of B$ in A$ starting at the position of M. If there is no occurrence, the result is 0.

Example • This is an example of how to use the POS(A$,B$,M) command:

```
10 LET A$="Hello World"
20 LET B$="o"
30 PRINT POS(A$,B$,6)
RUN
```

The result is:

```
8
```

TIME

This function returns the time past midnight (2400h) in seconds. If the Real-Time Clock is not installed, 0 is returned.

Example • This is an example of how to use the TIME command:

```
10 PRINT TIME
RUN
```

The result, assuming the time is one minute past midnight, is:

```
60
```

VAL(A$)

This function returns the numeric value represented by A$. The conversion is done in the same fashion as the INPUT command.

Example • This is an example of how to use the VAL(A$) command:

```
10 LET A$="123"
20 LET C=VAL(A$)
30 PRINT C
RUN
```

The result is:

```
123
```

String Functions

The string functions listed below return a string value:

CHR$(M)

This function returns the character at position M of the extended ASCII table. Using values of M greater than 255, the value is modulated by 255. M could not be 0. The numeral 1 is substituted.

Example • This is an example of how to use the CHR$(M) command:

```
10 LET A=97
20 PRINT CHR$(A)
30 PRINT CHR$(353) !Note that 353 is
modulated to 97
RUN
```

The result is:

```
a
a
```

DATE$

This function returns the current date in string form YYYYMMDD. If the Real-Time Clock is not installed, no data returns.

Example • This is an example of how to use the DATE$ command:

```
10 PRINT DATE$
RUN
```

The result, assuming the date is January 1, 2003 is:

```
20030101
```

EXTRACT$

This function returns a string.

- **Source** can be a port or a variable string name.
- **Beginning string** is the first string or character that is encountered to trigger the command.
- **Ending string** is the last string or character encounter to stop the command.

Important • If the EXTRACT$ command encounters a carriage return line feed before encountering the beginning character or the ending character, it returns null.

The EXTRACT$ command is used to get data from between two known points. For example, data in a comma-delimited file always has commas between records.

Example • This example shows how to extract the word Technologies from this string: Zebra,Technologies,Corporation.

This is what the program looks like to accomplish this:

```
10 LET A$ = Zebra,Technologies,Corporation,
20 LET DATA$ = EXTRACT$(A$,",",",",")
```

Example • This example shows how the EXTRACT$ command works from an open port:

```
10 OPEN #1: NAME "SER"
20 LET DATA$ = EXTRACT$(1,",",",",")
```

Notice how the quotes are used to show a literal character, in this case a comma.

Example • This example shows how the start and stop points are variable; a variable name is used instead of the literal:

```
10 LET B$ = ","
20 LET A$ = Zebra,Technologies,Corporation
30 LET DATA$ = EXTRACT$(A$,B$,B$)
```

LTRIM$(A$)

This function returns the string A$ with all leading spaces removed.

Example • This is an example of how to use the LTRIM$(A$) command:

```
10 LET A$=" Hello"
20 PRINT LTRIM$(A$)
RUN
```

The result is:

```
Hello
```

REPEAT$(A$,M)

This function returns a string containing M copies of A$.

Example • This is an example of how to use the REPEAT$(A$,M) command:

```
10 LET X=3
20 LET A$="Hello"
30 PRINT REPEAT$(A$,X)
RUN
The result is:
HelloHelloHello
```

RTRIM$(A$)

This function returns a string created from A$ with trailing spaces removed.

Example • This is an example of how to use the RTRIM$(A$) command:

```
10 LET A$="Hello "
20 LET B$="World"
30 PRINT RTRIM$(A$);
40 PRINT B$
RUN
```

The result is:

```
HelloWorld
```

STR$(X)

This function converts numeric type X to a string.

Example • This is an example of how to use the STR$(X) command:

```
10 LET A=53
20 PRINT STR$(A)
RUN
```

The result is:

```
53
```

TIME$

This function returns the time of day in format HH:MM:SS (hours:minutes:seconds). If the Real-Time Clock is not installed, no data is returned.

Example • This is an example of how to use the TIME$command:

```
10 PRINT TIME$
RUN
```

The result, assuming it is 10 a.m., is:

```
10:00:00
```

UCASE$(A$)

This function returns a string created from A$ with all characters in uppercase. Noncharacter values are not changed.

Example • This is an example of how to use the UCASE$(A$) command:

```
10 LET A$="Zebra Technologies"
20 PRINT UCASE$(A$)
RUN
```

The result is:

```
ZEBRA TECHNOLOGIES
```

GOTO

Description The GOTO statement is used to direct the interpreter to a specific line number. GOTO is followed by a line number that the program attempts to process next. Upon executing the GOTO statement, the interpreter continues running at the line number specified following GOTO. If the line number referenced does not exist, an error message displays — Error: Line does not exist.

Format GOTO

Example • This is an example of how to use the GOTO command:

```
10 PRINT "Zebra Printers"
20 GOTO 10
```

Comments The result displays Zebra Printers until the program is halted. This is a program command and must be preceded by a line number.

GOSUB-RETURN

Description GOSUB is followed by a line number that the program attempts to process next. Upon executing the GOSUB statement, the interpreter continues running at the line number specified following GOSUB. If the line number referenced does not exist, an error message displays — Error: Line does not exist.

Before executing the next line, the GOSUB command stores the line number of the GOSUB line. When the RETURN statement is called, the program moves back to the next line following the GOSUB.

Executing a RETURN statement without a corresponding GOSUB statement causes an error message to display — Error: Invalid RETURN statement.

GOSUB statements can be nested.

Format

```
GOSUB
RETURN
```

Example • This is an example of how to use the GOSUB-RETURN command:

```
10 PRINT "Call Subroutine"
20 GOSUB 1000
30 PRINT "Returned from Subroutine"
40 END
1000 PRINT "In Subroutine"
1010 RETURN
```

Comments These are program commands and must be preceded by line numbers.

IF Statements

Description If the value of the <Boolean expression> in an IF statement is true and a program line follows the keyword THEN, this program line is executed. If the value of the Boolean expression is false and a program line follows the keyword ELSE, this program line is executed. If ELSE is not present, then execution continues in sequence, with the line following the END IF statement.

Nesting of blocks is permitted, subject to the same nesting constraints as DO-LOOPs (no overlapping blocks).

ELSE IF statements are treated as an ELSE line followed by an IF line, with the exception that the IF shares the END IF line of the original IF statement.

Format

```
IF <Boolean expression> THEN
~~BODY~~
[ELSE IF <Boolean expression> THEN
~~BODY~~]*
[ELSE
~~BODY~~]
END IF
```

Example • This is an example of how to use the IF statement command:

```
10 IF A$="0" THEN
20 PRINT "ZBI IS FUN"
30 ELSE IF A$="1" THEN
40 PRINT "ZBI IS EASY"
50 ELSE
60 PRINT "X=0"
70 END IF
```

Comment This is a program command that is preceded by a line number.

INBYTE

Description This command forces the interpreter to pause until data is available. Use the DATAREADY function to determine if there is data on the port.

Format INBYTE [channel expression:] <A>

Parameter <A> = numeric or string expression. The received value replaces the current value held in the variable.

Example • This is an example of how to use the INBYTE statement command:

```
10 INBYTE A$ !takes one byte (char) from port #1
20 PRINT A$ !prints the character to the console
```

Comments In this example, the interpreter pauses until the data is entered and then continues processing. This command enters all bytes in a string or integer, including control codes.

This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

INPUT

Description If the variable is numeric and the value entered cannot be converted to a number, it writes as 0. This operation scans the data from left to right, shifting any number into the variable. It ignores any other character except the return character, which terminates the input, or Ctrl-C (^C) which terminates the program. The variable can be in string or numeric form.

Format

```
INPUT [<channel expression>:] <variable>
[,variable]*
```

If the [<channel expression>:] is omitted, the default port is 0. If an invalid port is specified, an error condition returns in the form of Error: Invalid port.

Example • This is an example of how to use the INPUT command:

```
10 INPUT A
20 PRINT A
```

If you entered 1234567891011, the number sent to the display is modulated by the MAXNUM value.

Example • This is an example of how to use the INPUT command:

```
10 OPEN #1: NAME "ZPL"
20 PRINT #1: "~HS"
30 FOR I = 1 TO 3
40 INPUT #1: A$
50 PRINT A$
60 NEXT I
```

In Example 2, a host status prints to the console after submitting the host status request ~HS to the ZPL port.

The Input/Output command of the ZBI interpreter is limited to the communications ports. File I/O is not supported.

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

LET

Description The LET command is used to assign value to a specific variable. The expression is evaluated and assigned to each variable in the variable list.

Format LET <variable> [,<variable>]* = <expression>

The variable types must match the expression type or an error message displays — Error: Variable types must be the same.

When a value is assigned to a string variable with a substring qualifier, it replaces the value of the substring qualifier. The length of the value of the string variable might change as a result of this replacement.

Example • This is an example of how to use the LET command:

```
10 LET A$= "1234"
15 LET A$(2:3)= "55" ! A$ NOW = 1554
20 LET A$(2:3)= "" ! A$ NOW = 14

10 LET A$= "1234"
15 LET A$(2:3)= A$(1:2) ! A$ NOW = 1124

10 LET A$= "1234"
20 LET A$(2:1)= "5" ! A$ NOW = 15234
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

LIST

Description This command numerically lists the program lines currently in memory.

Format `LIST [RANGE]`

Parameters

`[RANGE]` = an optional request for specific lines (or a line) of code. `[RANGE]` is a single pair of numbers separated by a hyphen.

Example • This is an example of how to use the `LIST` command:

```
LIST 20
LIST 90-135
```

The printer returns line 20, or in the second example, lines 90 through 135.

Comments This is an interactive command that takes effect as soon as it is received by the printer.

LOAD

Description This command transfers a program file previously stored in the printer's memory and opens it in the ZBI Program Memory.

If the program file does not exist, the ZBI Program Memory is cleared and no program is opened. An error message displays — Error: Invalid file name.

Format `LOAD <"filename">`

Parameter `<"filename">` = the name of the file to be loaded into memory. Drive location and file name must be in quotation marks.

Example • This is an example of how to use the `LOAD` command:

```
LOAD "PROGRAM1.BAS"
LOAD "E:PROGRAM1.BAS"
```

Comments This is an interactive command that takes effect as soon as it is received by the printer.

NEW

Description This command clears the interpreter's memory, including the line buffer and variables, but not any open ports.

Format `NEW`

Example • This is an example of how to use the `NEW` command:

```
NEW
```

Comments This is an interactive command that takes effect as soon as it is received by the printer.

NOT

Description The `NOT` statement is a Boolean operator. If the Boolean expression is true, the result is false. If the Boolean expression is false, the result is true.

Format `NOT <Boolean expression>`

Example • This is an example of how to use the `NOT` command:

```
10 LET A=1
20 IF NOT A=0 THEN
30 PRINT A
40 END IF
RUN
```

Comments Use this Boolean operator in conjunction with Boolean expressions.

Numeric Expressions

Description Base numerical expression can be either a constant, variable, or another numerical expression closed in by parentheses. Overflow cannot be detected. The result is undefined. The five types used (addition, subtraction, multiplication, division, and exponentiation) are listed below.

1. + (addition) Addition expressions use this format:
   `<numerical expression>+<numerical expression>`
2. – (subtraction) Subtraction expressions use this format:
   `<numerical expression>-<numerical expression>`
3. * (multiplication) Multiplication expressions use this format:
   `<numerical expression>*<numerical expression>`

4. / (division) Division expressions use this format:
   `<numerical expression>/<numerical expression>`
5. ^ (exponentiation) Exponentiation expressions use this format:
   `<numerical expression>^<numerical expression>`

In mathematics, order of precedence describes in what sequence items in an expression are processed. All expressions have a predefined order of precedence.

The order of precedence is ^, * /, + –.

* and / have the same precedence, and + and - have the same precedence. Items with the same order of precedence are processed from left to right.

For example, this expression 5+(8+2)/5 is processed as 8+2=10, followed by 10/5=2, then 5+2 to give a result of 7.

Functions and parentheses always have the highest order of precedence, meaning that they are processed first. The remaining items are specific to the type of expression (Boolean, numeric, and string).

Maximum value: 2,147,483,647

Minimum value: –2,147,483,648

Comments

- No floating point is supported.
- Variable names must start with a letter and can include any sequence of letters, digits, and underscore.
- Function and command names might not be used as variable names.
- Variable names are not case sensitive and are converted to uppercase by the interpreter.
- When using division, the number is always rounded down. For example, 5/2=2.

ON ERROR

Description The ON ERROR command can be used to prevent a program from halting in the event of an error. If an error occurs in a previous line during program execution, the ON ERROR statement calls the GOTO or GOSUB statement and allows the program to continue.

Format ON ERROR <GOTO/GOSUB> LN

If there is no error, this line is ignored.

Example • This is an example of how to use the ON ERROR command:

```
30 LET A = B/C
40 ON ERROR GOTO 100
...
100 PRINT "DIVIDE BY ZERO OCCURRED"
110 LET A = 0
120 GOTO 50
...
```

Comments This is a program command that is preceded by a line number.

Open

Description This command is used to open a port for transmitting and receiving data.

Format

```
OPEN #<channel expression>: NAME <string expression> [,
ACCESS <ACCESS type>]
```

Parameters

<channel expression> =

Accepted Values: 0 to 9

Default Value: a port must be specified

<string expression> = port name to open (SER, PAR, or ZPL)

<ACCESS type> =

INPUT for receiving only,

OUTPUT for transmitting only, and

OUTIN for transmitting and receiving.

A channel must be specified; the default value cannot be used. Access type OUTIN is used if access type is not specified.

Example • This is an example of how to use the OPEN command:

```
10 OPEN #1: NAME"ZPL"
```

If there are conflicts opening the port, an error message displays – Error: Unable to open port. If the port is already open, an error message displays — Error: Port already opened.

The port being opened no longer allows data to pass directly into its buffer, it disconnects, and the interpreter now controls the data flow.

Data already in the buffer stays in the buffer.

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

OR

Description The OR statement is a Boolean operator. If either of the expressions is true, the result is true; otherwise, the result is false.

Format

```
<Boolean expression> OR <Boolean expression>
```

Comments Use this Boolean operator in conjunction with Boolean expressions.

OUTBYTE

Description This command outputs all bytes in a string, including control codes.

Format `OUTBYTE [<channel expression>:] <A>`

Parameters `<A>` = a numeric or string expression.

If the parameter is a numeric expression, it must be a value of 0 through 255. If not, it is truncated. For a string, the first character is used. In case of a NULL string, 0 is sent.



Example • This is an example of how to use the OUTBYTE command:

```
Let A$="Hello"
OUTBYTE A$
```

The result is:

```
H
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

Ports <CHANNEL EXPRESSION>

Description Each printer has a list of ports that might be opened by the interpreter.

Each port is given a three-letter abbreviation that is used with the OPEN command to open that port. The standard port names are SER for the serial port and PAR for the parallel port. The port reference ZPL opens a channel to the printer's formatting engine that functions as a port not controlled by the ZBI interpreter.

PRINT

Description This command sends data to the printer to be printed.

Format PRINT [channel expression:] <expression> [,or; <expression>]* [;]

The expression can be either a string or a numeric expression.

Using a , to separate expressions adds a space between them.

Using a ; to separate expressions does not put a space between them.

Using a ; at the end of a line ends the print statement without a new line.

Example • This is an example of how to use the PRINT command:

```
10 LET A$= "This is an example"
20 LET B$= "of the PRINT Command."
30 PRINT A$, B$ ! adds a space between expressions
40 PRINT A$; B$ ! no space added
RUN
```

The result is:

```
This is an example of the PRINT Command.
This is an exampleof the PRINT Command.
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

REM

Description A numbered **remark** line is starts with REM and can include any text in any form after it. This line is ignored by the interpreter.

Format REM [comment]

Example • This is an example of how to use the REM command:

```
10 REM COMMAND LINES 20-100 PRINT A LABEL
```

Comments Remarks are used for program description and can be included as a separate program line or appended to the end of a program line. Also used for internal comments is the exclamation mark (!) statement.

RENUM

Description This command renumbers the lines of the program being edited. RENUM can reorganize code when line numbers become over- or under-spaced. The line references following GOTO and GOSUB statements are renumbered if they are constant numeric values. Renumbering does not occur if the line numbers are outside of the range limits of 1 to 10000.

Format RENUM <A>,<B>

Parameters

<A> = the number to start the renumbering sequence

<B> = the automatic increment between the new line numbers

Example • This is an example of how to use the RENUM command:

```
13 LET A=6
15 LET B=10
17 GOTO 13

>RENUM 10,5

0 LET A=6
15 LET B=10
20 GOTO 10
```

Comments This is an interactive command that takes effect as soon as it is received by the printer.

RESTART

Description If a program was halted by Ctrl-C or a BREAK command, the RESTART command can be used to reactivate the program from the point it stopped. RESTART functions similarly to RUN, except the program attempts to restart from the point it was last terminated. It also works in conjunction with the STEP command, picking up where STEP ends.

Format RESTART

Comments If the program has not been run or is finished, RESTART runs the program from the beginning. This is an interactive command that takes effect as soon as it is received by the printer.

RUN

Description This command activates the program being edited, starting with the lowest line number. If a line does not specify a new line to go to, the next line in numeric order is processed. When a higher line number does not exist, the RUN command stops.

Format RUN



Example • This is an example of how to use the RUN command:

```
10 PRINT "ZBI"
15 PRINT "Programming"
RUN
```

The result is:

```
ZBI
Programming
```

Comments Ports that are open when the application is activated remains open after the application has terminated. Variables also remain after the application has terminated.

This is an interactive command that takes effect as soon as it is received by the printer.

SEARCHTO$ (A,B$)

Description This function performs a search up to a string, which is defined by B$ on port A. The string the search yields is displayed.

Format SEARCHTO$(A,B$)

Parameters

A = port number (0 to 9) to which requested data is sent.

B$ = string variable or string array. If B$ is an array, this command searches for all non-null strings in the B$ array.



Example • This is an example of how to use the SEARCHTO$ (A,B$) command:

```
10 LET A$=SEARCHTO$(0,"Hello")
```

There are several ways to display that Hello World is the text string being searched in this example. A$ then equals Hello, the program performs the search, and only World remains on the port.

SEARCHTO$ (A,B$,C)

Description This function works similarly to SEARCHTO$(A,B$) and performs a search up to a string, which is defined by B$ on port A. The string the search yields is displayed. Unused characters up to the search string are sent to port C.

Format SEARCHTO$(A,B$,C)

Parameters

A = port number (0 to 9) the requested string is sent to.

B$ = string variable or string array. If B$ is an array, this command searches for all non-null strings in the B$ array.

C = port number (0 to 9) unused characters are sent to.



Example • This is an example of how to use the SEARCHTO$ (A,B$,C) command:

```
LET B$ = "Hello"
10 LET A$ = SEARCHTO$(0,"Hello",1)
```

There are several ways to display that Hello World is the text string being searched for this example. A$ is then equal Hello, the program performs the search, and only World remains on the port. The unused characters are sent to port 1.

Example • This example declares an array of variables, functions the same way only the list of possible variables is expanded:

```
DECLARE STRING B$(4)
LET B$(1) = "Hello"
LET B$(2) = "HELLO"
LET B$(3) = "hello"
LET B$(4) = "Hi"
10 LET A$ = SEARCHTO$(0,B$,0)
```

SETERR

Description This command sends a message to the printer to set the error flag. A logical interpreter flag is triggered in the printer. This error is referenced as a BASIC Forced Error.

Format `SETERR`

Comments This is a program command and must be preceded by a line number.

SLEEP

Description This command specifies the time that the interpreter pauses for label generation to receive greater priority. This command could be sent to the printer after sending a label format to be printed. The interpreter pauses in its processing for the amount of time specified.

Format `SLEEP <A>`

Parameters `<A>` = the time in seconds (0 to 500) the interpreter pauses.

Example • This is an example of how to use the `SLEEP` command:

```
10 SLEEP 450
```

Comments This is a program command and must be preceded by a line number.

STEP

Description If a program was stopped by a BREAK command, STEP attempts to execute the program one line from where it last ended. If the program has not been run or has been completed, this executes the lowest numbered line.

Format STEP

Example • This is an example of how to use the STEP command:

```
10 PRINT "Hello World"
20 BREAK
30 PRINT "30"
```

Entering STEP causes line 10 to be executed.

Entering RUN followed by STEP causes:

RUN — Execute to line 20 and stop.

STEP — Execute line 30 and stop.

Comments This is an interactive command that takes effect as soon as it is received by the printer.

STORE

Description This command saves the program currently in memory as the specified file name. This format is used:

```
STORE <"filename">
```

Parameters <"filename"> = the name of the file to be stored. Drive location and file name must be in quotation marks.

Example • This is an example of how to use the STORE command:

```
STORE "E:ZEBRA1.BAS"
```

Comments For a file name to be valid, it must conform to the 8.3 Rule: each file must have no more than eight characters in the file name and have a three-character extension. Here the extension is always .BAS (for example, MAXIMUM8.BAS).

This is an interactive command that takes effect as soon as it is received by the printer.

STRING CONCATENATION (&)

Description The basic string expression could be either a constant or a variable, and the concatenation (&) is supported.

Using the concatenation operator adds the second string to the first string.

Format `<string expression> & <string expression>`

Example • This is an example of how to use the `STRING CONCATENATION (&)` command:

```
10 LET A$= "ZBI"
20 LET B$= "Programming"
30 LET C$= A$ & B$
40 PRINT C$
```

The result is:

```
ZBI Programming
```

Comments If the concatenation causes the string to be greater than the maximum possible length of a string (255 characters), the first string is returned and an error message displays — Error: String size limit exceeded.

STRING VARIABLE

Description Maximum length: 255 characters

Variable names must start with a letter and can include any sequence of letters, digits, and underscore. The variable ends with a `$`.

Function and command names might not be used as a variable name.

Variable names are not case sensitive and are converted to uppercase by the interpreter.

UB-STRINGS

Description Using a substring operator on a string allows a specific portion of the string to be accessed. To determine the coordinates of the string portion to be used, count the spacing from the beginning to the end of the string, including spaces.

Format `StringVariable$(A:B)`

Parameters

`A` = the position of the first character in the desired string.

`B` = the position of the last character in the desired string.



Example • This is an example of how to use the `UB-STRINGS` command:

```
10 LET A$="Zebra Quality Printers"
20 LET B$=A$(1:13)
30 PRINT B$
```

The result is:

```
Zebra Quality
```

To calculate the position of Zebra Quality Printers in line 10 above, `Z` occupies position 1, `e` occupies position 2, `b` occupies position 3, `r` occupies 4, `a` occupies 5, the space occupies 6, and so on.

Comments If the `A` parameter is less than 1, it is automatically assigned a value of 1. Because the string is calculated starting with 1, the A parameter cannot be less than 1.

If `B` is greater than the length of the string, it is replaced with the length of the string. In this example, the `B` parameter can be no greater than 22.

If `A` is greater than `B`, a NULL string (`" "`), which points to the location of the smaller of `A` or the end of the string, is returned. This is used when adding a string in the middle of another string without removing a character. Refer to the `LET` command.

TRACE

Description This command is only valid when the DEBUG function is active.

Format TRACE <ON/OFF>

Parameters

<ON/OFF> = controls whether TRACE is active (on) or disabled (off).

If DEBUG is activated and the TRACE command is on, trace details are displayed. When any variables are changed, the new value displays as follows:

Variable$ = New Value

Every line processed has its line number printed.

If DEBUG is on, the line number prints before the command line is executed and variables print as their values are assigned.

```
10 LET A=5
20 GOTO 40
30 PRINT "Error"
40 PRINT A
RUN
```

The output is:

```
<TRACE> 10
<TRACE> A=5
<TRACE> 20
<TRACE> 40
5
```

Comments This can be an interactive command that takes effect as soon as it is received by the printer, or a program command that is preceded by a line number.

4

ZBI Tutorial

This tutorial provides you with examples of how Zebra BASIC Interpreter (ZBI) and Zebra Programming Language II (ZPL II) interact in bar code printing.

It is recommended that you run several examples to gain an understanding of how the interpreter works in conjunction with your printer.

ZBI allows you to maximize bar code printing options through custom programs written for a specific need.

To its advantage, ZBI bears a strong resemblance to many other BASIC interpreters and does not require you to learn an entirely new language from the ground up to take advantage of its features.

Starting ZBI

ZBI is enabled by sending a ZPL II command to the printer via one of the communication ports (serial, parallel, or Ethernet) in an interactive mode. This communication port is referred to as the console. Programs like HyperTerminal for Windows can be used to communicate with your printer.

These ZPL II commands are available to start the ZBI Interpreter: ~JI and ^JI. When the printer is turned on, it accepts ZPL II commands and label formats. However, to receive ZBI commands the printer must be initialized using ~JI or ^JI.

With a console application open, send one of these two commands to the printer:

1. **~JI** — when this command is received, the printer responds by sending a ZBI header with the program version and a new line containing an input prompt (>) back to the screen. This is an indication that the ZBI program lines or commands can be sent to the printer. While receiving programming commands, the printer echos the received characters back to the source. This echoing can be turned off with the ECHO command.
2. **^JId:o.x,b,c,d** — sending this command to the printer activates a ZBI session. The parameters have these functions:
   d = location of program to run after initialization
   Accepted Values: R:, E: and B:
   Default Value: location must be specified
   o = name of program to run after initialization
   Accepted Values: Accepted Values: any valid program name
   Default Value: Default Value: a name must be specified
   x = extension of program to run after initialization
   Fixed Value: .BAS
   b = console control
   Accepted Values: Y (console on) or N (console off)
   Default Value: Y
   c = echoing control
   Accepted Values: Y (echo on) or N (echo off)
   Default Value: Y
   d = memory allocation for ZBI
   Accepted Values: 20K to 1024K
   Default Value: 50K

If a second ~JI or ^JI command is received while the interpreter is running, the command is ignored.

ZBI Interpreter Modes

The ZBI Interpreter operates in these modes:

- The interactive mode allows commands to be entered and processed immediately. Command lines entered without line numbers are interactive and processed as soon as they are received by the printer.

Example • This is an example of how to use the NEW command:

```
NEW
```

- The program mode stores a series of command lines in memory and processes them in numerical order when a RUN command is entered. Line numbers must be greater than 0 and less than 10,000.

Example • This is an example of

```
10 LET A=10
20 LET B=3
```

A program is activated from the first numbered line by entering the RUN command. While the program is running, it can be halted with the Control-C (^C) command, sent to the printer from the console. The interrupted program continues where it left off by entering the RESTART command.

Ending A ZBI Session

An active ZBI session can be halted in one of these ways:

- Sending ZPL at the input prompt (>).
- Sending ~JQ at the input prompt (>).

Encrypting ZBI Files

The .baz extension provides security for all ZBI executables. The .baz extension is created when the ZBI store command is used.

^HZO = uploads .baz files to the printer.

Important • For security purposes, users cannot:

- open .baz files
- upload or download .baz files to .bas file format

Example • This is an example on how to generate a .baz file.

1. Create basic program as in the single line program below.
2. Type `10 print "ZPL".`
3. Save the file with the filename: `Store E:ZEBRA1.BAZ`
4. Recall the newly created file from the printer using the ^HZO command, as follows: `^XA^HZO,E:ZEBRA1.BAZ^XZ`
5. To use the new file, download the data received that follows the ^HZO command to any printer.

Limitations These commands do not work when loading a ZBI executable:

- `List`
- `Trace`
- `Debug`

Basic ZBI Examples

This section provides you with basic examples of ZBI:



Example • This example shows you the choices you have: print a configuration label, print text, or calibrate the printer. The choice you make corresponds with the ZPL that is sent to the printer:

```
  5 REM PORTS SHOULD BE CLOSED, BY ROUTINE, BEFORE
    OPENING
 10 CLOSE # 1
 20 CLOSE # 2
 25 REM OPENING PORT
 40 OPEN # 1 : NAME "ZPL"
 45 REM THE FOLLOWING WILL BE DISPLAYED ON THE TERMINAL
 50 PRINT "YOU HAVE THREE CHOICES"
 51 PRINT "PRESS 1 TO PRINT CONFIG LABEL" !prints to the
    console
 52 PRINT "PRESS 2 TO PRINT TEXT "
 53 PRINT "PRESS 3 TO CALIBRATE"
 54 PRINT "OR PRESS 7 TO EXIT"
 58 REM THE FOLLOWING ARE THE POSSIBLE INPUTS
 60 INPUT A !PRINTER IS EXPECTING AN INPUT
 61 IF A = 1 THEN
 62 GOTO 120
 63 ELSE
 65 IF A = 2 THEN
 66 GOTO 130
 67 ELSE
 70 IF A = 3 THEN
 71 GOTO 140
 72 ELSE
 75 IF A = 7 THEN
 76 GOTO 300
 80 END IF
 81 GOTO 51
 85 REM IF THERE IS AN IF STATEMENT ALWAYS HAVE AN ENDIF
 95 REM THE FOLLOWING WILL PRINT A CONFIGURATION LABEL
120 PRINT # 1 : "~WC" !~WC TELLS THE PRINTER TO PRINT
    CONFIGURATION LABEL
125 GOTO 50
```

```
130 PRINT "THIS TASK WILL SEND SAMPLE TEXT TO THE
    PRINTER"
131 PRINT "INPUT YOUR SAMPLE TEXT"
132 INPUT Y$
133 REM THE FOLLOWING STATEMENT WILL PRINT THE Y$ STRING
    VALUE
135 PRINT # 1 : "^XA^FO20,20^A0N50,50^FD" ; Y$ ; "^XZ"
136 GOTO 50
139 REM THE FOLLOWING COMMAND WILL TELL THE PRINTER TO
    CALIBRATE
140 PRINT # 1 : "~JG" !~JG ZPL COMMAND BEING SENT TO THE
    PRINTER
141 GOTO 50
300 PRINT "BEFORE CLOSING THE PRINTER WILL NOW PAUSE FOR
    10 SECONDS"
301 SLEEP 10 ! THE SLEEP COMMAND PAUSES THE PRINTER
310 END
```

Example • This example shows ZBI code using extract and search to commands:

```
 10 REM ALWAYS CLOSE PORTS BEFORE OPENING (ROUTINE)
 20 CLOSE # 1
 30 CLOSE # 2
 60 OPEN # 1 : NAME "ZPL"
 69 REM SPECIFYING VALUE FOR STRING BELOW
 70 LET B$ = "START"
 79 REM USING SEARCHTO$ COMMAND TO SEARCH STREAMING DATA
    FOR B$ OR THE WORD "START"
 80 LET Z$ = SEARCHTO$ ( 0 , B$ )
 89 REM SPECIFYING VALUES FOR THE STRINGS F$ AND G$ ("T"
    AND "W ")
 90 LET F$ = "T "
100 LET G$ = " W"
108 REM USING THE EXTRACT$ COMMAND TO SEARCH STREAMING
    DATA FOR F$ AND G$
109 REM X$ WILL BE THE DATA BETWEEN F$ AND G$
110 LET X$ = EXTRACT$ ( 0 , F$ , G$ )
250 PRINT # 1 : "^XA^F025,25^A050,50^FD" & X$ &"^FS^XZ"
1000 END
```

Example • This example shows ZBI code using ARRAYS:

```
  5 REM PORTS SHOULD BE CLOSED, BY ROUTINE, PRIOR TO
    OPENING
 10 CLOSE #1
 20 CLOSE #2
 40 OPEN #1: NAME "ZPL"
 45 REM WHEN USING ARRAYS, THE ARRAY HAS TO BE DECLARED
    BEFORE USING
 49 REM THIS ARRAY "NAME$" IS BEING DEFINED WITH 5 SPOTS
    FOR DATA
 50 DECLARE STRING NAME$ (5)
 57 REM THE FOLLOWING ALLOWS ONE TO INPUT 5 DIFFERENT
    VALUES AND THE VALUES
 58 REM ARE PLACED INTO THE ARRAY IN ORDER (1-5)
 59 PRINT "INPUT YOUR FIRST 5 NAMES"
 60 FOR INDEX = 1 TO 5 STEP 1
```

```
 70  INPUT NAME$(INDEX)
 80  NEXT INDEX
 89  REM THE FOLLOWING DECLARES AN ARRAY (NUMBERS$) AND
     ALLOWS 5 SPOTS FOR DATA
 90  DECLARE STRING NUMBERS$ (5)
 99  PRINT "INPUT YOUR FIRST 5 NUMBERS"
100  FOR INDEX = 1 TO 5 STEP 1
110  INPUT NUMBERS$(INDEX)
120  NEXT INDEX
128  REM THE FOLLOWING PRINTS OUT THE VALUES THAT HAVE
     BEEN STORED IN THE ARRAYS
130  PRINT #1 :
     "^XA^FO60,60^A0N50,50^FD"&NAME$(1)&"^FS";
140  PRINT #1 :
     "^FO60,120^A0N50,50^FD"&NUMBERS$(1)&"^FS^XZ"
150  PRINT #1 :
     "^XA^FO60,60^A0N50,50^FD"&NAME$(2)&"^FS";
160  PRINT #1 :
     "^FO60,120^A0N50,50^FD"&NUMBERS$(2)&"^FS^XZ"
170  PRINT #1 :
     "^XA^FO60,60^A0N50,50^FD"&NAME$(3)&"^FS";
180  PRINT #1 :
     "^FO60,120^A0N50,50^FD"&NUMBERS$(3)&"^FS^XZ"
190  PRINT #1 :
     "^XA^FO60,60^A0N50,50^FD"&NAME$(4)&"^FS";
200  PRINT #1 :
     "^FO60,120^A0N50,50^FD"&NUMBERS$(4)&"^FS^XZ"
210  PRINT #1 :
     "^XA^FO60,60^A0N50,50^FD"&NAME$(5)&"^FS";
220  PRINT #1 :
     "^FO60,120^A0N50,50^FD"&NUMBERS$(5)&"^FS^XZ"
300  END
```

Example • This ZBI code shows integer functions:

```
10 REM THIS PROGRAM WILL SHOW VARIOUS INTEGER FUNCTIONS
11 CLOSE # 1
12 CLOSE # 2
20 PRINT "ENTER YOUR LAST NAME"
25 INPUT A$ ! PRINTER IS EXPECTING A STRING INPUT
30 LET X = LEN ( A$ ) !THIS IS WHERE THE LENGTH IS
   DETERMINED
35 PRINT "YOUR LAST NAME (" ; A$ ; ") CONTAINS " ; X ;
   " CHARACTERS"
36 PRINT
40 PRINT "ENTER 2 DIFFERENT NUMBERS EACH FOLLOWED BY
   THE ENTER KEY"
45 INPUT A
46 INPUT B
55 LET Y = MIN ( A , B ) !THIS IS WHERE THE SMALLEST
   NUMBER IS DETERMINED
60 PRINT "THE SMALLEST NUMBER THAT YOU ENTERED WAS " ;
   Y
61 PRINT
65 PRINT "ENTER ANY ASCII CHARACTER FROM YOUR KEYBOARD"
66 INPUT B$
70 LET U = ORD ( B$ ) ! THIS IS WHERE THE ASCII
   EQUIVELANT IS DETERMINED
71 LET R = 34
75 PRINT "THE ASCII EQUIVELANT OF " , CHR$ ( R ) , B$ ,
   CHR$ ( 290 ) , "IS" , U !THIS LINE SHOWS TWO
   DIFFERENT WAYS TO PRINT THE " CHARACTER USING THE
   CHR$ FUNCTION
76 REM IN LINE 75 I AM USING "," UNLIKE THE ";" THAT
   WAS USED IN LINE 35
78 REM WHEN SEPERATING WITH THE "," CHARACTER A SPACE
   WILL BE INCLUDED
79 REM IF USING THE ";" NO SPACE WILL BE INCLUDED AS IN
   LINE 34
80 END
```

Example • This example shows ZBI code showing Numeric expressions:

```
10  REM This program will demonstrate the use of Numeric
    expressions
11  CLOSE # 1
12  CLOSE # 2
20  REM SIMPLE PROGRAM SHOWING HOW TO USE NUMERIC
    EXPRESSIONS
28  PRINT
30  PRINT "The first example will be addition,
    subtraction, mulitiplication , division and
    exponentiation"
31  PRINT " Input your first number"
32  INPUT A
33  PRINT " Input your second number"
35  INPUT B
38  LET X = A + B !simple math (addition)
39  LET Y = A - B !simple math (subtraction)
40  PRINT "If you add" , A , "plus" , B , "you get" , X
43  PRINT "If you subtract" , A , "minus" , B , "you
    get" , Y
50  LET Z = A * B !simple math (multiplication)
70  PRINT
71  PRINT "IF YOU MULTIPLY" , A , "AND" , B , "YOUR
    RESULT IS" , Z
75  LET J = A / B !simple math (division)
76  ON ERROR GOTO 100 !This statement allows the program
    to continue
78  PRINT "IF YOU DIVIDE" , A , "BY" , B , "YOU GET" , J
79  IF J = 0 THEN
80  GOSUB 400
82  GOTO 200
100 PRINT "THERE HAS BEEN A DIVIDE BY ZERO ERROR"
121 PRINT "THE SECOND NUMBER YOU ENTERED WAS A ZERO"
122 PRINT "THE ERROR WAS CAUSED BY TRYING TO DIVIDE" , A
    , "BY" , B
```

```
200 PRINT
201 PRINT " EXPONENTIATION USES THE ^ SYMBOL, X^2 WOULD
    BE X SQUARED"
202 LET G = A ^ B !exponentiation
205 PRINT A , "^" , B , "RESULTS IN" , G
300 END
400 PRINT
401 PRINT "YOUR RESULT WHEN YOU DIVIDED" , A , "BY" , B
    , "WAS ZERO"
405 PRINT "REMEMBER WHEN USING DIVISION THE NUMBER WILL
    ALWAYS BE ROUNDED DOWN"
407 RETURN
```

Example • ZBI example using E-mail server:

```
10  CLOSE #1
20  CLOSE #2
30  OPEN #1: NAME "EML" ! PORT SET FOR EML FOR EMAIL
    SERVER
31  OPEN #2: NAME "ZPL"
32  PRINT "ENTER YOUR FIRST NAME"
35  INPUT A$
36  PRINT "ENTER YOUR LAST NAME"
37  INPUT B$
40  PRINT "ENTER YOUR EMPLOYEE NUMBER"
42  INPUT C$
45  PRINT "Hello ";A$; " We are now ready to proceed"
400 PRINT #1 : "JDOE@ZEBRA.COM";CHR$(4); !SENDING
    MESSAGE TO JDOE
405 PRINT #1 : "FROM: PRINTER #1" ! MAIL FROM: DATA
    ("PRINTER #1")
410 PRINT #1 : "TO: MAIN LOG SERVER (EMAIL)" ! MAIL TO:
    DATA ("MAIN LOG SERVER")
500 PRINT #1 : "SUBJECT: EMPLOYEE "; B$ ;" ( ";C$;" )
    LOGGED IN"! SUBJECT : DATA
505 PRINT #1 : ""
510 PRINT #1 : "EMPLOYEE "; B$; " LOGGED IN" ! MAIL
    MESSAGE
550 PRINT #1 : CHR$(4) ! IMPORTANT TO END WITH EOT
600 CLOSE #1
700 CLOSE #2
```

In this program an e-mail message will be sent to JDOE@ZEBRA.COM <mailto:JDOE@ZEBRA.COM> that indicates that an employee has logged in

Example • ZBI using TCP port:

```
10  CLOSE #1
20  CLOSE #2
30  OPEN #1: NAME "TCP"
31  OPEN #2: NAME "ZPL"
32  PRINT "ENTER THE PRODUCT NAME"
35  INPUT A$
36  PRINT "ENTER QUANTITY"
37  INPUT B
40  PRINT "ENTER THE LOCATION AISLE/SHELF # EXAMPLE 3/4
    "
42  INPUT C$
45  PRINT #2 : "^XA^FO100,100^A0N,50,50^FD";A$;"^FS";
    !SENDING ZPL LOCALLY
46  PRINT #2 : "^FO100,200^A0N,50,50^FD";B;"^XZ"
400 PRINT #1 : "10.3.50.79 6101";CHR$(4); !OPENING TCP
    PORT
405 PRINT #1 : "! 0 200 200 400 1" !CPCL STARTS
410 PRINT #1 : "JOURNAL"
415 PRINT #1 : "PAGE-WIDTH 400"
420 PRINT #1 : "T 4 0 100 20 AISLE/SHELF"
500 PRINT #1 : "T 4 0 100 100 ";C$
501 PRINT #1 : "T 4 0 100 150 QUANTITY"
505 PRINT #1 : "T 4 0 100 200 ";B
510 PRINT #1 : "T 4 0 100 250 DESCRIPTION"
520 PRINT #1 : "T 4 0 100 300 ";A$
530 PRINT #1 : "PRINT" !CPCL ENDS WITH PRINT
550 PRINT #1 : CHR$(4);
600 CLOSE #1
700 CLOSE #2
```

You enter the product name, quantity , aisle/shelf location and price. The local label prints description (product name) quantity and price, while the remote printer IP address 10.3.50.79 using TCP port 6101 (mobile printer using CPCL) prints the product description, quantity and the product location (aisle/shelf).

Example • ZBI using UDP port. In this example Item,Quantity,Price PRINTER# are being sent from the printer. Log generates automatically the Month, Date, Time and IP address.

Text file from server below:

Jun	30	13:59:45	10.3.9.195	4	45	1.99	PRINTER5
Jun	30	14:00:01	10.3.9.195	3	23	8.69	PRINTER5
Jun	30	14:00:14	10.3.9.195	40	3	1.99	PRINTER5
Jun	30	14:00:24	10.3.9.195	5	65	5.99	PRINTER5

```
10  CLOSE #1
20  CLOSE #2
30  OPEN #1: NAME "ZPL"
40  OPEN #2: NAME "UDP"
45  LET C$ = "PRINTER5"
50  PRINT "ENTER PRODUCT NUMBER 1-45"
55  INPUT A
60  PRINT "ENTER QUANTITY"
65  INPUT B
70  PRINT "ENTER $ AMOUNT EXAMPLE 6.99"
75  INPUT A$
80  PRINT #2 : "10.3.50.79 514";CHR$(4);
90  PRINT #2 : A ;" ";B;" ";A$;" ";" ";" ";C$;
91  REM The extra spaces in above line are for
92  REM columns in ACCESS database where no data is sent
    or needed from the printer
95  PRINT #2: CHR$(4);
100 CLOSE #1
105 CLOSE #2
```

A

RFID Commands

This section contains the ZPL commands for RFID-specific applications.

For more information about the RFID commands, refer to the *RFID Programming Guide* (part number 58978L-001). A copy is available on the User CD provided with your printer and online at http://www.zebra.com/manuals.

RFID Command Overview

In addition to reading or encoding RFID tags, the RFID ZPL commands also provide for RFID exception handling, such as setting the number of read/write retries before declaring a transponder defective (set with ^RT and ^WT) or setting the number of labels that will be attempted if an error occurs (set with ^RS).

For example, if an RFID label fails to program correctly or if the transponder cannot be detected, the printer ejects the label and prints VOID across it. The printer will try to print another label with the same data and format for the number of RFID labels specified by the ^RS command. If the problem persists, the printer follows the error handling instructions specified by the ^RS command: the printer may remove the problematic format from the print queue and proceed with the next format (if one exists in the buffer), or it may place the printer in Pause or Error mode.



Important • Consider the following before using any command in this section:

- Each command lists the printers that support it. Before using a particular command, verify that your printer is listed.
- If a parameter in the following tables is designated as *not applicable* for a particular printer, any value entered for the parameter will be ignored, but the place holder for the field is required.

^HR

Calibrate RFID Transponder Position

Description This command initiates an RFID transponder calibration for a specific RFID label and returns the results to the host computer. This calibration is used to determine the optimal programming position for RFID media that may not meet the transponder placement specifications for the printer.

Note • You do not need to perform transponder calibration for RFID media that meets the transponder placement specifications for the printer. To order media that is designed for use with your RFID printer, contact your authorized Zebra reseller.

During transponder calibration, the printer feeds the RFID label one-dot row at a time while taking readings (via the READ TAG command and the WRITE TAG commands) to profile the RFID transponder. Based on the results, the printer determines the optimal programming position for the label and returns a results table to the host. The calibrated value is used as the programming position for the ^RS command, can be overwritten by the ^RS command, and is saved to nonvolatile memory (the value is saved even if the power is turned off).

This calibration takes into account the print mode, backfeed mode, and tear off position. The RUN option in the RFID TAG CALIB front panel parameter performs the same calibration but does not create a results table.

Important • If a label format specifies a value for parameter p (read/write position of the transponder) in the ^RS command, that value will be used for the programming position for all RFID labels until a new position is specified or until the printer is turned Off (**O**) and then back On (**I**).

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^HRa,b

This table identifies the parameters for this format.

Parameters	Details
a = start string	User text to appear before the results table. *Accepted values:* any string less than 65 characters *Default value:* start
b = end string	User text to appear after the results table. *Accepted values:* any string less than 65 characters *Default value:* end

Comments

- Based on the recommended transponder placement position for most RFID labels, the printer's default RFID programming position is zero for the R110*PAX*4. For other RFID printers, the default programming position is the label length minus 1 mm (0.04 in.). To return to the default programming position at any time, use the RESTORE option in the RFID TAG CALIB front panel parameter.
- At the end of calibration, a results table is returned to the host. Each line in the results table appears as:

  Row, Read Result, Write Result

  where

  Row = the dot row where calibration occurred

  Read Result = results of calibration (R = read, " " = unable to read)

  Write Result = results of calibration (W = write, " " = unable to write)

Example • If the following command is sent to the printer:

```
^XA^HR^XZ
```

The printer starts the transponder calibration and returns a results table such as the following:

```
start
position=195
215, ,
214, ,
213, ,
212, ,
211, ,
210, ,W
209,R,
208, ,
207, ,
206, ,W
205,R,
204, ,
203, ,
202, ,W
201,R,W
200,R,W
199,R,W
198,R,W
197,R,W
196,R,W
195,R,W <---****
194,R,W
193,R,W
192,R,W
191,R,W
190,R,W
189,R,
188, ,
187, ,
186, ,
185, ,
.
.
.
end
```

In this example, the optimal programming position is 195. This is identified at the top of the table (`position=195`) and with an the arrow (`<---****`) in the table.

^RB

Define EPC Data Structure

Description This command defines the structure of EPC data, which can be read from or written to an RFID transponder. For more information about EPC specifications, refer to the EPC Global web site.

RFID transponders can have different partitions defined. This command specifies the number of partitions and how many bits are in each partition.



Important • All parameters in this command are persistent and will be used in subsequent formats if not provided. The values are initially set to the default values.

Printers That Support This Command

- RXi
- R110*PAX*4

Format `^RBn,p0,p1,p2, ..., p15`

This table identifies the parameters for this format.

Parameters	Details
`n` = total bit size of the field	*Accepted values:* 1 to *n*, where *n* is the bit size of the tag. *Default value:* 96
`p1 ... p15` = partition sizes	Specify each partition size. These must add up to the total bit size. *Accepted values:* 1 to 64 bit total bit size *Default value:* 1



Example 1 • The following command specifies that there are 96 bits used with three fields. Fields 1, 2, and 3 contain 10, 26, and 60 bits, respectively.

```
^RB96,10,26,60
```

The ZPL code to write to a tag with this format would look like this:

```
^RFW,E^FD1000.67108000.1122921504606846976^FS
```

When writing to the tag, the data will be stored in the tag in the following way:

- Field 1 contains 1000. This value is stored in the first 10 bits
- Field 2 contains 67108000. This value is stored in the next 26 bits.
- Field 3 contains 1122921504606846976. This value is stored in the remaining 60 bits.

Example 2 • The following command specifies that there are 64 bits used with eight 8-bit fields.

```
^RB64,8,8,8,8,8,8,8,8^FS
```

The ZPL code to write to a tag with this format would look like this:

```
^RFW,E^FD1.123.160.200.249.6.1.0^FS
```

When writing to the tag, each set of data is written in its respective 8-bit field.

Example 3 • This example uses the SGTIN-64 standard, which defines 64-bit structure in the following way:

	Header	Filter Value	Company Prefix Index	Item Reference	Serial Number
SGTIN-64	2 bits	3 bits	14 bits	20 bits	25 bits
	10 (binary value)	8 (decimal capacity)	16,383 (decimal capacity)	9 to 1,048,575 (decimal capacity*)	33,554,431 (decimal capacity)

* Capacity of Item Reference field varies with the length of the company prefix.

The ZPL code to write to a tag with this format would look like this:

```
^XA
^RB64,2,3,14,20,25
^RFW,E^FD0,3,12345,544332,22335221^FS
^XZ
```

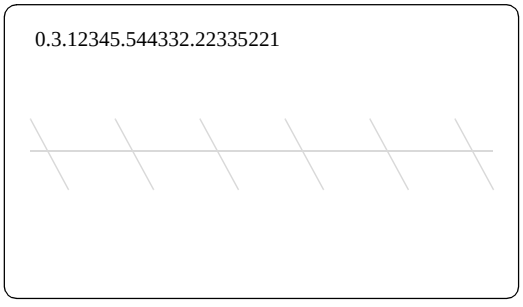
These commands would put

- 0 in the header
- 3 as the filter value
- 12345 as the company prefix
- 544332 as the item reference
- 22335221 as the serial number

To read this EPC data and print the results on the label, you would use the following code:

```
^XA
^RB64,2,3,14,20,25
^FO50,50^A0N,40^FN0^FS
^FN0^RFR,E^FS
^XZ
```

The resulting label would look like this:



^RF

Read or Write RFID Format

Description This command allows you to read or write to an RFID tag.

Note • When using this command to read a tag, you may use a field variable to print the tag data on the label or to return the data to the host.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RFo,t,b

This table identifies the parameters for this format.

Parameters	Details
o = operation	The action to be performed. *Accepted values:* • W = write to the tag • L = write with LOCK (if supported by tag type) • R = read the tag *Default value:* W
t = type	*Accepted values:* • A = ASCII • H = Hexadecimal • E = EPC (ensure proper setup with the ^RB command) *Default value:* H
b = starting block number	*Accepted values:* 0 to *n*, where *n* is the maximum number of blocks for the tag. *Default value:* 0

Examples •

Write ASCII

This example writes 96-bit data.

```
^XA
^RS4
^RFw,a^FD00 my data^FS
^XZ
```

Write Hex

This example writes 64-bit data.

```
^XA
^RS3
^RFW,H^FD1122334455667788^FS
^XZ
```

Write EPC

This example writes 96-bit data, as specified by the ^RB command.

```
^XA
^RB96,8,3,3,20,24,38
^RFw,e^FD16,3,5,78742,146165,1234567891^FS
^XZ
```

^RM

Enable RFID Motion

Description This command enables or disables RFID motion. By default, labels automatically print at the end of the format. This command allows you to inhibit the label from actually moving when it reaches the program position, which is useful for debugging, setup, and custom applications. This parameter is not persistent (carried over from label to label).

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RMe

This table identifies the parameters for this format.

Parameters	Details
e = enable	*Accepted values:* Y (Yes, move the label), N (No, do not move the label) *Default value:* Y

^RN

Detect Multiple RFID Tags in Encoding Field

Description This command enables or disables detection of multiple RFID tags in the encoding field. By default, the printer checks for more than one tag in the field before attempting to read or write. If more than one tag is found, the label over the antenna support is voided, and the `RFID ERR STATUS` parameter on the front panel displays `MULTIPLE TAGS`. To speed up printing and encoding by up to 200 ms, the check may be disabled. This parameter is persistent (carried over from label to label).

Printers That Support This Command

- R110*PAX*4

Format ^RNe

The following table identifies the parameters for this format.

Parameters	Details
e = enable	*Accepted Values:* Y (Yes, check for multiple tags), N (No, do not check for multiple tags) *Default Value:* Y

~RO

Reset Advanced Counters

Description The ~RO command resets the advanced counters used by the printer to monitor label generation in inches and centimeters, the number of labels printed, and the number of valid and voided RFID labels. Any single error during programming of an RFID tag will result in that label being considered "void" by the counter.

Four resettable counters are available. The values for the counters are displayed on the printer configuration label.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ~ROc

This table identifies the parameters for this format:

Parameters	Details
c = counter number	*Accepted Values:* • 1 (counter 1) • 2 (counter 2) • 3 (valid RFID label counter) • 4 (voided RFID label counter) *Default Value:* None. If a value is not specified, the command is ignored.

Example 1 • This example shows how the counter portion of the printer configuration labels looks when counter 1 is reset by sending ~RO1.

Before

```
296862 IN.......... NONRESET CNTR
296862 IN.......... RESET CNTR1
296862 IN.......... RESET CNTR2
753289 CM.......... NONRESET CNTR
753289 CM.......... RESET CNTR1
753289 CM.......... RESET CNTR2
92928 LABLS........ NONRESET CNTR
92928 LABLS........ RESET CNTR1
92928 LABLS........ RESET CNTR2
```

After

```
296876 IN.......... NONRESET CNTR
0 IN............... RESET CNTR1
296876 IN.......... RESET CNTR2
753323 CM.......... NONRESET CNTR
0 CM............... RESET CNTR1
753323 CM.......... RESET CNTR2
92930 LABLS........ NONRESET CNTR
0 LABLS............ RESET CNTR1
92930 LABLS........ RESET CNTR2
```

Example 2 • This example shows how the counter portion of the printer configuration labels looks when the RFID counters are reset by sending ~RO3 and ~RO4.

Before

```
02/10/05.......... RTC DATE
07:21............. RTC TIME
507............... RFID VALID CTR
4................. RFID VOID CTR
HIGH.............. RFID READ PWR
HIGH.............. RFID WRITE PWR
RFID OK........... RFID ERR STATUS
Class 0........... RFID TAG TYPE
TM: 20050201...... RFID VERSION
1950 IN........... NONRESET CNTR
1950 IN........... RESET CNTR1
1950 IN........... RESET CNTR2
```

After

```
02/10/05.......... RTC DATE
07:21............. RTC TIME
0................. RFID VALID CTR
0................. RFID VOID CTR
HIGH.............. RFID READ PWR
HIGH.............. RFID WRITE PWR
RFID OK........... RFID ERR STATUS
Class 0........... RFID TAG TYPE
TM: 20050201...... RFID VERSION
1951 IN........... NONRESET CNTR
1951 IN........... RESET CNTR1
1951 IN........... RESET CNTR2
```

^RR

Specify RFID Retries for a Block

Description This command changes the number of times that the printer attempts to read or write to a particular block of a single RFID tag. By default, the printer will attempt six retries. This command is persistent and will be used in subsequent formats if not provided.

Important • This command is not the same as the number of labels to try parameter in the ^RS command.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RRn

This table identifies the parameters for this format.

Parameters	Details
n = number of retries	*Accepted values:* 0 to 10 *Default value:* 0

Examples •

Set read block retries to 5

```
^XA
^FN1^RR5^RFR,H^FS
^HV1^FS
^XZ
```

Set write block retries to 2

```
^XA
^RR2^RFW,H^FD1234^FS
^XZ
```

^RS

RFID Setup

Description This command sets up parameters including tag type, read/write position of the transponder, and error handling.

Important • Use care when using this command in combination with ^RT or ^RFR for reading tag data. Problems can occur if the data read from the tag is going to be printed on the label. Any data read from the transponder must be positioned to be printed above the read/write position. Failure to do this will prevent read data from being printed on the label.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RSt,p,v,n,e,a,c,s

This table identifies the parameters for this format.

Parameters	Details
t = tag type	*Accepted values:* • 0 = None • 1 = EPC Class 0 • 2 = EPC Class 0 Plus • 3 = EPC Class 1 64-bit • 4 = EPC Class 1 96-bit • 5 = ISO 18000-06B *Default value:* 4 **Note •** Because some countries restrict the frequencies that can be used for RFID, not all tag types listed may be supported by your RFID printer. If you specify a tag type that is not supported, the printer will use the default value.
p = read/write position of the transponder	Sets the read/write position of the transponder in the vertical (Y axis) in dot rows from the top of the label. Set to 0 (no movement) if the transponder is already in the effective area without moving the media. **Important •** If a label format specifies a value for this parameter, this value will be used for the programming position for all labels until a new position is specified or until the printer is powered Off (**O**) and then back On (**I**). *Accepted values:* 0 to label length *Default value:* • For the R110*PAX*4: zero • For other RFID printers: label length minus 1 mm (1/16 in.)

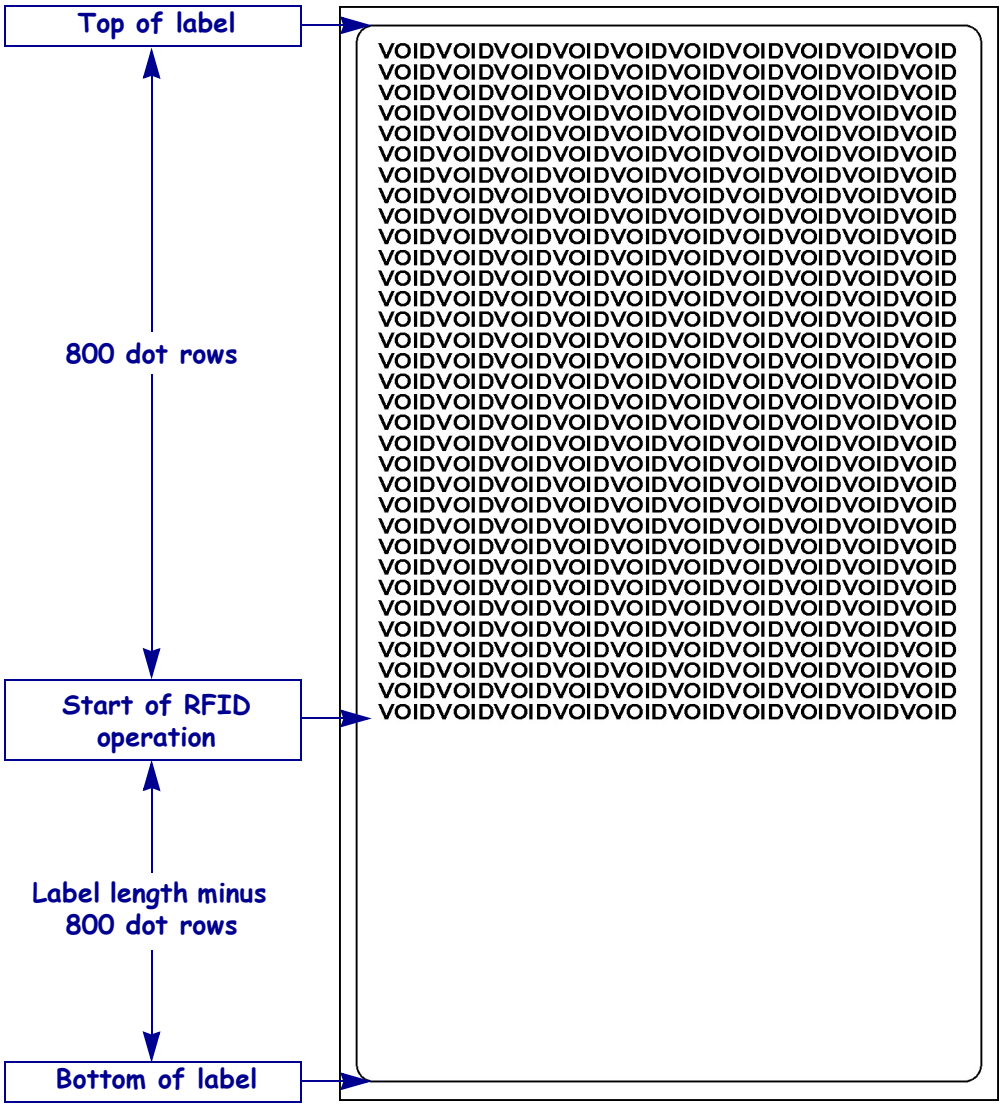
Parameters	Details
v = length of void printout	Sets the length of the void printout in vertical (Y axis) dot rows. *Accepted values:* 0 to label length *Default value:* label length
n = number of labels	The number of labels that will be attempted in case of read/encode failure. *Accepted values:* 1 to 10 *Default value:* 3
e = error handling	If an error persists after the specified number of labels are tried, perform this error handling action. *Accepted values:* • N = No action (printer drops the label format causing the error and moves to the next queued label) • P = Place printer in Pause mode (label format stays in the queue until the user cancels) • E = Place printer in Error mode (label format stays in the queue until the user cancels) *Default value:* N **Note** • You can set the printer to send an error message to the host for each failure. To enable or disable this unsolicited error message, refer to the ^SX and ^SQ ZPL commands. Use V for the condition type for an RFID error.
a = signals on applicator	When the value for parameter p (read/write position of the transponder) is nonzero, this parameter changes the number of start and stop print signals required for printing. In Single mode, one start print command is required. In Double mode, two are required, so the printer will resume printing only after the second start print command is received. *Accepted values:* • S (single) signal • D (double) signal *Default value:* S
c = certify tag with a pre-read*	* Not used in this version of the printer. The encoder does this automatically.
s = void print speed	If a label is voided, the speed at which “VOID” will be printed across the label. *Accepted values:* any valid print speed *Default value:* the printer’s maximum print speed

Example 1 • This example sets the printer to move the media to 800 dots from the top of the media [or label length minus 800 from the bottom (leading edge) of the media] and voids the rest of the media in case of an error. The printer will try to print two labels and then will pause if printing and encoding fail.

```
^XA
^RS,800,,2,P^FS
^XZ
```

Figure 1 shows the resulting voided label. Note where the void starts. The media has been moved 800 dot rows from the top of the label (label length minus 800 dot rows from the bottom (leading edge) of a label) to bring the transponder into the effective area to read/write a tag. If the printer fails the operation, the rest of the media is voided.

Figure 1 • Sample Void Label, Remainder of Label Voided

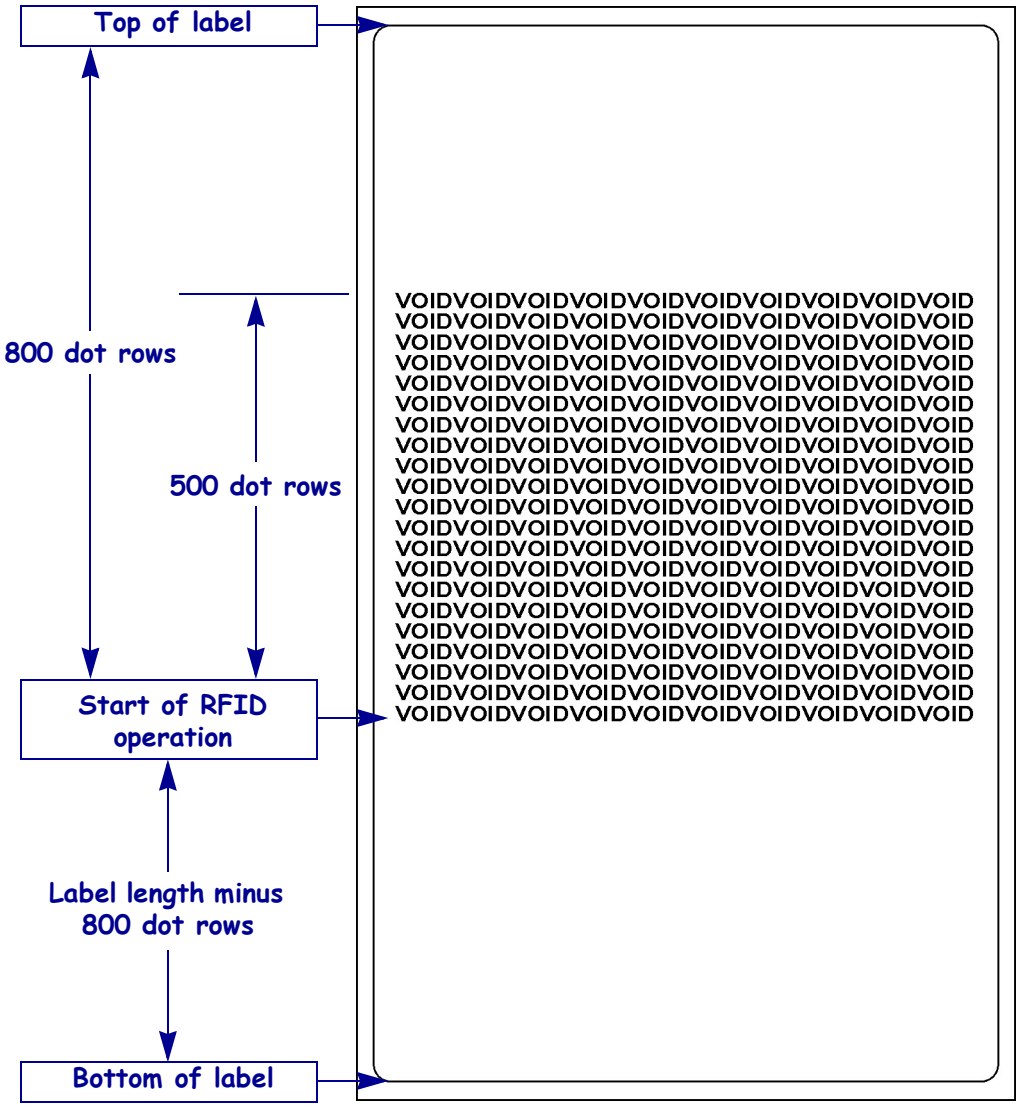


Example 2 • This example sets the printer to move the media to 800 dots from the top of the media [or label length - 500 from the bottom (leading edge) of the media] and prints “VOID” 500 dots in vertical length (Y axis) in case of an error.

```
^XA
^RS,800,500,2,P^FS
^XZ
```

Figure 2 shows the resulting voided label. Note where the void starts. The media has been moved 800 dot rows from the top of the label [label length minus 800 dot rows from the bottom (leading edge) of a label] to bring the transponder into the effective area to read/write a tag. If the printer fails the operation, an area that is 500 dot rows of the media is voided instead of the entire rest of the media.

Figure 2 • Sample Void Label, 500 Dot Row Area Voided



^RT

Read RFID Tag

Description This command tells the printer to read the current RFID tag data. The data can be sent back to the host via the ^HV command.



Note • It is recommended that you use the ^RF, ^RM, and ^RR commands rather than the ^RT command. The ^RT command is provided only for backward-compatibility with label formats that were developed for other Zebra RFID printers.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RT#,b,n,f,r,m

This table identifies the parameters for this format.

Parameters	Details
# = number to be assigned to the field	*Accepted values:* 0 to 9999 *Default value:* 0
b = starting block number	*Accepted values:* 0 to *n*, where *n* is the maximum number of blocks for the tag. *Default value:* 0
n = number of blocks to read*	*Accepted values:* 1 to *n*, where *n* is the maximum number of blocks for the tag type minus the starting block number. For example, if the tag has 8 blocks (starting with block 0) and you start with block 6, *n* can be 2. This would return block 6 and block 7 information. *Default value:* 1
f = format	*Accepted values:* • 0 (ASCII) • 1 (Hexadecimal) *Default value:* 0
r = number of retries	Changes the number of times that the printer attempts to read a particular block of a single RFID tag. (Same retry rules as the ^RR command.) *Accepted values:* 0 to 10 *Default value:* 0
m = motion	Enables or disables RFID motion for the current field. *Accepted values:* • 0 (Feed label after writing.) • 1 (No feed after writing. Other ZPL may cause a feed.) *Default value:* 0

*Not applicable for RFID printers supported by this Programming Guide Volume One.

Example • This sample reads a tag, prints the data on a label, and sends the string `Tag Data:`*xxxxxxxx* back to the host. The data read will go into the `^FN1` location of the format. The printer will retry the command five times, if necessary.

```
^XA
^FO20,120^A0N,60^FN1^FS
^RT1,,,,5^FS
^HV1,,Tag Data:^FS
^XZ
```

^RW

Set RFID Read and Write Power Levels

Description This command sets the read and write power levels. This function is useful when using different tag types or transponders that require different power levels to obtain the best read and write abilities.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RWr,w

This table identifies the parameters for this format:

Parameters	Details
r = read power	Sets the power level to match the desired output as calibrated in the factory. *Accepted Values:* H (high), M (medium), or L (low) *Default Value:* H
w = write power	Sets the power level to match the desired output as calibrated in the factory. *Accepted Values:* H (high), M (medium), or L (low) *Default Value:* H

^RZ

Set RFID Tag Password

Description This command lets you define the password for the tag during writing.



Important • Only certain tags support this feature, so check to ensure that this command can be used with your particular tag type.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^RZp

The following table identifies the parameters for this format.

Parameters	Details
p = password	*Accepted values:* 00 to FF (hexadecimal) *Default value:* 00

^WT

Write Tag

Description This command allows you to program the current RFID tag.

Note • It is recommended that you use the ^RF, ^RM, ^RR, and ^WV commands rather than the ^WT command. The ^WT command is provided only for backward-compatibility with label formats that were developed for older Zebra RFID printers.

Important • Check the amount of data memory available for the tag that you will be using. If more is sent than the memory can hold, the printer truncates the data.

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^WTb,r,m,w,f,v

This table identifies the parameters for this format.

Parameters	Details
b = block number	This parameter is tag-dependent. For most tags, use block 0. For EPC Class 0 Plus, block 0 is EPC data, and block 1 is user data. *Accepted values:* 0 to *n*, where *n* is the maximum number of blocks for the tag. *Default value:* 0
r = number of retries	Changes the number of times that the printer attempts to write to a particular block of a single RFID tag. (Same function as the ^RR command.) *Accepted values:* 0 to 10 *Default value:* 0
m = motion	Enables or disables RFID motion. (Same function as the ^RM command.) *Accepted values:* • 0 (Feed label after writing.) • 1 (No Feed after writing. Other ZPL may cause a feed.) *Default value:* 0
w = write protect	*Accepted values:* • 0 (Not write protected.) • 1 (Write protect.) *Default value:* 0

Parameters	Details
f = data format	*Accepted values:* • 0 (ASCII) • 1 (Hexadecimal) *Default value:* 0
v = verify valid data	For reliability, some manufacturers encode tags with known data (such as A5A5). This parameter flags whether the preprogrammed data is verified. (Same function as the ^WV command) *Default value:* N *Accepted values:* • N (Do not verify) • Y (Verify valid data [Hex A5A5 in the first two bytes] before writing)

Example • This sample encodes data “RFIDRFID” and will try writing up to five times, if necessary.

```
^XA
^WT,5^FDRFIDRFID^FS
^XZ
```

^WV

Verify RFID Write Operation

Description If write verify is enabled, this command verifies the RFID write operation to ensure that the tag about to be programmed contains the hex data “A5A5” in the first two bytes. This parameter is not persistent (carried over from label to label).

Printers That Support This Command

- RXi
- R110*PAX*4

Format ^WVe

This table identifies the parameters for this format.

Parameters	Details
e = enable	*Accepted values:* Y or N *Default value:* N

B

Wireless Print Server Commands

This section contains new or modified ZPL commands for the Wireless Print Server.

^KP

Define Printer Password

Description The ^KP command defines the password that must be entered to modify the printer's parameters. This is different from the wireless password.

Format ^KPa

This table identifies the parameters for this format:

Parameters	Details
a = mandatory four-digit password	*Accepted Values:* 0000 to 9999 *Default Value:* 1234

Comments If you forget the printer's password, you can return all printer parameters to the factory defaults, which resets the password to **1234**. Print a configuration label to use as a baseline before using this option because all printer parameters are set back to their defaults. Refer to the user guide for your printer for instructions for printing a configuration label.

^NB

Search for Wired Print Server during Network Boot

Description The ^NB command tells the printer to search for a wired print server at bootup.

Format ^NBa

This table identifies the parameters for this format:

Parameters	Details
a = Check for Wired Print Server at Boot Time	*Accepted Values:* C (check), S (skip check) *Default Value:* S

Table 1 shows the results of this check:

Table 1 • Results of Check for Wired Print Server

Wired Print Server Connected?	Check for Wired Print Server?	Results
Yes	Skip	The printer skips the check for a wired print server. The wired print server is not acknowledged, and the Wireless Print Server is used as the primary print server.
Yes	Check	The printer checks for a wired print server. If the wired print server is detected, it is used as the primary print server. If it is not detected, the Wireless Print Server is used as the primary print server.
No	Skip	The printer uses the Wireless Print Server as the primary print server without taking the time to check for a wired print server.
No	Check	During bootup, the printer tries for 70 seconds to detect a wired print server. If the printer cannot find one, it uses the wireless print server as the primary print server.

^NN

Set SNMP

Description The ^NN command sets the SNMP (Simple Network Management Protocol) parameters.

Format ^NNa,b,c,d,e,f

This table identifies the parameters for this format:

Important • If a parameter does not have a default value, the printer uses the previously defined value.

Parameters	Details
a = system name	Same as printer name. *Accepted Values:* Up to 17 alphanumeric characters
b = system contact	Any contact information as desired (such as a name or phrase). *Accepted Values:* Up to 50 alphanumeric characters
c = system location	The printer's model information. *Accepted Values:* Up to 50 alphanumeric characters
d = get community name	*Accepted Values:* Up to 20 alphanumeric characters *Default Value:* public
e = set community name	*Accepted Values:* Up to 20 alphanumeric characters *Default Value:* public
f = trap community name	*Accepted Values:* Up to 20 alphanumeric characters *Default Value:* public

^NP

Set Primary/Secondary Device

Description The ^NP command specifies to use the printer's or the print server's LAN/WLAN settings at boot time. The default is to use the printer's settings.

When the printer is set as the primary device, you can set it up using ZPL commands or the SetWLAN utility, and any wired print server inserted into the printer will get these settings.

Important • If you use the printer as primary, any wired print server inserted into the printer loses the original settings if the printer is set to check for the wired print server (see *^NB on page 361*) and using gleaning only for IP protocol does not work.

Format ^NPa

This table identifies the parameters for this format:

Important • If a parameter does not have a default value, the printer uses the previously defined value.

Parameters	Details
a = Device to Use as Primary	*Accepted Values:* P (printer), M (MPS/print server)

^NS

Change Wired Networking Settings

Description The ^NS command changes the wired print server network settings.

Format ^NSa,b,c,d,e,f,g,h,i

This table identifies the parameters for this format:

Important • If a parameter does not have a default value, the printer uses the previously defined value.

Parameters	Details
a = IP resolution	*Accepted Values:* A = (all) B = (BOOTP) C = (DHCP and BOOTP) D = (DHCP) G = (gleaning only) R = (RARP) P = (permanent) **Note •** Use of gleaning only is not recommended when the Wireless Print Server is installed.
b = IP address	*Accepted Values:* Any properly formatted IP address in the xxx.xxx.xxx.xxx format.
c = subnet mask	*Accepted Values:* Any properly formatted subnet mask in the xxx.xxx.xxx.xxx format.
d = default gateway	*Accepted Values:* Any properly formatted gateway in the xxx.xxx.xxx.xxx format.
e = WINS server address	*Accepted Values:* Any properly formatted WINS server in the xxx.xxx.xxx.xxx format.
f = connection timeout checking	*Accepted Values:* Y (yes), N (no)
g = timeout value	Time, in seconds, before the connection times out. *Accepted Values:* 0 through 9999
h = ARP broadcast interval	Time, in minutes, that the broadcast is sent to update the device's ARP cache.
i = base raw port number	The port number that the printer should use for its RAW data. *Accepted Values:* 0 through 99999 *Default Value:* 9100

Example • This example sets all boot protocol.

```
^XA
^NSp,192.168.0.1,255.255.255.0,192.168.0.2
```

IP Address

Subnet

Default Gateway

^NT

Set SMTP

Description The ^NT command sets the Simple Mail Transfer Protocol (SMTP) parameters, which allows you to set the e-mail settings for alerts.

Format ^NTa,b

This table identifies the parameters for this format:



Important • The parameters in this table that do not have a *Default Value* use what ever is currently defined.

Parameters	Details
a = SMTP Server Address	*Accepted Values:* Any properly formatted server address in the xxx.xxx.xxx.xxx format
b = Print Server Domain	*Accepted Values:* Any properly formatted print server domain in the xxx.xxx.xxx.xxx format

^NW

Set Web Authentication Timeout Value

Description The ^NW command sets the timeout value for the printer home page. The printer prompts for the printer password only the first time that certain screens are accessed until 1) the web authentication timeout value is reached (default value is 5 minutes) or 2) the printer is reset. At that time, the printer prompts for the password again.

Format ^NWa

The following table identifies the parameters for this format.

Parameters	Details
a = Timeout Value	The timeout value in minutes for an IP address to be authenticated to the printer web pages. *Accepted Values:* 0 (no secure pages can be accessed without entering the printer password) to 255 minutes *Default Value:* 5

^WA

Set Antenna Parameters

Description The ^WA command sets the values for the receive and transmit antenna.

Format ^WAa,b

This table identifies the parameters for this format:

Parameters	Details
a = receive antenna	*Accepted Values:* D (diversity), L (left), R (right) *Default Value:* D
b = transmit antenna	*Accepted Values:* D (diversity), L (left), R (right) *Default Value:* D

^WE

Set Wireless Encryption Values

Description The ^WE command sets WEP (wireless encryption) values.

Format ^WEa,b,c,d,e,f,g,h

Important • Be sure to include the exact number of commas required in this command when setting encryption keys (parameters e through h). A missing or extra comma in this command causes the keys to be stored in the wrong slots and can disable the wireless system.

This table identifies the parameters for this format:

Parameters	Details
a = encryption mode	*Accepted Values:* OFF, 40 (40-bit encryption), 128 (128-bit encryption) *Default Value:* OFF
b = encryption index	Tells the printer which encryption key to use. *Accepted Values:* 1 (key 1), 2 (key 2), 3 (key 3), 4 (key 4) *Default Value:* 1
c = authentication type	*Accepted Values:* O (open system), S (shared key) *Default Value:* O **Note •** If you enabled Shared Key authentication with Encryption Mode set to OFF, this value resets to Open.
d = encryption key storage	*Accepted Values:* H (hex key storage), S (string key storage) *Default Value:* S
e, f, g, h = encryption keys 1 through 4	*Accepted Values:* The actual value for the encryption key The encryption mode affects what can be entered for the encryption keys: • For 40-bit, encryption keys can be set to any 5 hex pairs or any 10 alphanumeric character. • For 128-bit, encryption keys can be set to any 13 hex pairs or any 26 alphanumeric character. **Note •** When using hex storage, do not add a leading 0x on the WEP key.

Example 1 • This example sets encryption to 40-bit, activates encryption key 1, and sets encryption key 1 to the string 12345. Encryption Key Storage parameters are left blank with commas as placeholders for the fields. The printer uses the default values for these parameters, as follow:

```
^WE40,,,,12345
```

Example 2 • .This example sets encryption to 128-bit, activates encryption key 2, and sets encryption keys 1 and 2 to hex values:

```
^WE128,2,,H,12345678901234567890123456,98765432109876543
210987654
```

The value for encryption key 1 is stored and can be activated in the future by the following command:

```
^WE128,1
```

Example 3 • This example sets encryption to 128-bit, activates encryption key 4, and sets encryption key 4 to a hex value:

```
^WE128,4,,H,,,,98765432109876543210987654
```

Values are not required for encryption keys 1 through 3 when setting encryption key 4. In this example, commas are used as placeholders for the fields for encryption keys 1 through 3. Any previously stored values for these encryption keys do not change.

Important • Make sure that you include the exact number of commas required to get to the slot for encryption key 4 (parameter `h`).

^WI

Change Wireless Network Settings

Description The ^WI command changes the wireless network settings.

Format ^WIa,b,c,d,e,f,g,h,i

This table identifies the parameters for this format:

Important • If a parameter does not have a default value, the printer uses the previously defined value.

Parameters	Details
a = IP resolution	*Accepted Values:* A = all B = BOOTP C = DHCP and BOOTP D = DHCP G = gleaning only R = RARP P = permanent **Note •** Use of gleaning only is not recommended when the Wireless Print Server is installed.
b = IP address	*Accepted Values:* Any properly formatted IP address in the xxx.xxx.xxx.xxx format.
c = subnet mask	*Accepted Values:* Any properly formatted subnet mask in the xxx.xxx.xxx.xxx format.
d = default gateway	*Accepted Values:* Any properly formatted gateway in the xxx.xxx.xxx.xxx format.
e = WINS server address	*Accepted Values:* Any properly formatted WINS server in the xxx.xxx.xxx.xxx format.
f = connection timeout checking	*Accepted Values:* Y (yes), N (no)
g = timeout value	Time, in seconds, before the connection times out. *Accepted Values:* 0 through 9999
h = ARP broadcast interval	Time, in minutes, that the broadcast is sent to update devices ARP cache.
i = base raw port number	The port number that the printer should use for its RAW data. *Accepted Values:* 0 through 99999 *Default Value:* 9100

^WL

Set LEAP Parameters

Description The ^WL command sets the LEAP mode, user name, and password.

Note • LEAP is only available with some wireless cards.

Format ^WLa,b,c

This table identifies the parameters for this format.

Parameters	Details
a = mode	*Accepted Values:* OFF, ON
b = user name	*Accepted Values:* Any 4 to 40 alphanumeric characters
c = password	*Accepted Values:* Any 4 to 40 alphanumeric characters

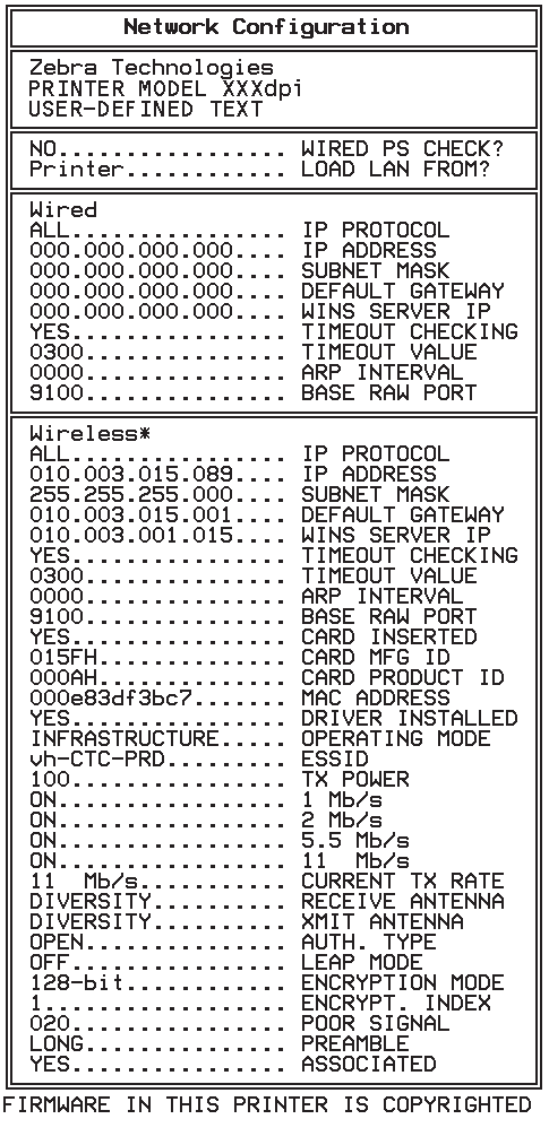
~WL

Print Network Configuration Label

Description The ~WL command generates a network configuration label (Figure 3).

Format ~WL

Figure 3 • Network Configuration Label



^WP

Set Wireless Password

Description The ^WP command sets the four-digit wireless password (not the same as the general printer password). If the wireless password is **0000**, the Wireless Print Server runs in an "unprotected" mode, which means that you do not need to enter the password through the front panel to view or modify wireless settings.

If a wireless password is set, the values for the following parameters will not appear through the front panel until the wireless password is entered:

- MAC Address
- ESSID
- Auth Type
- Leap Mode (if applicable)
- Encryption Mode
- Encryption Index
- Reset Network

Format ^WPa,b

This table identifies the parameters for this format.

Parameters	Details
a = old wireless password	*Accepted Values:* 0000 through 9999 *Default Value:* 0000
b = new wireless password	*Accepted Values:* 0000 through 9999 *Default Value:* 0000

^WR

Set Transmit Rate

Description The ^WR command changes the transmission parameters.

Format ^WRa,b,c,d,e

This table identifies the parameters for this format:

Important • If a parameter does not have a default value, the printer uses the previously defined value.

Parameters	Details
a = rate 1	Sets the 1 Mb/s transmit rate. *Accepted Values:* Y (On), N (Off)
b = rate 2	Sets the 2 Mb/s transmit rate. *Accepted Values:* Y (On), N (Off)
c = rate 5.5	Sets the 5.5 Mb/s transmit rate. *Accepted Values:* Y (On), N (Off)
d = rate 11	Sets the 11 Mb/s transmit rate. *Accepted Values:* Y (On), N (Off)
e = transmit power	*Accepted Values:* 1, 5, 20, 30, 50, 100

~WR

Reset Wireless Card

Description The ~WR command reinitializes the wireless card and the print server when the Wireless Print Server is running.

Format ~WR

^WS

Set Wireless Card Values

Description The ^WS command sets the wireless card values for ESSID, Operating Mode, and Card Preamble.

Format ^WSe,o,p

This table identifies the parameters for this format:

Parameters	Details
e = ESSID value	*Accepted Values:* Any value up to 32 characters, including all ASCII and Extended ASCII characters, including the space character. When this parameter is left blank, the ESSID is not changed. *Default Value:* 125
o = operating mode	*Accepted Values:* I (infrastructure), A (adhoc) *Default Value:* I
p = wireless card preamble	*Accepted Values:* L (long), S (short) *Default Value:* L



Notes •

Index

A

abort download graphic, 127
advanced counter
  reset, 250
advanced counter reset, 345
alphanumeric default font
  change, 108
antenna parameter setting via ZPL, 368
applicator reprint, 247
authentication timeout for printer home page, 367
authentication type
  setting via ZPL command, 369
auxiliary port
  set, 192, , 194

B

backfeed sequence
  change, 203, , 204
bar code field default, 101
battery
  set condition, 190
battery status, 160
bitmap font
  download, 121
box, 151

C

cache on, 114
calibration
  RFID tag using ZPL (^HR), 335
cancel all, 184
cancel format, 200
caret
  change, 105, , 106
change alphanumeric default font, 108
change backfeed sequence, 203, , 204
change caret, 105, , 106
change delimiter, 107
change international font, 110
change memory letter designation, 113
change network settings, 236
change tilde, 116
change wired network settings, 364
circle, 153
CODABLOCK, 43
  considerations for ^FD character set, 45
  considerations for the ^BY, 45
code 11, 17
code 128
  subsets, 49
  subsets A and C, 52
  subsets a, b, and c, 47
code 39, 21
code 49, 25
  automatic mode, 27
  field data character set, 27
code 93, 39
  full ASCII mode, 40
code validation, 117
comment, 150
communications diagnostics, 188
  enable, 188
configuation
  update, 207

configuration
using ZPL commands, 359
configuration label
print, 274
contact information, xxi
counter reset (~RO), 345
current partially input format
cancel, 209
currently connected printer
set transparent, 237
customer support, xxi

D

darkness
set, 252
data matrix, 97
date for real time clock
set, 266
define EPC data structure (^RB), 338
define language, 213
define password, 216
define printer name, 214
define printer password via ZPL, 360
delete object, 177
delimiter
change, 107
description information
display, 175
detect multiple RFID tags (^RN), 344
diagnostics
disable, 189
diagonal line, 154
directory label
print, 275
disable diagnostics, 189
discharge mode
battery, 211
display description information, 175
download bitmap font, 121
download encoding, 123
download format, 124
download graphic
abort, 127
download graphics, 125, , 131
download scalable font, 128
download true type font, 129
download unbounded true type font, 130

E

EAN-13, 63
EAN-8, 35
Electronic Product Code (EPC)
define EPC data structure (^RB), 338
ellipse, 155
enable RFID motion (^RM), 343
encoding
download, 123
select, 253
encryption
set LEAP parameters, 372
set values via ZPL, 369
end format, 281
erase download graphics, 133
erase stored formats, 132
ESSID
setting via ZPL, 377

F

feedback
suppress, 278
field
field reverse, 144
orientation, 149
parameter, 143
separator, 145
typeset, 146
variable, 148
field block, 133, , 134
field clock
real time clock, 136
field data, 137
field hexadecimal indicator, 138
field number, 141
field orientation, 149
field origin, 142
field parameter, 143
field reverse print, 144
field separator, 145
field typeset, 146
field variable, 148
flash memory
initialize, 185
font identifier, 119
font name
to call font, 15
fonts
p-v, 14

format
cancel, 200
download, 124
end, 281
pause, 200
recall, 279
set, 277
formats
erase stored, 132

G

graphic
box, 151
circle, 153
diagonal line, 154
ellipse, 155
field, 156
recall, 280
symbol, 158
graphic field, 156
graphics
download, 125, , 131
erase download, 133
upload, 174
graphing sensor calibration, 191

H

head test
fatal, 198
interval, 206
non-fatal, 199
head test fatal, 198
head test interval, 206
head test non-fatal, 199
host
directory list, 172
graphic, 163
identification, 165
RAM status, 166
status return, 167
host directory list, 172
host graphic, 163
host identification, 165
host RAM status, 166
host status return, 167
host verification command (^HV), 171

I

image
load, 179
move, 181
save, 182
image load, 179
image move, 181
image save, 182
industrial 2 of 5, 67
initialize Flash memory, 185
interleaved
2 of 5, 19
international font
change, 110

K

kill battery, 211

L

label
maximum length, 225
reverse print, 219
shift, 220
top, 221
label home, 217
label length, 218
set, 196
language
define, 213
LEAP mode
setting using ZPL, 372
liability, xvii
LOGMARS, 73

M

map clear, 222
maximum label length, 225
media
darkness, 223
feed, 224
tracking, 227
type, 229
media darkness, 223
media sensor
set, 264
media sensor calibration, 187
set, 187
media tracking, 227
media type, 229

memory letter designation
change, 113
mirror image
printing, 240
mode protection, 228
modify head warning, 232
motion in RFID label, 343
MSI, 75
multiple field origin locations, 139

N

network
change settings, 236
connect, 233
ID number, 234
network boot command, 361
network configuration label
print via ZPL, 373
network connect, 233
network ID number, 234
network printers
set all transparent, 235
network settings
setting via ZPL, 371
number of retries for block (^RR), 347

O

object delete, 177
offset for real time clock
set, 259
optional memory
reset, 186

P

password
define, 216
set printer password, 360
set wireless password, 374
passwords
RFID tag password, 355
pause
programmable, 242
pause format, 200
PDF417, 30
consideration for ^FD, 33
POSTNET, 103
power on
reset, 202
primary/secondary device
setting via ZPL, 363
print
start, 248
width, 249
print mode, 226
print network configuration label
via ZPL command, 373
print orientation, 241
print quantity, 243
print rate, 244
print start, 248
print width, 249
printer
sleep, 282
printer name
define, 214
printer password setting, 360
printer sleep, 282
printer web pages
set timeout value, 367
printhead resistance
set, 263
printing mirror image of label, 240
programmable pause, 242
proprietary statement, xvii

Q

QR code
normal mode, 83
quantity
print, 243

R

read power
change using ZPL, 354
read RFID tag
read or write RFID format (^RF), 341
read RFID tag (^RT), 352
real time clock
set language, 256
set mode, 256
real time clock date format
select, 212
real time clock time format
select, 212
recall format, 279
recall graphic, 280
related documents, xxiii
reprint
after error, 210
applicator, 247

reset
  power on, 202
reset advanced counter, 250, , 345
reset advanced counters (~RO), 345
reset optional memory, 186
reset wireless card, 376
return data to host computer (^HV), 171
RFID
  calibrate RFID tag using ZPL (^HR), 335
  change read power using ZPL, 354
  change write power using ZPL, 354
  detect multiple tags, 344
  enable motion, 343
  number of retries for block (^RR), 347
  RFID setup command (^RS), 348
  verify write operation (^WV), 358
ribbon tension
  set, 208

S

scalable font, 12
  download, 128
select encoding, 253
sensor calibration
  graphing, 191
serial communications
  set, 251
serialization data, 257
serialization field
  standard ^FD string, 254
set all network printers transparent, 235
set auxiliary port, 194
set battery condition, 190
set darkness, 252
set dots
  millimeter, 197
set dots per millimeter, 197
set label length, 196
set RFID tag password (^RZ), 355
set serial communications, 251
set units of measurements, 230
SGTIN-64 standard programming example, 340
slew
  home position, 239
slew given number
  dot rows, 238
slew to home position, 239
SMTP parameters
  setting via ZPL, 366
SNMP parameters
  setting via ZPL, 362
specify number of retries for block (^RR), 347
start print, 260
start ZBI, 192
support, xxi
symbol, 158, , 162

T

tear-off adjust position, 271
terminate ZBI, 201
tilde
  change, 116
time for real time clock
  set, 266
timeout value for printer home page, 367
transfer object, 272
transmit rate setting via ZPL, 375
transponders
  calibration using ZPL (^HR), 335
true type font
  download, 129

U

unbounded true type font
  download, 130
units of measurement
  set, 230
UPC/EAN extensions, 90
UPC-A, 95
UPC-E, 37
update configuration, 207
upload graphics, 174
UPS maxicode, 60
  considerations for ^FD, 61
use font name to call font, 15

V

verify RFID write operation (^WV), 358

W

web authentication timeout value, 367
width
  print, 249
wired print server
  ^NB to set check for, 361
  change network settings, 364
wireless card
  reset via ZPL, 376
  setting values via ZPL, 377

wireless password
  setting via ZPL, 374
Wireless Print Server
  change network settings, 371
  ZPL commands, 359
write power
  change using ZPL, 354
write RFID format (^RF), 341
write RFID tag (^WT), 356

Z

ZBI
  start, 192
  terminate, 201
Zebra support, xxi
ZebraNet Alert
  halt, 262
  set, 267
ZPL
  set, 270
ZPL commands
  ^B7, 30
  Wireless Print Server commands, 359



Zebra Technologies Corporation
333 Corporate Woods Parkway
Vernon Hills, Illinois 60061.3109 U.S.A.
Telephone: +1 847.634.6700
Facsimile: +1 847.913.8766

Zebra Technologies Europe Limited
Zebra House
The Valley Centre, Gordon Road
High Wycombe
Buckinghamshire HP13 6EQ, UK
Telephone: +44 (0) 1494 472872
Facsimile: +44 (0) 1494 450103

Customer Order # 45541L
Manufacturer Part # 45541L-001 Rev. A

© 2005 ZIH Corp.